



MANUEL DE REFERENCE DU LANGAGE DE MODELISATION FIGARO

Marc Bouissou, Sybille Humbert, Jean-Christophe Houdebine

Juin 2016

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Synthèse

Ce document est destiné aux personnes cherchant à se familiariser avec le langage Figaro et ses possibilités (chapitres 1 et 2) et aux personnes devant écrire ou modifier des bases de connaissances. Il illustre par de nombreux exemples la syntaxe du langage, définie formellement dans le document associé « Syntaxe du langage de modélisation Figaro » et la sémantique associée. Il donne également une définition formelle de la sémantique du langage Figaro d'ordre 0 qui est le point de départ de tous les traitements.

Une fois que le lecteur aura assimilé le contenu du présent document il pourra utiliser seulement le document plus court « Syntaxe du langage de modélisation Figaro ».

- Le premier chapitre décrit les objectifs du langage et explique pourquoi ce langage a été développé.
- Le deuxième chapitre décrit les principes de modélisation des systèmes en Figaro. Il décrit notamment le principe d'exploitation des règles d'un modèle Figaro en chaînage avant et en chaînage arrière.
- Le troisième chapitre donne une définition formelle de la sémantique du langage, grâce à quelques notions mathématiques simples.
- Le quatrième chapitre décrit les éléments lexicaux du langage.
- Le cinquième chapitre décrit principalement la syntaxe des expressions utilisables en Figaro.
- Le sixième chapitre décrit de façon détaillée la syntaxe de définition des classes d'objets Figaro.
- Le septième chapitre décrit les possibilités du langage d'abord plus complexe.
- Le huitième chapitre s'intéresse à la détection automatique des incohérences dans les modèles et donne des conseils pour écrire des bases de connaissances dont la cohérence est assurée par construction.
- Enfin, en annexe se trouvent la liste des évolutions du langage et un index des principales notions manipulées dans ce manuel.

Ce document constitue la quatrième version du manuel de référence du langage Figaro. Les modifications apportées à la précédente version comprennent entre autres :

- d'une part, les changements dus aux évolutions du langage nécessitées par l'application KB3 avec l'introduction de nouvelles informations associées aux pannes, et la notion nouvelle d'objet-système qui a permis de remplacer ce qui était écrit dans un pseudo-type appelé GLOBAL. En outre, des fonctions exploitables uniquement par simulation de Monte Carlo ont pu être ajoutées grâce à la création d'un nouvel outil de quantification reposant sur cette méthode, appelé YAMS.
- d'autre part l'ajout d'un chapitre donnant une définition formelle de la sémantique du langage et d'un autre décrivant divers mécanismes permettant d'éviter ou bien de détecter des incohérences dans les modèles. Ces chapitres reprennent la teneur de deux articles publiés au congrès ESREL en 2002.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Sommaire

SYNTHESE	2
SOMMAIRE.....	3
PRELIMINAIRES	6
1. INTRODUCTION	7
1.1. CONTEXTE ET HISTORIQUE	7
1.1.1. <i>Qu'est-ce qu'une étude de sûreté de fonctionnement ?</i>	7
1.1.2. <i>Vers l'automatisation</i>	7
1.1.3. <i>Démarche pour l'automatisation</i>	8
1.2. POURQUOI UN LANGAGE DE MODELISATION SPECIFIQUE ?	9
2. PRINCIPES DE MODELISATION EN FIGARO	10
2.1. LES CLASSES ET LES OBJETS	10
2.1.1. <i>Définitions</i>	10
2.1.2. <i>Structure d'un modèle Figaro</i>	11
2.2. MODELES D'ORDRE 1 ET D'ORDRE 0	11
2.3. L'HERITAGE ENTRE CLASSES	12
2.4. COMMUNICATION ENTRE LES OBJETS	13
2.4.1. <i>Champs interface d'un type</i>	14
2.4.2. <i>Champs interface d'un objet</i>	15
2.4.3. <i>Relations et ensembles</i>	15
2.4.3.1. <i>Identifiant d'une relation</i>	15
2.4.3.2. <i>Cardinalité</i>	15
2.4.4. <i>Utilisation des champs d'interface</i>	16
2.4.5. <i>Modélisation des liens matériels entre objets</i>	16
2.5. CHAMPS DE CLASSES ET D'OBJETS	16
2.6. LIEN ENTRE LES CONCEPTS FIGARO ET UML	17
2.7. REGLES FIGARO	17
2.7.1. <i>Principe de modélisation des systèmes</i>	17
2.7.2. <i>Formalisme d'une règle Figaro</i>	19
2.7.2.1. <i>Forme d'une règle d'occurrence</i>	19
2.7.2.2. <i>Forme d'une règle d'interaction</i>	19
2.7.2.3. <i>Condition et actions d'une règle, utilisation des champs d'interface</i>	20
2.7.3. <i>Principes d'utilisation des règles</i>	22
2.7.3.1. <i>Application en chaînage avant</i>	22
2.7.3.2. <i>Application en chaînage arrière</i>	26
3. DEFINITION FORMELLE DU LANGAGE FIGARO 0.....	28
3.1. ELEMENTS DU MODELE.....	28
3.2. INFERENCE REALISEE DANS LES REGLES D'INTERACTION	29
3.3. SEMANTIQUE DU MODELE COMPLET	29
4. LES ELEMENTS LEXICAUX DU LANGAGE FIGARO	30
4.1. L'ALPHABET	30
4.2. LES VALEURS NUMERIQUES	31
4.3. LES IDENTIFICATEURS	31
4.4. LES MOTS-CLES.....	32
4.4.1. <i>Les opérateurs</i>	32
4.4.2. <i>Les mots-clés</i>	32
5. LES EXPRESSIONS.....	33
5.1. PRELIMINAIRES	33
5.1.1. <i>Champs valués et variables globales</i>	33
5.1.2. <i>Définir un ensemble</i>	33
5.1.2.1. <i>Notions d'ensembles et de variables liées</i>	33
5.1.2.2. <i>Utilisation d'un champ d'interface pour définir un ensemble</i>	34
5.1.2.3. <i>Utilisation d'un type pour définir un ensemble</i>	35
5.1.2.4. <i>Limite de Figaro</i>	36

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

5.1.2.5.	Restriction d'un ensemble.....	36
5.1.3.	Accès à un champ de type ou d'objet.....	37
5.1.3.1.	Accès aux champs du type décrit.....	37
5.1.3.2.	Accès aux champs d'une variable liée à un ensemble d'objets.....	37
5.1.3.3.	Accès au champ d'un objet défini par un ensemble de cardinal 1.....	38
5.1.3.4.	Accès aux champs d'un objet nommé.....	38
5.1.3.1.	Accès aux variables globales.....	38
5.2.	EXPRESSIONS SIMPLES.....	39
5.2.1.	Opérations arithmétiques.....	39
5.2.2.	Opérations booléennes.....	40
5.2.3.	Priorités entre les opérateurs.....	41
5.3.	LES QUANTIFICATEURS.....	42
5.3.1.	Le quantificateur IL_EXISTE.....	42
5.3.2.	Le quantificateur QSOIT.....	43
5.3.3.	Le quantificateur IL_EXISTE AU_MOINS.....	44
5.3.4.	Le quantificateur POUR_TOUT.....	45
5.3.5.	Le quantificateur SOIT.....	45
5.3.6.	Combinaison de quantificateurs.....	45
5.4.	LES OPERATEURS BOOLEENS.....	45
5.4.1.	L'opérateur INCLUS DANS.....	46
5.4.2.	L'opérateur MARCHE.....	46
5.4.3.	L'opérateur PANNE.....	47
5.4.4.	L'opérateur AU_MOINS ... PARMI.....	47
5.5.	OPERATEURS NUMERIQUES COMPLEXES.....	48
5.5.1.	L'opérateur CARDINAL.....	48
5.5.2.	L'opérateur SOMME.....	48
5.5.3.	L'opérateur PRODUIT.....	49
5.6.	OPERATEURS SPECIFIQUES.....	49
5.7.	PORTEE D'UNE VARIABLE.....	50
6.	DESCRIPTION D'UN TYPE FIGARO.....	50
6.1.	STRUCTURE GENERALE D'UNE DEFINITION DE CLASSE.....	51
6.1.1.	Déclaration d'un TYPE.....	51
6.1.2.	Structure générale de description d'un TYPE.....	51
6.2.	LES CHAMPS VALUES.....	52
6.2.1.	Préliminaires.....	52
6.2.1.1.	Etat initial d'un modèle Figaro.....	52
6.2.1.2.	Valeur par défaut.....	53
6.2.1.3.	Notion d'édition.....	54
6.2.1.4.	Remarque sur le calcul de l'état initial d'un modèle Figaro.....	55
6.2.2.	INTERFACE.....	55
6.2.3.	Les constantes.....	56
6.2.3.1.	CONSTANTE.....	56
6.2.3.2.	PARAMETRE_LOI.....	57
6.2.4.	Les variables d'état.....	58
6.2.4.1.	Notion de variable réinitialisée.....	58
6.2.4.1.1.	Variables non réinitialisées.....	58
6.2.4.1.2.	Variables réinitialisées.....	58
6.2.4.2.	Catégories de variables d'état en Figaro.....	59
6.2.4.3.	ATTRIBUT.....	59
6.2.4.4.	PANNE.....	60
6.2.4.5.	EFFET.....	61
6.3.	LES REGLES.....	61
6.3.1.	Les conditions.....	62
6.3.2.	Les actions.....	62
6.3.2.1.	Affectation d'une valeur à une variable.....	63
6.3.2.2.	Action sur un ensemble d'objets.....	63
6.3.2.3.	Résolution de système d'équations.....	64
6.3.3.	Les règles d'occurrence.....	64
6.3.3.1.	Syntaxe d'une règle d'occurrence.....	67
6.3.3.2.	Transitions normales.....	70
6.3.3.3.	Défaillances.....	71
6.3.3.4.	Indisponibilités.....	73

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

6.3.3.5.	Réparations	73
6.3.4.	Les règles d'interaction	73
7.	NOTIONS AVANCEES	76
7.1.	LA PRECOMPILATION ET LES MACROS.....	76
7.2.	ELEMENTS GLOBAUX.....	78
7.3.	HERITAGE ET SURCHARGE	78
7.4.	LES GROUPES DE REGLES	79
7.5.	LES ETAPES.....	80
7.5.1.	Déclaration des étapes	81
7.5.2.	Regroupement des règles par étapes.....	82
7.5.3.	Application des règles d'interaction en chaînage avant	82
7.5.4.	Application des règles d'interaction en chaînage arrière	84
7.6.	LES EQUATIONS.....	84
7.6.1.	Systèmes d'équations.....	84
7.6.2.	Equations.....	84
8.	COHERENCE DES MODELES.....	85
8.1.	RELATION ENTRE INCOHERENCE A L'ORDRE 1 ET A L'ORDRE 0	86
8.1.1.	Contrôles syntaxiques	86
8.1.2.	Cohérence du comportement	86
8.2.	TYPLOGIE DES INCOHERENCES POSSIBLES	87
8.3.	GRAPHE DE DEPENDANCES ENTRE VARIABLES	88
8.4.	METHODES SURES D'ECRITURE DE BASES DE CONNAISSANCES	89
8.4.1.	Avoir un graphe de dépendances pyramidal	89
8.4.2.	Raisonner en inférence monotone.....	90
8.4.3.	Bien exploiter les étapes.....	91
8.5.	SEQUENCEMENT AUTOMATIQUE DES REGLES D'INTERACTION	91
8.5.1.	Définition des interdépendances entre règles.....	91
8.5.2.	Ordonnement des règles	92
8.5.3.	Utilisation de l'outil de séquenement des règles	92
8.6.	DETECTION D'INCOHERENCES SUR UN MODELE FIGARO 0	92
8.7.	CONCLUSION SUR LA COHERENCE DES MODELES	93
9.	REFERENCES	95
10.	ANNEXE 1 : EVOLUTIONS DU LANGAGE FIGARO	96
10.1.	AJOUTS POUR LA SIMULATION DE MONTE CARLO	96
10.2.	AJOUTS POUR LA GENERATION D'ARBRES DE DEFAILLANCES.....	96
10.1.	REMPLACEMENT DU PSEUDO-TYPE GLOBAL PAR LES OBJETS SYSTEME	96
11.	ANNEXE 2 : INDEX.....	97

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Préliminaires

Ce document est destiné aux personnes cherchant à se familiariser avec le langage Figaro et ses possibilités (chapitres 1 et 2) et aux personnes devant écrire ou modifier des bases de connaissances. Il décrit de façon détaillée la syntaxe du langage et la sémantique associée.

La seconde version du manuel (indice B) intégrait le retour d'expérience principalement issu de projets opérationnels, KB3 et TOPASE, et de l'enseignement de Figaro à l'Université de Rouen. Par ailleurs, elle tenait compte de la clarification de certains concepts du langage Figaro faite suite à la réécriture en C++ des traitements d'instanciation et de vérification de bases de connaissances.

La troisième version du manuel (indice C) décrivait le langage tel qu'il a été stabilisé lors du développement de la version 3 de KB3, avant 2005.

Cette quatrième version (indice D) est relative à l'état du langage en 2016 ; elle a fait l'objet de remaniements et ajouts importants par rapport à l'indice C (cf. la synthèse).

Conventions utilisées dans ce manuel

Ce manuel contient des descriptions de syntaxe pour les différentes expressions du langage Figaro. Cette notation utilise les méta-caractères suivants :

Méta-caractère	Description	Exemple
[]	Signifie que l'information entre les crochets est optionnelle.	[DE_TYPE type]
	Est utilisé pour exprimer une alternative.	UN UNE

Ce manuel utilise les conventions typographiques suivantes :

Machine à écrire

Indique que les lignes correspondantes font partie d'un programme Figaro.

Machine à écrire gras

Permet de faire la différence entre des identificateurs de l'utilisateur et les mots clés du langage dans les exemples. Les mots-clés seront toujours en gras.

Italique

Indique que le mot fait référence à un champ d'objet ou à un type d'un exemple.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

1. Introduction

Le langage Figaro est un langage spécialisé (Domain Specific Language) créé en 1989 permettant de modéliser les systèmes pour réaliser des études de sûreté de fonctionnement et, par extension, des calculs probabilistes sur des modèles stochastiques discrets à temps continu. Le but de ce chapitre est de présenter rapidement le domaine d'application du langage et ses principaux objectifs.

1.1. Contexte et historique

1.1.1. Qu'est-ce qu'une étude de sûreté de fonctionnement ?

L'objectif d'une étude de sûreté de fonctionnement d'un système est d'évaluer la fiabilité, la disponibilité ou la productivité d'un système [1]. De façon classique, une telle étude se déroule en 3 phases.

Dans une première phase, le fiabiliste en charge de l'étude doit **modéliser le système** à étudier, c'est-à-dire élaborer une représentation du système qui permette de répondre aux objectifs de l'étude. L'activité de modélisation consiste principalement à décrire le système (topologie, composants, limites du système) et à formaliser les hypothèses qui sont utilisées pour l'étude (à quel niveau de détail sont décrits les composants, quels sont les modes de défaillance retenus pour chaque composant, quelle modélisation est adoptée pour décrire des comportements comme la propagation de fluide ou le flux électrique).

Dans une deuxième phase, le fiabiliste **construit les modèles de fiabilité** appropriés à l'étude du système, en utilisant le modèle et les hypothèses élaborés dans la première phase. Il existe plusieurs types de modèles de fiabilité utilisables [1] (Arbre de défaillances, Graphe de Markov, Réseau de Petri, BDMP ...). Chacun de ces modèles permet de réaliser un certain type d'analyse. Par exemple :

- Une modélisation par Arbre de défaillances permet de représenter de façon très détaillée la logique d'un système, mais ne peut pas prendre en compte les dépendances entre composants ;
- Une modélisation par Graphe de Markov permet de prendre en compte les dépendances entre composants, mais est limitée par l'explosion combinatoire du nombre d'états à considérer.

Chaque type de modèle a donc ses limites et est associé à un certain nombre d'hypothèses à respecter. Par exemple, une modélisation par Arbre de défaillances suppose l'indépendance des composants du système. Le choix du type de modèle de fiabilité utilisé pour l'étude d'un système particulier est de la responsabilité du fiabiliste en charge de l'étude.

Enfin, dans une dernière phase, les modèles de fiabilité construits sont **quantifiés par des codes de calcul** appropriés afin d'obtenir les résultats attendus concernant la fiabilité, disponibilité ou productivité du système.

La réalisation d'une étude de sûreté de fonctionnement d'un système est donc une activité pluridisciplinaire, qui fait à la fois appel à des connaissances sur le fonctionnement du système et à des connaissances fiabilistes sur la modélisation des systèmes et le contenu mathématique des méthodes de traitement des modèles.

De façon classique, les activités de modélisation du système et d'élaboration des modèles de fiabilité sont réalisées "manuellement" par le fiabiliste. Les résultats de l'étude sont consignés dans un document décrivant :

- la modélisation du système réalisée (qui est décrite de façon plus ou moins détaillée) ;
- les modèles de fiabilité construits ;
- les résultats obtenus.

1.1.2. Vers l'automatisation

Depuis 1986, EDF a dû mener à bien plusieurs Etudes Probabilistes de Sûreté (EPS) de centrales nucléaires, chaque EPS nécessitant la réalisation d'un grand nombre d'études de sûreté de fonctionnement de systèmes élémentaires. Par ailleurs, depuis 1995, des demandes émergent à EDF pour introduire les techniques de sûreté de fonctionnement dans le processus de conception des systèmes.

Les travaux réalisés dans le cadre de ces projets ont montré les limites de l'approche manuelle classique pour réaliser les études de sûreté de fonctionnement des systèmes.

Manque de traçabilité et de transparence des études

Le raisonnement mené par le fiabiliste pour construire un modèle de fiabilité contient souvent beaucoup

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

d'implicite. Ce manque de transparence entraîne deux types de problèmes.

- Difficulté pour mettre à jour les études
Il est souvent difficile, pour celui qui n'a pas réalisé l'étude, de se réapproprier les modèles de fiabilité pour les faire évoluer. Ce problème se pose notamment dans le cadre des EPS pour lesquelles les modèles construits doivent évoluer dans le temps.
- Difficulté pour diffuser les études à des non fiabilistes
Comme nous l'avons déjà souligné, la réalisation d'une étude de fiabilité fait appel à des connaissances différentes, ce qui nécessite de faire appel à des interlocuteurs différents (concepteurs du système, exploitants). Il est important que ces interlocuteurs non fiabilistes puissent comprendre et valider les hypothèses et résultats sans pour autant comprendre le détail des modèles de fiabilité construits.

Difficulté pour garantir l'homogénéité des études au sein d'un même projet

Pour mener à bien des études probabilistes de grands systèmes complexes (EPS par exemple), il est nécessaire de réaliser et d'intégrer un grand nombre d'études de systèmes élémentaires. Ces études sont le plus souvent prises en charge par différents intervenants et il est difficile de garantir leur cohérence et leur homogénéité (choix du niveau de détail, méthode de prise en compte des modes de défaillances, ...).

Délais nécessaires aux études

La réalisation d'une étude de système "à la main" est un travail important, qui nécessite des délais conséquents. Ces délais rendent difficile l'intégration de telles études à des processus de conception de systèmes (processus itératif nécessitant de comparer "rapidement" des architectures de systèmes).

Ainsi, pour **pallier les limites des études de sûreté de fonctionnement manuelles**, des recherches ont été menées à EDF dès 1986 pour automatiser ces études. L'objectif de ces recherches était de mettre en place des outils permettant :

- de **modéliser** simplement les systèmes,
- de **formaliser, tracer et automatiser** le raisonnement du fiabiliste pour construire les modèles de fiabilité à partir de la modélisation d'un système particulier,

1.1.3. Démarche pour l'automatisation

Lorsqu'on réalise l'étude de sûreté de fonctionnement d'un système, on a besoin de collecter un certain nombre d'informations sur chaque composant du système. Ainsi, pour une pompe motorisée, on se demandera, par exemple, quel est le tableau électrique d'alimentation de la pompe, quel est l'état initial de la pompe (marche ou arrêt), etc.

Pour modéliser un composant, on lui associe donc (implicitement ou non) un certain nombre de propriétés. Il est clair que ces propriétés sont le plus souvent **communes à tous les composants d'un même type**. (Toutes les pompes motorisées des systèmes thermohydrauliques possèdent un tableau électrique d'alimentation et un état initial).

Ainsi, toutes les études de sûreté de fonctionnement relatives à une même classe de systèmes (systèmes thermohydrauliques, systèmes électriques, ...) peuvent utiliser un même **modèle générique** pour décrire les systèmes.

Il apparaît dès lors naturel, pour automatiser les études de sûreté de fonctionnement, de capitaliser les modèles génériques de composants, dans des **bases de connaissances** (cf. Figure 1). Par exemple, une base de connaissances dédiée aux systèmes thermohydrauliques contient la description des composants de type pompe, vanne, clapet, etc.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

*description de
classes de
composants*

*- description du
système
- configuration*

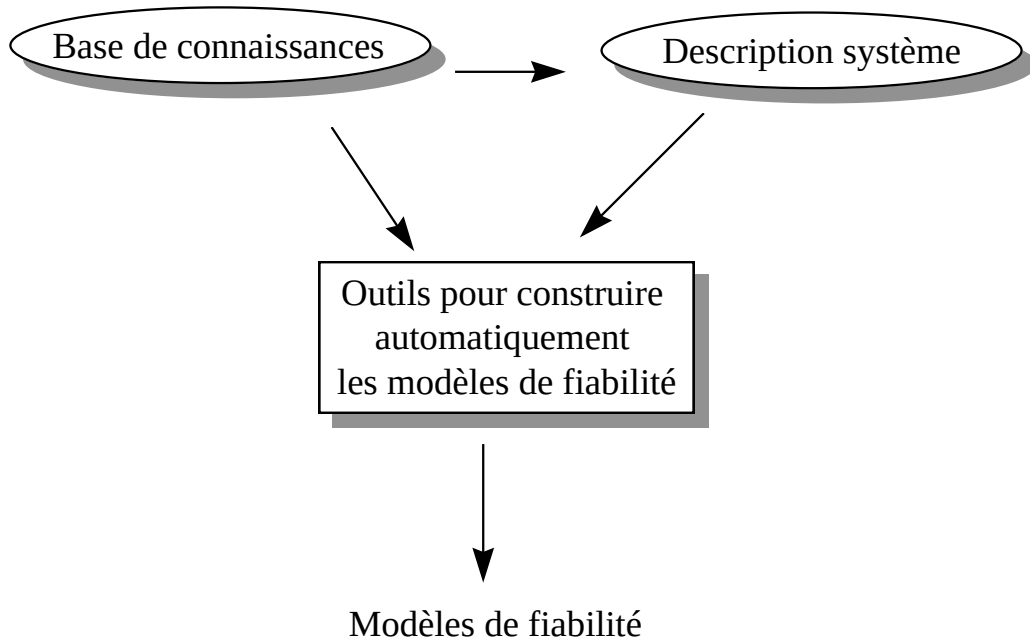


Figure 1. Démarche pour l'automatisation

Avec cette approche, la **description du système** à étudier (modélisation de la topologie du système, des composants contenus dans le système), est réalisée en s'appuyant sur les modèles génériques des composants de la base de connaissances, qu'il suffit d'instancier en fonction de leur emplacement dans la topologie du système et de leurs caractéristiques propres.

On génère ensuite automatiquement les modèles de fiabilité requis, en utilisant le **modèle du système**, constitué de la **base de connaissances** et de la **description du système** à étudier.

Remarque : La connaissance représentée dans une base de connaissances dépend bien sûr de la classe des systèmes à étudier mais aussi, le plus souvent, du type d'étude de sûreté de fonctionnement que l'on cherche à automatiser (du type de modèle de fiabilité que l'on veut générer automatiquement avec la base de connaissances).

1.2. Pourquoi un langage de modélisation spécifique ?

Pour mettre en œuvre cette démarche d'automatisation des études de sûreté de fonctionnement des systèmes, le premier problème consiste à trouver un **langage** permettant au fiabiliste de représenter simplement ses connaissances sur un système.

Ce langage doit répondre aux exigences suivantes :

- Les connaissances doivent être "**faciles**" à représenter pour le fiabiliste. Autrement dit, le langage choisi doit être simple à appréhender pour le fiabiliste. Il doit lui permettre de manipuler simplement les concepts nécessaires à la formalisation de son raisonnement (en intégrant le maximum de notions liées à la fiabilité).

- * Le langage doit permettre la description de connaissances génériques (Bases de connaissances) ;

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- * Il doit fournir un formalisme simple pour la description de systèmes particuliers à partir de la modélisation générique de la base de connaissances. Ce formalisme doit pouvoir être associé à une interface graphique (description de la topologie et des composants du système) ;
- * Il doit permettre de modéliser les systèmes avec le **même** formalisme, quels que soient les modèles de fiabilité à construire. Autrement dit, ce langage doit être une généralisation des modélisations fiabilistes courantes (Arbre de défaillances, Graphe de Markov, Réseau de Petri, ...).

- Les modèles construits avec ce langage (Bases de connaissances et descriptions de systèmes) doivent être **lisibles** par des non-spécialistes.

- Le formalisme utilisé doit être **évolutif** afin de permettre son extension à la modélisation de système pour d'autres types d'études que les études de sûreté de fonctionnement (maintenance, diagnostic).

- Enfin, les modèles construits avec ce langage doivent être **maintenables**.

- * On doit pouvoir modifier facilement la connaissance contenue dans un modèle ;
- * Les connaissances doivent être **structurées** et **formalisées** pour permettre l'accès rapide aux informations que l'on recherche ;
- * Le langage doit être associé à des outils de débogage.

L'analyse des langages informatiques existants et disponibles sur le marché montre qu'aucun de ces langages n'est adapté à ces exigences. Les langages à vocation générale (PROLOG, ADA, C++, Java, Python ...) ne satisfont pas directement aux contraintes de lisibilité des modèles et de simplicité d'appréhension par le fiabiliste.

Ainsi, pour répondre au problème de la modélisation des systèmes dans le domaine des études de sûreté de fonctionnement, EDF a développé son propre langage de modélisation : **le langage Figaro**.

2. Principes de modélisation en Figaro

2.1. Les Classes et les Objets

Le langage Figaro est un langage hybride au sens où c'est à la fois un langage objet (qui gère des classes et des objets) et un langage inspiré de techniques d'intelligence artificielle (qui gère des mécanismes d'inférences sur des règles). Cette section présente de manière générale les notions principales de la partie "Objet" de Figaro. La partie "Intelligence artificielle" sera présentée ultérieurement.

2.1.1. Définitions

Classe : Description commune à tous les composants des systèmes étudiés qui ont le même comportement et les mêmes caractéristiques. Chaque classe est identifiée par un nom et contient une série de caractéristiques qui sont les caractéristiques communes aux composants qu'elle décrit.

Objet : Tout composant d'un système. Chaque objet est identifié par un nom particulier.

En Figaro, chaque objet d'un système appartient à une classe. On dit aussi que l'objet d'une classe est une **instance de la classe**.

Un objet hérite des caractéristiques de sa classe, en les précisant : ce sont les informations écrites en gras dans l'exemple ci-dessous.

Exemple : l'objet S002PO est une instance de la classe 'pompe' :

Objet S002PO

Caractéristiques : état initial = **arrêt**,

tableau d'alimentation = **tableau T1**

Les objets d'un système particulier appartenant à la même classe C forment **l'ensemble** des objets de la classe C du système.

Une classe en Figaro est appelée un **TYPE**.

TYPE pompe ;

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Les caractéristiques permettant de décrire chaque TYPE ou chaque OBJET sont des champs valués ou des règles. Les **champs valués** permettent de définir les propriétés des objets, notamment les différentes variables qui caractériseront l'état d'un objet.

Exemple de TYPE

```

TYPE         pompe ;
ATTRIBUT      (* paragraphe permettant de définir les variables d'état
                  des objets du type pompe *)
                  debit DOMAINE REEL ;
                  etat  DOMAINE 'arret' 'marche' ;

```

Exemples d'OBJETS

```

OBJET S002PO EST_UNE pompe ;
ATTRIBUT
    debit = 2.5e3 ;
    etat  = 'arret' ;

```

```

OBJET S001PO EST_UNE pompe ;
ATTRIBUT
    debit = 5000 ;
    etat  = 'marche' ;

```

Comme nous le verrons plus loin dans ce chapitre, les **règles** permettent de définir le comportement dynamique des objets, c'est à dire les règles d'évolution des variables d'état des objets au cours du temps (par exemple, le démarrage d'une pompe en secours d'une autre, ...).

2.1.2. Structure d'un modèle Figaro

Maintenant que nous avons défini les notions de "classe" et d' "objet", nous pouvons préciser **comment** le langage Figaro est utilisé pour automatiser les études de sûreté de fonctionnement des systèmes. Le principe est le suivant :

- Pour une catégorie de systèmes (systèmes thermohydrauliques, systèmes électriques, ...), on décrit en Figaro dans une **base de connaissances** chaque TYPE de composants appartenant à la catégorie de systèmes étudiée. Une base de connaissances Figaro contient donc un modèle de description commun à tous les systèmes de la classe. C'est un ensemble de TYPES.
- Pour étudier un système particulier, on élabore une **description de ce système** en utilisant la base de connaissances :
 - * Chaque objet du système est créé comme une instance d'un type de la base de connaissances.
 - * Pour chaque objet, des valeurs spécifiques au système sont attribuées à chaque champ valué caractéristique du type de l'objet.

L'association (base de connaissances + ensemble des objets du système) constitue un modèle complet du système à étudier. Ce modèle est appelé le **modèle Figaro** du système.

Des traitements permettent ensuite d'exploiter le modèle Figaro du système pour générer automatiquement les modèles de fiabilité requis.

2.2. Modèles d'ordre 1 et d'ordre 0

Le modèle Figaro décrit au paragraphe précédent est dit « modèle d'ordre 1 » en référence au vocabulaire utilisé en intelligence artificielle pour désigner des bases de règles s'appliquant à des ensembles d'objets non définis de manière explicite.

Une règle de comportement est le plus souvent commune à tous les objets décrits par une même classe. C'est pourquoi le langage Figaro permet d'écrire, pour chaque type d'une base de connaissances, des règles génériques à tous les objets décrits par le type (factorisation de la connaissance).

Toute exploitation de ce modèle va passer par une opération d'instanciation qui en fera un « modèle d'ordre 0 »

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

ne contenant que des règles s'appliquant à des variables bien définies et uniques du système modélisé. Cette opération d'instanciation, possible parce que le nombre d'objets qui compose le système est fixé et n'évoluera pas au cours d'une simulation, présente plusieurs avantages :

- On va faire une seule fois un certain nombre de tests qui permettront de simplifier les règles, voire d'en supprimer complètement certaines lorsque leurs conditions sont toujours évaluées à FAUX.
- Le modèle à l'ordre 0 sera facile à compiler de manière efficace pour permettre des traitements rapides.
- La sémantique du modèle à l'ordre 0 peut être définie formellement (cf. cette définition au chapitre 3) de manière simple.
- Il est possible de faire des contrôles de cohérence supplémentaires sur un modèle d'ordre 0.

Dans un modèle Figaro d'ordre 0, il n'y a que des objets. C'est un modèle « à plat », sans aucune hiérarchie.

La suite du chapitre 2 décrit le langage d'ordre 1, sauf mention explicite de l'ordre 0.

2.3. L'héritage entre classes

L'héritage permet de factoriser des informations et ainsi d'avoir des bases de connaissances plus concises, où l'information est mieux structurée.

Prenons l'exemple d'un système thermo-hydraulique. Il contient à la fois des motopompes et des turbopompes. Ces deux types de pompe ont des caractéristiques communes qu'il est judicieux de mettre en commun. Nous allons donc définir une classe générique (celle des pompes) et deux classes spécialisées (celle des pompes motorisées et celles des turbopompes).

```

TYPE           pompe ;
ATTRIBUT
    debit DOMAINEREEL ;
    etat DOMAINE 'arret' 'marche' ;

TYPE           pompe_motorisee SORTE_DE pompe ;
ATTRIBUT
    puissance_moteur DOMAINE ENTIER ;

TYPE           turbo_pompe SORTE_DE pompe ;
ATTRIBUT
    capacite_turbine DOMAINE ENTIER ;

```

La première classe définit les caractéristiques communes des pompes.

Les deuxième et troisième classes héritent de la première, c'est-à-dire qu'elles disposent des mêmes caractéristiques générales (débit et état), auxquelles elles ajoutent leurs propres caractéristiques (puissance du moteur et capacité de la turbine).

La classe *pompe_motorisée* est donc composée d'une partie de *pompe* et d'une partie de *pompe_motorisée*. Une instance de cette classe, c'est-à-dire un objet du type *pompe_motorisée*, devra initialiser les 3 champs (débit, état et puissance du moteur). Un objet de type *pompe* quant à lui ne dispose toujours que des champs débit et état. Etant donné qu'un objet de type *pompe_motorisée* "contient" un objet de type *pompe*, on considère qu'un objet de type *pompe_motorisée* est aussi une instance de *pompe*.

Il est possible dans une sous-classe de redéfinir certaines caractéristiques de la classe dont elle hérite. Par exemple, la classe *pompe_motorisée* pourrait redéfinir le comportement d'une pompe ou bien certaines de ses caractéristiques (pour, par exemple, particulariser le genre d'une interface ou ajouter une valeur énumérée à un domaine), etc. Cela s'appelle de la **surcharge**. Le mécanisme de surcharge permet de redéfinir un champ ou un comportement dans une classe inférieure, c'est-à-dire une classe qui hérite. Cela permet de particulariser une caractéristique générique. Par exemple, un autre état possible d'une pompe motorisée est la demande de démarrage. On aurait donc :

```

TYPE           pompe_motorisee SORTE_DE pompe ;
ATTRIBUT
    puissance_moteur DOMAINE ENTIER ;
    etat DOMAINE 'marche' 'arret' 'demande demarrage' ;

```

Dans cet exemple, on surcharge la facette **DOMAINE** de l'attribut *etat*. Toute caractéristique ou toute règle ayant un nom peut ainsi être surchargée.

Ce petit exemple a permis de présenter les principaux aspects de la notion d'héritage. Ceux-ci sont les suivants :

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- Factorisation d'information dans des classes supérieures (celles qui sont héritées). On **met en commun** plusieurs attributs et règles dans une même classe.
- Spécification des attributs et des comportements dans les classes inférieures (celles qui héritent). On **ajoute** des attributs et des règles dans les classes spécialisées.
- Redéfinition de certains comportements dans des classes inférieures. On **surcharge** les caractéristiques des classes mères.

Cette méthode d'approche est caractéristique des langages objets.

Il est possible en Figaro qu'une classe hérite de plusieurs classes : cela s'appelle de l'héritage multiple. La classe profitera de chacune des caractéristiques des classes héritées. La nouvelle classe dispose alors de l'**union** des caractéristiques des classes héritées. Elle peut alors ajouter des caractéristiques ou bien en redéfinir comme nous venons de le voir. Dans l'exemple suivant, la classe pompe_motorisée est une pompe et un composant électrique.

```

TYPE         pompe ;
ATTRIBUT

    debit DOMAINEREEL ;
    etat DOMAINE 'arrêt' 'marche' ;

TYPE         composant_electrique ;
ATTRIBUT

    puissance_moteur DOMAINE ENTIER ;

TYPE         pompe_motorisee SORTE_DE pompe composant_electrique ;

```

Néanmoins, certains problèmes peuvent apparaître lorsque deux caractéristiques héritées de classes différentes portent le même nom : il n'est pas possible de connaître la provenance de cette caractéristique.

```

TYPE pompe ;
ATTRIBUT

    etat DOMAINE 'marche' 'arrêt' ;

TYPE appareil_motorise ;
ATTRIBUT

    etat DOMAINE 'marche_rapide' 'marche_lente' 'arrêt' ;

TYPE pompe_motorisee

    SORTE_DE pompe appareil_motorise ;
    (* Quel est le DOMAINE de l'ATTRIBUT etat ? *)

```

Les règles de gestion des mécanismes de surcharge et d'héritage mises en œuvre dans Figaro sont complètement explicitées en **référence** [2]. Dans ce manuel, nous nous contenterons d'exposer les principes de ces mécanismes et les interdits associés (cf. Notions avancées).

2.4. Communication entre les objets

Un système est un ensemble de composants en interaction. Pour assurer les fonctions du système, les composants sont **reliés** entre eux. On peut distinguer trois types de liens principaux :

- Les liens topologiques : les composants sont reliés entre eux pour propager les flux principaux du système. Exemple : tuyauteries pour propager le fluide dans un système thermohydraulique, câbles pour propager le courant dans les systèmes électriques ;
- Les liens fonctionnels : le fonctionnement d'un composant du système est dépendant d'autres composants. Exemple : une pompe en secours d'une autre ne démarre que si la pompe normale est défaillante ;
- Les liens entre les composants du système et les systèmes support. Exemple : un tableau Basse Tension alimente le moteur d'une pompe.

La modélisation des liens entre les objets d'un système est un point important de l'élaboration d'un modèle, puisque le comportement de chaque objet dépend du comportement des objets qui lui sont reliés.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Lorsque l'on dispose d'un système particulier à modéliser, on connaît la structure du système, et on est donc capable de représenter facilement les liens entre les objets du système. En revanche, lorsqu'on élabore un modèle générique pour une catégorie de systèmes (description des classes dans une base de connaissances), on ne connaît que les relations (potentielles ou obligatoires) existant entre les classes. Par exemple, un objet quelconque de type pompe_motorisée est relié à un et un seul objet de type tableau_électrique.

Le langage Figaro permet à la fois :

- de représenter, pour chaque type décrit dans une base de connaissances, les relations potentielles ou obligatoires existant entre un objet quelconque de ce type et les objets des autres types (écriture d'un modèle générique), voire des objets de même type.
- de "renseigner précisément" ces relations au niveau de chaque objet d'un système particulier (d'instancier ces relations) afin de définir précisément l'ensemble des objets du système liés à cet objet particulier.

Les liens entre les objets d'un système sont représentés en Figaro par les **champs d'interface**.

2.4.1. Champs interface d'un type

Lorsqu'on définit un type (une classe d'objets), on doit définir **les relations** existant entre un objet quelconque de ce type et d'autres objets. Plus précisément, les champs interface d'un type permettent de définir :

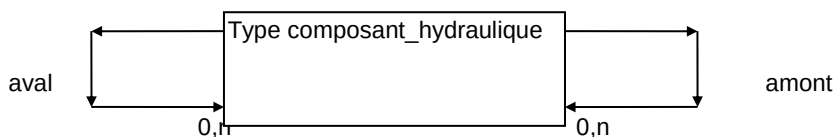
- les relations existant avec les autres objets, de même type ou de types différents,
- la cardinalité de chaque relation, qui permet de préciser notamment si celle-ci est obligatoire ou non.

Exemple 1

Pour fonctionner, les objets de type "Pompe-motorisée" ont besoin d'un et un seul tableau électrique d'alimentation. En Figaro, une telle relation s'écrit :

```
TYPE pompe_motorisee ;
INTERFACE alim_elec GENRE tableau_electrique CARDINAL 1;
```

Remarque : Exemple 2 : Dans un système thermohydraulique, chaque composant est connecté en amont à zéro ou plusieurs composants et en aval à zéro ou plusieurs composants. Ces deux relations, qui ne sont pas obligatoires, se décrivent par le schéma suivant :



Ce qui se lit :

- Un objet de type composant_hydraulique "possède en amont" 0 ou n objets de type composant_hydraulique.
- Un objet de type composant_hydraulique "possède en aval" 0 ou n objets de type composant_hydraulique.

En Figaro, ces relations s'écrivent :

```
TYPE composant_hydraulique ;
INTERFACE
  amont GENRE composant_hydraulique CARDINAL 0 JUSQUA INFINI;
  aval GENRE composant_hydraulique CARDINAL 0 JUSQUA INFINI;
```

Dans une base de connaissances (définition des types), les champs interface permettent de **définir les relations** existant entre les types d'objets. Chaque champ interface contient **l'identifiant d'une relation**.

Une relation définie entre deux types n'est pas symétrique.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

2.4.2. Champs interface d'un objet

Dans une description de système (définition des objets), les champs interface permettent de mettre des **ensembles précis** d'objets en relation. Autrement dit, les **relations définies** dans les types sont **renseignées** (ou instanciées) dans les objets.

Exemple 1 : la pompe p1 est alimentée par le tableau basse tension bt1.

```
p1 EST_UNE pompe_motorisee ;
INTERFACE alim_elec = bt1.
```

Exemple 2 : la pompe p1 est connectée en amont aux deux vannes v1 et v2 et en aval à la vanne v3.

```
p1 EST_UNE pompe ;
INTERFACE      amont = v1, v2 ;
                  aval = v3 ;
```

Pour un objet particulier, chaque champ interface est une liste d'objets, c'est à dire qu'il contient des noms d'objets.

Il est très important de noter que les champs interface d'un objet **doivent** être des champs constants en Figaro. Cela signifie que les liens existants entre les objets d'un système ne pourront pas évoluer au cours de la vie de ce système. Autrement dit, les règles Figaro ne permettront pas d'ajouter un objet à une interface ou d'enlever un objet d'une interface.

Un objet ne peut être mis en interface avec lui-même.

2.4.3. Relations et ensembles

2.4.3.1. Identifiant d'une relation

Il est important de noter que l'identifiant d'une relation peut être assimilé à un nom d'ensemble (au sens mathématique du terme). Reprenons les deux exemples précédents :

- * Si x est un objet quelconque de type pompe_motorisee, *alim_elec DE x* désigne l'ensemble des objets de type tableau_électrique qui sont en relation avec x dans un système quelconque décrit à l'aide de la base de connaissances. Dans l'exemple ci-dessus, *alim_elec DE p1* désigne le singleton {bt1}.
- * Si x est un objet quelconque de type composant_hydraulique, *amont DE x* désigne l'ensemble des objets de type composant_hydraulique qui sont connectés en amont à x dans un système quelconque décrit à l'aide de la base de connaissances. Dans l'exemple ci-dessus, *amont DE p1* désigne l'ensemble {v1, v2}.

2.4.3.2. Cardinalité

Lorsqu'on définit une relation entre un type A et un type B dans une base de connaissances, il est important de spécifier des contraintes sur le cardinal de l'ensemble des objets de type B qui seront en relation avec un objet quelconque de type A. Cela permet notamment de spécifier si la relation est obligatoire ou non. Plusieurs possibilités sont offertes en Figaro, que nous allons illustrer à l'aide de l'exemple suivant :

```
TYPE composant ;
INTERFACE amont GENRE composant;
```

- Si on ne spécifie aucune contrainte sur la relation 'amont', cela signifie qu'un composant peut n'avoir aucun composant en amont (la relation n'est **pas obligatoire**) ou qu'il peut en avoir plusieurs, sans limitation de nombre. Le cardinal de l'ensemble des objets de type composant en amont d'un objet de type composant est indéfini (l'ensemble peut être vide).

```
TYPE composant ;
INTERFACE amont GENRE composant;
```

Remarque : on aurait pu aussi écrire :

```
TYPE composant ;
INTERFACE amont GENRE composant CARDINAL 0 JUSQUA INFINI ;
```

- Si on spécifie que la cardinalité de la relation 'amont' est égale à n, cela signifie qu'un composant doit

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

obligatoirement avoir n composants en amont. Le cardinal de l'ensemble des objets de type composant en amont d'un objet de type composant est égal à n.

```
TYPE composant ;
INTERFACE amont GENRE composant CARDINAL 3;
```

Dans ce cas, si n=1, on dit que l'interface est **monovaluée**. Ce cas particulier est important car il permettra d'écrire des règles avec une syntaxe plus simple que dans le cas général.

- Si on spécifie que la cardinalité de la relation 'amont' est comprise dans l'intervalle p,q cela signifie qu'un composant doit obligatoirement avoir entre p et q composants en amont. Le cardinal de l'ensemble des objets de type composant en amont d'un objet de type composant est compris entre p et q.

```
TYPE composant ;
INTERFACE amont GENRE composant CARDINAL 2 JUSQUA 8;
```

Dans ce cas, si p = 0, la relation n'est pas obligatoire.

2.4.4. Utilisation des champs d'interface

Les champs d'interface seront utilisés pour modéliser l'influence des objets reliés à un composant sur le comportement de ce composant.

Ainsi, comme nous le verrons lors de la présentation des règles Figaro, mettre le type B en interface du type A (en relation avec le type A) permettra d'utiliser les caractéristiques du type B pour modéliser le comportement du type A.

Ainsi, la définition des interfaces existant entre les types décrits dans une base de connaissances est une étape importante de la conception d'une base de connaissances.

2.4.5. Modélisation des liens matériels entre objets

Lorsque deux objets sont reliés entre eux par des liens matériels (tuyauterie, câble, ...), il faut noter que l'on peut envisager deux types de modélisation pour ces liens, en fonction des objectifs de la modélisation.

Si ces liens physiques ont des caractéristiques propres utiles à la modélisation du comportement du système, il faut définir des types particuliers pour ces liens (chaque lien sera un objet).

Par exemple, si pour une classe de systèmes électriques, les caractéristiques des câbles (impédance, longueur) sont importantes pour la modélisation, on définira un type 'cable_electrique' dans la base de connaissances. Ce type sera lui-même interfacé avec les autres types du modèle (chaque câble aura un composant en amont et en aval).

En revanche, si les liens physiques n'ont pas de caractéristiques propres utiles à la modélisation du comportement du système, ils ne correspondront pas à des objets particuliers et les champs d'interface seront suffisants pour les modéliser.

Par exemple, si pour une classe de systèmes thermohydrauliques on s'intéresse uniquement à la propagation du flux entre les composants (le flux passe ou ne passe pas), les composants hydrauliques seront reliés directement (par des champs interface 'amont' et/ou 'aval') sans passer par un objet lien.

2.5. Champs de classes et d'objets

Comme nous l'avons vu précédemment, un type est décrit par des champs communs aux objets du type, et par des règles que nous étudierons dans la partie suivante.

Les champs caractéristiques d'un type ou d'un objet peuvent être de trois natures différentes :

- * Les **champs constants** représentent les caractéristiques des objets qui n'évoluent pas au cours de la vie du système (longueur d'un câble de connexion par exemple). Ces champs sont univalués.
- * Les **champs interface d'un type** permettent de définir les relations potentielles ou obligatoires existant entre un objet de ce type et des objets des autres types, voire du même type. Les **champs interface d'un objet** permettent de renseigner les objets d'un système en relation avec cet objet. Ce sont des champs d'objets, c'est-à-dire contenant des noms d'objets, qui peuvent être multivalués.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- * Les **variables d'état** caractérisent l'état des objets et sont susceptibles d'évoluer au cours de la vie du système. Ces champs sont univalués.

Il est important de noter que, en Figaro, les **champs interface sont des champs constants**. Autrement dit, on ne peut pas modéliser en Figaro des processus qui auraient pour conséquence de modifier les relations existant entre les objets d'un système.

Prenons l'exemple simple de la description d'un type "câble électrique" caractérisé par sa longueur.

```
Exemple :
TYPE cable_electrique ;
CONSTANTE
longueur DOMAINE REEL ;
(*le domaine de définition de la constante est l'ensemble des réels*)
```

Avec la définition de ce type, on peut créer une instance particulière de "câble électrique" pour décrire un système, en affectant au champ "longueur" la valeur pertinente pour le système étudié :

```
Exemple :
OBJET cable1 EST_UN cable_electrique ;
CONSTANTE
longueur = 25 ;
```

2.6. Lien entre les concepts Figaro et UML

Il est facile de faire une correspondance entre les concepts principaux de Figaro et les concepts les plus couramment utilisés dans un méta modèle décrit en UML. Le tableau ci-dessous résume cette correspondance.

Concept UML	Concept Figaro
Classe	Type
Bloc (de SysML)	Objet
Héritage	Héritage
Attribut	Constante, attribut, effet, panne
Relation	Interface

Contrairement à Figaro, UML ne fait pas une distinction très claire entre classes et objets. Mais dans la spécialisation SysML d'UML, on décrit les systèmes avec des « blocs » qui s'apparentent plus à des objets qu'à des classes.

Ainsi, il est facile d'utiliser un éditeur graphique UML pour représenter les principaux éléments du méta modèle d'une base de connaissances Figaro. Une telle représentation graphique peut être un support de réflexion intéressant pour la construction d'une nouvelle base de connaissances ou pour représenter de manière synthétique le contenu d'une base existante.

2.7. Règles Figaro

En langage Figaro, le comportement dynamique (c'est-à-dire au cours du temps) des objets est modélisé sous forme de règles. Pour bien comprendre ce qu'est une règle en Figaro, il est nécessaire de comprendre le **principe général de modélisation des systèmes** utilisé dans un modèle Figaro. Cette section donne des explications simples, avec beaucoup d'exemples. La section 3 donnera une définition formelle de la sémantique du langage, base du fonctionnement des outils fondés sur Figaro.

2.7.1. Principe de modélisation des systèmes

Etat de modèle et événement

L'état d'un objet à un instant donné se définit comme l'ensemble des valeurs prises à cet instant par les variables d'état de cet objet. **L'état d'un système** à un instant donné est complètement défini par l'état des objets qui constituent le modèle Figaro à cet instant.

L'état du système va évoluer en fonction du temps, suite à des **événements** pouvant survenir sur les objets du

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

système.

L'occurrence d'un événement est conditionnée par l'état courant du système. Autrement dit, un événement ne peut se produire que si les variables d'état du modèle vérifient une certaine condition.

Si la pompe P1 est en marche

Alors, il peut se produire l'événement "Défaillance en fonctionnement de la pompe"

Un événement correspond à un phénomène aléatoire, qui se produit en fonction d'une certaine loi de probabilité, ou à un phénomène déterministe qui se produit au bout d'un temps constant donné (si la condition d'occurrence de l'événement est maintenue pendant cette durée).

Si la pompe P1 est en marche,

Alors, il peut se produire l'événement "Défaillance en fonctionnement de P1" suivant une loi de probabilité exponentielle de paramètre $1e-3/h$.

Le langage Figaro permet donc de modéliser des processus markoviens (l'occurrence d'un événement ou d'une transition dépend **uniquement** de l'état courant du système) ou des processus semi-markoviens (l'occurrence d'un événement dépend de l'état courant du système et du temps écoulé dans cet état). C'est très semblable à ce qui se passe dans un réseau de Petri stochastique.

Conséquences d'un événement sur le système

Un événement survenant sur un composant du système va modifier l'état du composant, ce qui revient à modifier les variables d'état associées à ce composant. Par exemple :

L'occurrence de l'événement "Défaillance en fonctionnement de P1" modifie une variable d'état de P1 qui passe de "marche" à "panne".

La modification de l'état du composant touché par l'événement va entraîner toute une série de conséquences sur les autres composants du système et sur le système lui-même. (Les effets d'un événement sur un composant particulier vont se "propager" sur tout le système). On peut citer pour illustrer les deux exemples suivants :

- Lorsque le composant P1 est en panne, la file d'alimentation passant par P1 est perdue.
- Lorsque le composant P1 est en panne, la pompe de secours associée à P1 va être sollicitée au démarrage.

Cycle de vie d'un système

On considère que la vie d'un système est composée d'une suite de cycles, chaque cycle pouvant être décomposé en deux étapes :

- Lorsque le système est dans un état donné, certains événements peuvent se produire. Dans un premier temps, l'occurrence d'un événement sur un composant aura des conséquences **directes et locales** sur l'état de ce composant.
- Dans un deuxième temps, la modification de l'état des variables du composant va entraîner des conséquences sur les autres composants du système, **propageant** ainsi les effets de l'événement pour obtenir un nouvel état de système.

Une fois que le système est dans ce nouvel état, d'autres événements peuvent se produire.

Pour permettre de modéliser le comportement des systèmes en fonction de ce principe, le langage Figaro utilise un formalisme en règles. Deux types de règles existent en Figaro :

- Les **règles d'occurrence** permettent de modéliser les événements qui peuvent se produire sur un objet : conditions d'occurrence de l'événement en fonction de l'état du système, loi de probabilité associée, conséquences sur l'état du composant.
- Les **règles d'interaction** permettent de propager les conséquences d'un événement à tout le système en exprimant les relations existant entre l'état d'un objet et l'état des autres objets du système.

Avant d'étudier comment ces règles sont utilisées pour exploiter un modèle Figaro, nous allons définir plus précisément leur formalisme.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

2.7.2. Formalisme d'une règle Figaro

Le formalisme d'une règle Figaro diffère selon qu'il s'agit d'une règle d'occurrence ou d'une règle d'interaction.

2.7.2.1. Forme d'une règle d'occurrence

Une règle d'occurrence modélise l'influence sur le système d'un événement pouvant survenir sur les objets d'un système. De façon schématique, ces règles sont de la forme :

SI <condition>

IL_PEUT_SE_PRODUIRE <événement>

PROVOQUE <actions>

- * La condition est une expression booléenne qui exprime la condition que doit vérifier l'état du système pour qu'un événement se déclenche sur un objet. (Si la condition est évaluée à VRAI, l'événement peut se produire).
- * Les actions décrivent les conséquences de l'événement sur les variables d'état de l'objet touché par l'événement.

Exemple de règle d'occurrence

```

TYPE pompe;
ATTRIBUT
    etat DOMAINE 'arret' 'sollicite' 'marche' 'panne';
CONSTANTE
    lambda PAR_DEFAULT 1e-3 ;
OCCURRENCE
    SI etat = 'marche'
    IL_PEUT_SE_PRODUIRE
    TRANSITION defaillance_en_fonctionnement
        LOI EXP ( lambda )
        PROVOQUE etat <-- 'panne' ;

```

Cette règle d'occurrence **générique** aux objets de la classe 'pompe' se lit :

Soit o un objet quelconque de type pompe,

Si 'etat' de o est 'marche',

Il peut se produire l'événement 'défaillance en fonctionnement' de o au bout d'un temps de loi exponentielle de paramètre 'lambda'

Qui provoque le passage à 'panne' de la variable 'etat' de o.

2.7.2.2. Forme d'une règle d'interaction

Une règle d'interaction modélise les relations logiques existant entre l'état d'un objet et l'état des autres objets du système. De façon schématique, ces règles sont de la forme :

SI <condition>

ALORS <actions>

SINON <actions>

Pour ces règles, la condition (expression booléenne introduite par SI) permet le déclenchement d'un ensemble d'actions affectant les variables d'état des objets du système. (Si la condition est évaluée à VRAI, les actions introduites par ALORS se déclenchent ; si la condition est évaluée à FAUX, les actions introduites par SINON se déclenchent).

Exemple de règle d'interaction

```

TYPE pompe;
ATTRIBUT
    etat DOMAINE 'arret' 'sollicite' 'marche' 'panne';
INTERFACE
    pompe_secours GENRE pompe CARDINAL 1;

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

INTERACTION

SI `etat = 'panne'`

ALORS

`etat DE pompe_secours <-- 'sollicite' ;`

Dans cet exemple, l'ensemble des objets de type pompe en relation "pompe_secours" avec une pompe quelconque est un ensemble de cardinal 1. Ainsi, "etat DE pompe_secours" désigne l'état de l'unique pompe en secours d'une pompe quelconque. Cette règle d'interaction générique aux objets de la classe 'pompe' se lit :

Soit **o** un objet quelconque de type pompe,

Si 'etat' de **o** est 'panne',

Alors 'etat' de l'unique objet de type pompe en relation "pompe_secours" avec **o** passe à 'sollicité'.

2.7.2.3. Condition et actions d'une règle, utilisation des champs d'interface

En principe, le comportement des objets d'une classe dépend principalement de l'état de ces objets et de l'état des objets des classes avec lesquels ils sont en relation (classes en interface). Ainsi, pour une classe donnée :

- la condition d'une règle est une expression booléenne utilisant les propriétés de la classe et les propriétés des classes en interface,
- les actions d'une règle affectent les variables de la classe et les variables des classes en interface. (Par définition, les actions d'une règle d'occurrence affectent uniquement les variables de la classe).

Pour illustrer, nous allons donner quelques exemples qui montrent comment sont utilisés les champs interface d'un type pour écrire les règles de comportement de ce type.

Exemple 1

Prenons l'exemple des objets de type pompe. Dans cet exemple, le type pompe est hiérarchisé en deux sous-types : les pompes qui fonctionnent en temps normal (type pompe_normale) et les pompes qui fonctionnent en secours des pompes normales (type pompe_secours).

Chaque objet de type pompe_secours permet de secourir une ou plusieurs pompes normales. Autrement dit, chaque objet de type pompe_secours est relié à une ou plusieurs pompes normales. La règle de comportement que nous souhaitons modéliser est la suivante :

Si toutes les pompes normales secourues par la même pompe de secours sont en panne, alors le système demande le démarrage de cette pompe secours.

Le comportement des objets de type pompe_secours dépendant de l'état des objets de type pompe normale, il faut créer une relation entre le type pompe_secours et le type pompe_normale. Cette relation est identifiée par le champ interface 'secourue' et elle est de cardinalité 1 à INFINI.

TYPE pompe;

ATTRIBUT

`panne DOMAINE BOOLEEN ;`

TYPE pompe_normale **SORTE_DE** pompe;

TYPE pompe_secours **SORTE_DE** pompe;

ATTRIBUT

`demande_demarrage DOMAINE BOOLEEN ;`

INTERFACE

`secourue GENRE pompe_normale CARDINAL 1 JUSQUA INFINI;`

INTERACTION

SI `QSOIT x UN secourue ON_A panne DE x = VRAI`

ALORS `demande_demarrage <-- VRAI ;`

Cette règle d'interaction se lit :

Soit **o** un objet quelconque de type pompe_secours,

Si $\forall x \in \{\text{pompes normales secourues par } o\}$, **x** est en panne,

Alors, il y a demande de démarrage de **o**.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Dans cet exemple, la condition de la règle dépend des caractéristiques des objets d'une classe en interface de la classe pompe_secours. L'action de la règle affecte uniquement les objets de la classe décrite. **Autrement dit, cette règle d'interaction permet de modifier l'état d'un objet de la classe en fonction de l'état des objets des classes en interface.**

Exemple 2

Dans ce nouvel exemple, on suppose que chaque pompe normale d'un système est secourue par une ou plusieurs pompes secours. La règle de comportement que nous souhaitons modéliser est la suivante :

Si une pompe normale est en panne, alors le système demande le démarrage de toutes les pompes de secours associées.

```

TYPE pompe;
ATTRIBUT
    panne DOMAINE BOOLEEN;

TYPE pompe_secours SORTE_DE pompe;
ATTRIBUT
    demande_demarrage DOMAINE BOOLEEN;

TYPE pompe_normale SORTE_DE pompe;
INTERFACE
    secours GENRE pompe_secours CARDINAL 1 JUSQUA INFINI;
INTERACTION
SI panne = VRAI
ALORS
POUR_TOUT x UN secours FAIRE demande_demarrage DE x <--VRAI;

```

Cette règle d'interaction se lit :

Soit o un objet quelconque de type pompe_normale,

Si o est en panne,

Alors, $\forall x \in \{\text{pompes secours en secours de } o\}$, il y a demande de démarrage de x.

Dans cet exemple, la condition de la règle dépend uniquement des caractéristiques des objets de la classe décrite. L'action de la règle affecte les objets de la classe pompe_secours en interface. **Autrement dit, cette règle d'interaction permet de modifier l'état des objets des classes en interface en fonction de l'état des objets de la classe décrite.**

Problèmes potentiels

Les exemples ci-dessus montrent que la définition et l'utilisation des interfaces doivent être faites avec beaucoup de soin, de façon à ne pas écrire des incohérences. Des problèmes peuvent par exemple se poser s'il existe deux relations différentes entre deux classes.

Dans l'exemple simple suivant, le type t1 est en relation avec t2 par l'interface r12 et, réciproquement, le type t2 est en relation avec t1 par l'interface r21. La règle 'Règle_1' modifie l'état d'un objet de type t1 en fonction de l'état de l'objet de type t2 interfacé. La règle 'Règle_2' modifie l'état d'un objet de type t2 en fonction de l'état de l'objet de type t1 interfacé. Dans cet exemple, ces deux règles sont incohérentes car elles ne permettent pas de conclure sur l'état des objets de ces deux types.

```

TYPE t1;
ATTRIBUT
    a DOMAINE BOOLEEN;
INTERFACE
    r12 GENRE t2 CARDINAL 1;
INTERACTION
Règle_1
SI b(r12) = VRAI
ALORS a <-- FAUX
SINON a <-- VRAI;

TYPE t2;

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

ATTRIBUT

b DOMAINE BOOLEEN;

INTERFACE

r21 GENRE t1 CARDINAL 1;

INTERACTION

Règle_2

SI a(r21) = VRAI

ALORS b <-- VRAI

SINON b <-- FAUX;

Interface monovaluée et interface multivaluée

Il est important de noter que l'utilisation d'une interface monovaluée (de cardinal 1) diffère de celle d'une interface multivaluée. Comme nous le verrons plus précisément dans la suite de ce manuel, un champ interface monovalué est utilisé comme un champ normal, alors qu'un champ interface multivalué ne peut être géré que par des quantificateurs (IL_EXISTE, IL_EXISTE AU_MOINS, QQSOIT, POUR_TOUT, SOIT) ou des opérateurs complexes (SOMME POUR_TOUT, PRODUIT POUR_TOUT...).

2.7.3. Principes d'utilisation des règles

Exploiter un modèle Figaro, c'est appliquer l'ensemble des règles de la base de connaissances aux objets décrivant le système à étudier.

Deux types d'application des règles Figaro sont possibles, correspondant à deux types de traitements : application en chaînage avant et application en chaînage arrière.

Dans tous les cas, l'exploitation se fait sur le modèle instancié à l'ordre 0, ainsi que nous l'avons vu au § 2.2.

2.7.3.1. Application en chaînage avant

Le traitement en chaînage avant est un traitement cyclique qui s'effectue en deux étapes (Figure 2) :

- a) Déclenchement d'un événement sur un objet du système
- b) Propagation des effets de l'événement sur le système par application cyclique des règles d'interaction jusqu'à stabilisation de l'état.

a) Déclenchement d'un événement

Lorsque le système est dans un état donné, certains événements peuvent se produire. Cela signifie que les conditions de certaines règles d'occurrence de certains objets sont vérifiées. Le moteur choisit une¹ règle d'un objet et l'applique (exécute les actions provoquées par l'événement), ce qui modifie les variables d'état de l'objet. Le système se trouve alors dans un état que nous qualifierons d'incomplet, puisque seules les conséquences de l'événement sur l'objet affecté sont prises en compte. Un état incomplet n'est pas un état réel du système.

b) Application des règles d'interaction

Pour propager les conséquences de l'événement à tout le système, le traitement applique ensuite les règles d'interaction. Pour cela, il procède comme suit :

Passage dans les règles d'interaction

Le traitement applique les règles d'interaction objet par objet.

Pour chaque objet, le traitement prend la "première" règle de l'objet et détermine si elle est applicable en fonction de l'état courant du système (initialement, cet état est l'état incomplet obtenu après tirage de la règle d'occurrence). Si la règle est applicable, il l'applique, calcule le nouvel état courant du système et passe à la règle suivante de l'objet.

Lorsque toutes les règles d'un objet ont été examinées, le traitement passe à l'objet suivant.

Remarque : Il est très important de noter qu'un nouvel état du système est calculé après chaque application d'une règle d'interaction pour un objet. Ainsi, si une règle d'un objet n'est pas applicable initialement, elle peut devenir applicable après un passage dans les premières règles.

Lorsque le traitement est passé une fois dans chacune des règles d'interaction, un nouvel état du système est

¹ Nous verrons plus tard que, dans certains cas, plusieurs événements peuvent être appliqués simultanément.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

obtenu. Mais ce n'est pas forcément un état complet correspondant à un état réel du système : les règles sont à nouveau appliquées de la première à la dernière (dans le même ordre que lors du premier « tour »). Si, à la fin de deux tours successifs, l'état obtenu est le même, on dit qu'il y a convergence et on peut arrêter l'inférence. Cela signifie que l'on a atteint un « point fixe ».

Exemple :

Prenons l'exemple simple de 3 composants thermohydrauliques A, B et C connectés en série. Dans la base de connaissances, nous avons la règle d'interaction suivante :

```

TYPE composant;
INTERFACE
    montant GENRE composant;
ATTRIBUT
    alimente DOMAINE BOOLEEN;
INTERACTION
    r1
    SI QQSUIT x UN montant ON_A alimente DE x = FAUX
    ALORS alimente <-- FAUX;

```

Cette règle générique r1 se lit :

Soit o un objet quelconque de type composant,

Si $\forall x \in \{\text{composants en amont de } o\}$, x n'est pas alimenté,

Alors o n'est pas alimenté.

La base de faits telle qu'initialisée par l'utilisateur est la suivante :

```

alimente DE A = VRAI
alimente DE B = VRAI
alimente DE C = VRAI

```

Supposons qu'un événement survenu sur le système ait provoqué alimenté DE A = FAUX. L'état incomplet du système après cet événement est donc :

```

alimente DE A = FAUX
alimente DE B = VRAI
alimente DE C = VRAI

```

La règle d'interaction r1 instanciée sur l'objet B est la suivante :

```

OBJET B;
INTERACTION
    r1
    SI alimente DE A = FAUX
    ALORS alimente DE B = FAUX ;

```

La règle r1 de l'objet B est donc applicable. Après application, le nouvel état du système est :

```

alimente DE A = FAUX
alimente DE B = FAUX
alimente DE C = VRAI

```

Compte tenu de ce nouvel état, la règle r1 instanciée sur l'objet C est applicable (elle ne l'était pas initialement). Le nouvel état obtenu pour le système est :

```

alimente DE A = FAUX
alimente DE B = FAUX
alimente DE C = FAUX

```

Application des règles d'interaction jusqu'à stabilisation

Les règles d'interaction permettent de propager les effets d'un événement de proche en proche. Il faut donc **faire autant de passages dans ces règles qu'il est nécessaire** pour que toutes les conséquences d'un événement

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

soient prises en compte. Lorsque le point fixe est atteint², cela signifie que toutes les conséquences de l'événement ont été prises en compte.

Après passage dans les règles d'interaction, on obtient un nouvel état du modèle (état complet) pour lequel **d'autres événements peuvent éventuellement se produire**.

Exemple :

Reprenons l'exemple précédent en supposant que le système examine l'objet C avant l'objet B. Dans ce cas, lors du premier passage, seule la règle instanciée sur l'objet B est appliquée et l'état obtenu après premier passage est :

```

alimente DE A = FAUX
alimente DE B = FAUX
alimente DE C = VRAI

```

Lors du deuxième passage dans les règles d'interaction, la règle instanciée sur l'objet C sera appliquée et on obtiendra le même état final que précédemment.

Remarque sur l'ordre d'application des règles d'interaction

Il est très important de noter que **l'ordre d'application des règles d'interaction ne doit avoir aucune conséquence sur le résultat obtenu**. (C'est le cas dans notre exemple où, quel que soit l'ordre de prise en compte des objets, le résultat obtenu est le même – seul diffère le nombre de « tours » des règles nécessaire pour stabilisation de l'état).

Ainsi, le concepteur de base de connaissances devra veiller à la **commutativité des règles** d'interaction, car il ne maîtrise pas l'ordre de tirage des règles par le moteur.

Pour des cas complexes, nous verrons plus loin dans ce manuel (cf. § 7.5) qu'il est possible d'introduire un ordre partiel dans l'application des règles d'interaction, en regroupant celles-ci en sous-ensembles de règles (étapes) et en appliquant chaque sous-ensemble l'un après l'autre. Le chapitre 8 consacré à la cohérence des modèles explique les précautions à prendre dans l'utilisation des étapes.

c) Exploitation d'un modèle Figaro en chaînage avant

L'exploitation d'un modèle Figaro en chaînage avant, la plus « naturelle », permet d'explorer tous les états d'un système ainsi que les événements reliant ces états. Ce type d'exploitation peut donc être utilisé :

- En simulation interactive : l'utilisateur choisit lui-même la suite de transitions qu'il veut simuler et il peut ainsi voir les effets sur le système à chaque transition
- Pour construire la matrice du **graphe d'états** du système et calculer ainsi la probabilité de chaque état (cela n'est possible que si le modèle est markovien, c'est-à-dire si toutes les transitions sont de loi instantanée ou exponentielle).
- Pour explorer les **séquences** du graphe de manière à calculer les probabilités des séquences menant à un état de panne du système. (Une séquence est un ensemble d'événements menant de l'état initial du système à un état particulier).
- Pour faire une simulation de Monte Carlo.

² Un tel point fixe n'existe pas toujours. Cf. le § 8 sur la cohérence des modèles pour assurer son existence.

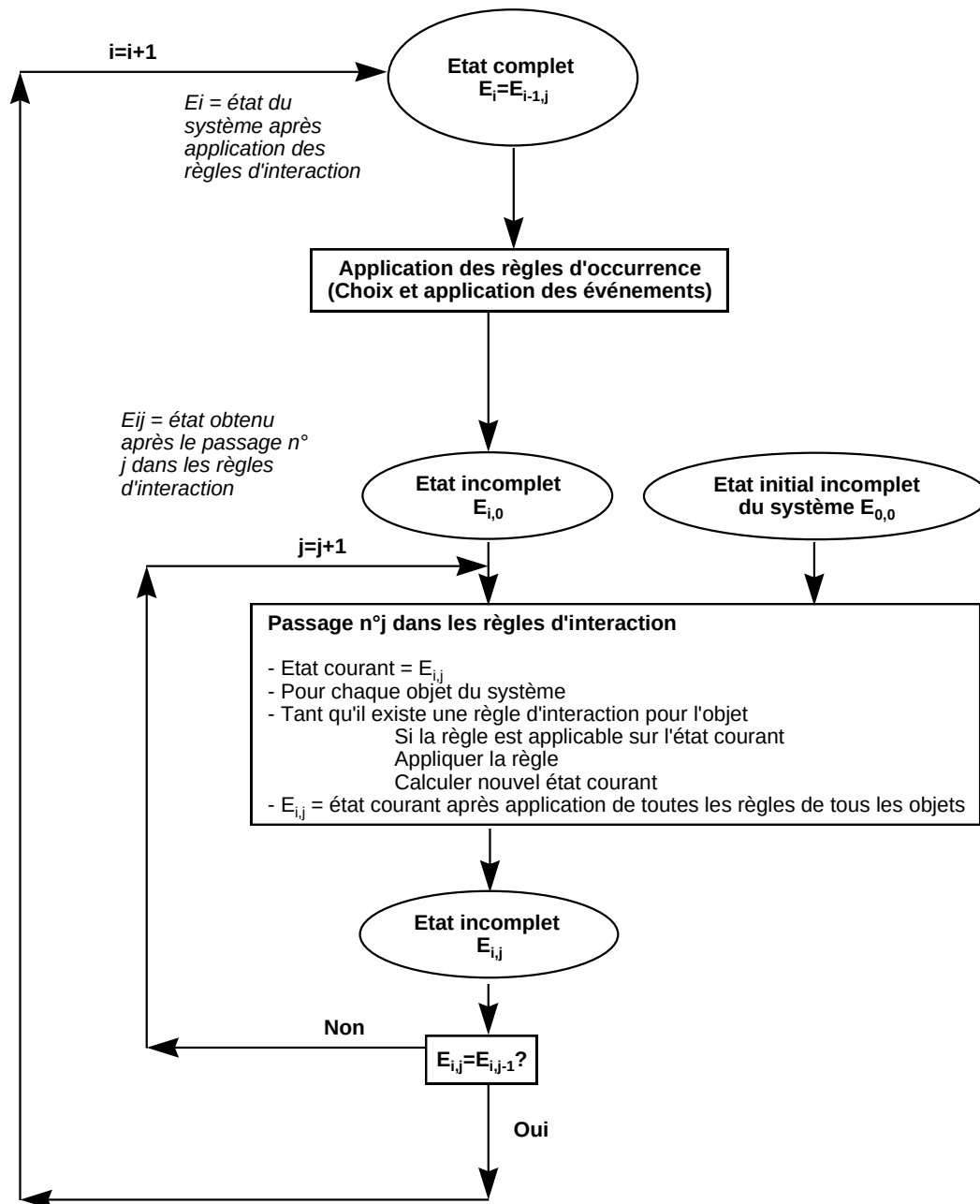


Figure 2. Schéma de principe du chaînage avant en Figaro

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

2.7.3.2. Application en chaînage arrière

L'application des règles Figaro en chaînage arrière [3] est utilisée pour générer automatiquement un **Arbre de défaillances** à partir :

- du modèle Figaro 0 associé au système à étudier,
- de la description de l'événement indésirable que l'on cherche à étudier, qui est défini par un fait du type : égalité d'une variable du modèle avec une valeur donnée.

Exemple : Un modèle Figaro représentant un système thermohydraulique contient des objets représentant des composants thermohydrauliques. Chaque objet est décrit notamment par une variable booléenne "alimenté" qui définit si l'objet est alimenté ou non en fluide. Si l'objet B1 représente le but du système à alimenter, l'événement indésirable peut être décrit par le fait "alimenté (B1) = FAUX".

Pour construire l'Arbre de défaillances, le traitement procède en deux étapes :

- construction d'un "arbre des causes" (composé uniquement de portes logiques) par chaînage arrière sur les règles d'interaction du modèle Figaro,
- transformation de "l'arbre des causes" en Arbre de défaillances en utilisant les règles d'occurrence du modèle ou bien les modèles de fiabilité associés aux pannes pour créer des modèles de défaillance probabilistes au niveau des feuilles de l'arbre.

a) Construction de "l'arbre des causes" par chaînage arrière sur les règles d'interaction

Pour construire "l'arbre des causes" associé à l'événement indésirable, on applique **les règles d'interaction** du modèle en chaînage arrière.

"L'arbre des causes" associé à un événement indésirable est un arbre explicitant les relations logiques existant entre l'événement indésirable et des événements de base. Un événement de base est un fait que l'on ne peut pas expliquer sous forme d'autres faits figurant dans les règles d'interaction. Il correspond :

- * soit à une défaillance d'un composant du système (par exemple "défaillance à la sollicitation de la pompe p1 = VRAI"),
- * soit à une expression représentative de l'état du système (par exemple "position de la vanne v1= ouvert").

Transformer un "arbre des causes" en arbre de défaillances consiste à remplacer toutes les feuilles de l'arbre des causes qui ne sont pas des défaillances par VRAI ou FAUX suivant la valeur du fait considéré dans l'état initial du système.

Pour construire l'arbre des causes associé à un événement indésirable, on applique le processus suivant.

Partant du fait à expliquer (de l'événement indésirable à expliquer), on sélectionne toutes les règles d'interaction (RV1, ..., RVn) concluant sur ce fait (dont les actions provoquent le fait), ainsi que toutes les règles d'interaction (RF1, ..., RFp) dont la conclusion est incompatible avec ce fait (dont les actions donnent une valeur différente à la variable).

Le raisonnement appliqué est alors le suivant. Pour que le fait soit vérifié, il faut et il suffit qu'une des règles RV1...RVn (de conditions CV1...CVn) s'applique et qu'aucune des règles RF1...RFp (de conditions CF1...CFp) ne s'applique. On construit alors l'élément d'arbre suivant :

Événement indésirable = (CV1 ou CV2....ou CVn) et non (CF1 ou CF2 ... ou CFp)

Le chaînage arrière continue en appliquant le même raisonnement sur chacune des conditions non développées de l'arbre. Le raisonnement en chaînage arrière est donc un **raisonnement inductif** qui permet d'expliquer un fait particulier.

Remarque : Il est important de noter que, pour que le traitement en chaînage arrière soit applicable, les règles d'interaction doivent notamment vérifier les contraintes suivantes :

- la condition d'une règle doit être une expression booléenne simple du type

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- "variable (objet) = valeur",
- les actions d'une règle doivent être du type "variable (objet) <-- valeur".

Le chaînage s'arrête lorsque, pour chaque feuille de l'arbre :

- aucune règle d'interaction ne conclut sur le fait représenté par la feuille ou sur un fait incompatible,
- la feuille a déjà été explicitée dans l'arbre des causes.

"L'arbre des causes" obtenu est donc un arbre logique comprenant des portes ET, OU, k/n et éventuellement des négations, et où les feuilles de l'arbre sont des expressions booléennes simples (tests sur la valeur d'une variable).

b) Transformation de l'Arbre des causes en Arbre de défaillances

Pour transformer l'arbre des causes en Arbre de défaillances, le traitement utilise les règles d'occurrence du modèle et les modèles de fiabilité associés aux pannes. Pour chaque feuille de l'arbre des causes :

- Si il existe une règle d'occurrence applicable (c'est-à-dire dont la condition est vérifiée pour l'état initial du modèle) dont les actions provoquent le fait associé à la feuille, la feuille est remplacée par l'événement de la règle d'occurrence et suivant son type (instantané ou temporisé) un modèle de feuille adéquat est créé.

Exemple :

Une feuille de l'arbre contient le fait "défaillance à la sollicitation de la pompe P1 = VRAI" (def_soll (P1) = VRAI). Le modèle Figaro contient la règle d'occurrence suivante pour le type pompe.

```

TYPE pompe ;
ATTRIBUT
    def_soll DOMAINE BOOLEEN ;
CONSTANTE
    etat_initial DOMAINE 'marche' 'arret' ;
OCCURRENCE
    SI etat_initial = 'arret'
    IL_PEUT_SE_PRODUIRE
        DEFAILLANCE def_soll
        LIBELLE "défaillance à la sollicitation"
        LOI INS (gamma_pompe)
        PROVOQUE
            def_soll <-- VRAI ;

```

Cette règle d'occurrence est applicable pour l'objet P1 qui est initialement à l'arrêt. Le traitement remplace donc la feuille de l'arbre des causes par l'événement de base décrit par la règle "défaillance à la sollicitation de la pompe P1", auquel est associée une probabilité de défaillance gamma_pompe.

- Si la feuille de l'arbre correspond au fait qu'une panne est à VRAI et si il existe un modèle de fiabilité associé à cette panne, ce modèle est directement mis dans la feuille. Cette deuxième possibilité a été introduite pour permettre de générer des arbres de défaillances avec des modèles de feuilles variés, éventuellement complexes, qu'il n'aurait pas été possible d'inférer à partir des règles d'occurrence. Ce mécanisme est en quelque sorte en concurrence avec le précédent. Il faut donc choisir lequel on veut exploiter, et éventuellement utiliser des groupes de règles distincts pour l'exploitation du modèle en chaînage avant et en chaînage arrière ; on peut ainsi utiliser les règles d'occurrence pour le chaînage avant et les modèles de fiabilité associés aux pannes pour le chaînage arrière.
- Si il n'existe pas de règle d'occurrence provoquant le fait associé à la feuille et si ce fait n'est pas une panne associée à un modèle de fiabilité, l'expression booléenne associée est évaluée en fonction de l'état initial du modèle et l'arbre des causes est simplifié en fonction du résultat obtenu. Les règles de simplification utilisées sont simples : une feuille VRAI sous une porte ET est éliminée, une feuille VRAI sous une porte OU provoque la mise à VRAI de la porte OU, etc.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Exemple :

Reprenons l'exemple ci-dessus en supposant qu'aucune règle d'occurrence du modèle ne provoque `def_soll (P1) = VRAI`. Dans ce cas, si la valeur de la variable `def_soll` est VRAI pour P1 dans l'état initial du système, la feuille est mise à VRAI et l'arbre est simplifié en conséquence.

L'arbre de défaillance construit permet donc d'expliquer un **état indésirable** du système (état défini par la donnée de l'événement indésirable) par tous les événements de base (défaillances de composant) pouvant survenir dans le système à partir d'un **état initial** donné.

3. Définition formelle du langage Figaro 0

Le langage Figaro d'ordre 1 est complexe et il est donc difficile d'en donner une définition formelle : celle-ci serait d'une longueur déraisonnable, et illisible. En revanche, il est possible de donner une définition formelle simple du langage Figaro d'ordre 0.

Dans la mesure où le langage d'ordre 1 n'est qu'un moyen commode de générer des modèles d'ordre 0 qui sont le point de départ de tout traitement, il est suffisant de définir formellement le langage d'ordre 0.

Afin de garder un exposé simple, nous écarterons dans la présente description la possibilité, offerte par le langage Figaro, de définir des lois probabilistes de type Dirac, servant à modéliser des déclenchements déterministes d'événements au bout de temps fixés. Nous ne détaillerons pas non plus le comportement temporel de l'automate : nous nous contenterons de définir les états vers lesquels l'automate peut évoluer à partir d'un état donné, et si le changement d'état est instantané ou a lieu après un temps non nul. Ces simplifications nous éviteront d'avoir à définir un échéancier, une notion relativement complexe qui encombrerait quelque peu le propos. La distinction entre états instantanés et états non instantanés ou "tangibles" est une **nécessité** pour donner au langage un pouvoir de modélisation suffisant. En effet, ces deux types d'états ne sont pas traités de la même manière, ainsi qu'on le verra dans le paragraphe définissant comment l'automate peut évoluer à partir d'un état donné.

3.1. Eléments du modèle

Soit L un ensemble de lois probabilistes, appartenant à l'une des deux catégories suivantes : lois continues sur $[0, +\infty]$, lois discrètes associant des probabilités à un nombre fini de modalités.

Un modèle en langage Figaro 0 est un nuplet $(\mathcal{E}, O, T, I, \sigma, Y_0, V_0)$ composé des éléments suivants :

$\mathcal{E}$ est le produit cartésien $E_1 \otimes E_2 \dots \otimes E_n$ des domaines des composantes de X , un vecteur fini de variables d'état $(x_1, x_2, \dots, x_n)$ dont la valeur définit l'état du modèle à l'instant t . Chaque variable d'état x_i est soit un booléen, soit un entier, soit un réel, soit une variable dite « énumérée » pouvant prendre ses valeurs dans un ensemble E_i fini. X est en fait la concaténation V, Y de deux vecteurs V et Y . V regroupe les variables dites "**essentiels**" et Y les variables dites "**déduites**".

V_0 est la valeur initiale de V , et Y_0 est la valeur dite "de réinitialisation" de Y . **Y est une fonction de V et de Y_0 définie à l'aide de la fonction I décrite plus loin.**

T est l'ensemble des **groupes de transitions** du modèle. Une **transition** est une application de $\mathcal{E}$ dans $\mathcal{E}$, qui à tout vecteur d'état fait correspondre un autre vecteur d'état, en général obtenu par la modification d'un faible nombre de variables essentielles. Un groupe de transitions, quant à lui, est un couple dont les éléments sont un ensemble de transitions dites "liées", et une loi de probabilité de L . Le groupe de transitions contient une seule transition si sa loi n'est pas de type loi discrète. Si la loi est de type discret, le groupe de transitions contient autant de transitions que la loi a de modalités. Une transition représente un changement local de l'état d'un composant du système modélisé, par exemple l'apparition d'un mode de défaillance. *Un groupe de transitions associé à une loi discrète modélise les diverses issues possibles d'un processus non déterministe considéré comme instantané* ; par exemple, les divers résultats d'un jet de dé, ou le choix entre démarrage et refus de démarrage d'un composant. Par abus de langage, on confondra la notion de groupe de transitions et de transition pour les groupes ne contenant qu'une transition. La notion de groupe de transitions est une originalité du langage Figaro par rapport à l'ensemble des formalismes destinés à décrire des automates non déterministes, à commencer par les réseaux de Petri. Son importance apparaîtra plus loin, lorsque nous décrirons comment l'automate peut évoluer à partir d'un état pour lequel *plusieurs groupes de transitions instantanées* sont applicables.

O est une fonction de $\mathcal{E}$ dans $\mathcal{P}(T)$. En pratique, O est définie par l'ensemble des **règles dites "d'occurrence"**. Ces règles permettent d'associer un ensemble (éventuellement vide) de groupes de transitions à n'importe quel

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

état X du modèle. Les transitions appartenant aux groupes de $O(X)$ sont dites "**valides**" dans l'état X .

I est une fonction de $\mathcal{E}$ dans $\mathcal{E}$, qui à tout vecteur d'état X **pour lequel I est définie** fait correspondre un autre vecteur d'état. La fonction I est en pratique définie par les **règles dites "d'interaction"** du modèle Figaro (et éventuellement un **système d'équations linéaires**), et cette fonction **dépend éventuellement de l'ordre σ des règles**.

La fonction I est définie comme la composée d'un ensemble fini de fonctions de $\mathcal{E}$ dans $\mathcal{E}$, notées $I_0, I_1, \dots, I_P$. Autrement dit, $I(X) = I_P(I_{P-1}(\dots I_0(X) \dots))$.

Les règles d'interaction du modèle sont regroupées en "étapes", correspondant aux différentes fonctions $I_1, \dots, I_P$. Quant à I_0 , c'est une fonction particulière de réinitialisation des variables déduites : $I_0(V, Y) = V, Y_0$

3.2. Inférence réalisée dans les règles d'interaction

La fonction correspondant à chaque étape est définie par un **automate déterministe** dont le fonctionnement est celui d'un moteur d'inférence simple. Soit $\{R_k\}$ l'ensemble des règles d'une étape donnée (pour alléger les notations, nous omettons l'indice d'étape). La règle R_k est une fonction de $\mathcal{E}$ dans $\mathcal{E}$, qui à tout vecteur d'état X fait correspondre un autre vecteur d'état $R_k(X)$. Chaque règle est en pratique composée d'une **condition** calculée par une fonction booléenne de l'état X , permettant son déclenchement, et **d'actions**, qui sont le plus souvent des instructions d'affectation de variables d'état, par des valeurs constantes ou par des valeurs calculées à partir de X . Il existe un autre type d'action possible, employé uniquement pour l'instant dans la base de connaissances Topase : la résolution d'un système d'équations linéaires, qui permet de calculer une nouvelle valeur de X .

Le moteur d'inférence applique cycliquement les règles (R_k) *ordonnées suivant l'ordre σ* jusqu'à obtenir la même valeur pour X à la fin de deux "tours" successifs des règles.

Appliquer une règle consiste à évaluer sa condition, et, si celle-ci est vraie, réaliser l'action correspondante. En particulier, si la condition n'est pas réalisée, le vecteur X n'est pas modifié.

Ce type de fonctionnement des règles d'interaction permet une grande souplesse de modélisation, notamment pour tout ce qui est propagation de flux dans un système. Il a même permis, dans une première version de l'outil Topase, de calculer les courants et tensions en tout point d'un circuit électrique selon les lois d'Ohm, avant que ce calcul puisse être réalisé de manière bien plus performante par la résolution d'un système d'équations linéaires.

3.3. Sémantique du modèle complet

Les différents éléments d'un modèle Figaro 0 étant décrits, il est facile de définir la sémantique du langage Figaro 0. Cette sémantique équivaut au fonctionnement de l'automate **non déterministe** suivant :

Initialisation :

Un état initial **incomplet** est défini par l'utilisateur par la donnée de V_0 (seule contrainte : respect du domaine de V). Puis calcul de $X_0 = I(V_0, Y_0)$ qui est l'état initial complet du modèle.

Notons au passage que cette façon de définir l'état initial du système permet de résoudre un certain nombre de problèmes épineux pour un modèle complexe : le risque de la définition d'un état incohérent (ex : on a deux composants électriques en série, l'intensité de courant est nulle dans l'un mais pas dans l'autre), voire l'impossibilité pour l'utilisateur de définir l'état initial (ce serait le cas, par exemple, s'il lui fallait donner les courants et tensions en tous points d'un circuit électrique). Ainsi, la définition de l'état initial est facile et sûre pour l'utilisateur : il a peu de variables à initialiser, et ce sont des variables à la signification claire et faciles à déterminer (comme la position d'une vanne, le nombre de composants mis en service à l'instant initial...).

Evolution à partir d'un état courant X :

$O(X)$ définit les groupes de transitions applicables à partir de X .

- Si cet ensemble est vide, l'état courant est absorbant.
- S'il est réduit à un groupe contenant une seule transition t , l'unique état que peut prendre l'automate, à partir de l'état X , est $I(t(X))$.
- S'il contient plusieurs groupes de transitions, deux cas sont à considérer (la décomposition ci-après constitue une partition de l'ensemble des situations possibles) :
- $O(X)$ contient uniquement des transitions de lois continues. Dans ce cas, l'état X est dit "tangibile" et le prochain état que prendra l'automate sera $I(t(X))$, t étant l'une quelconque des transitions de lois continues (non déterminisme).

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

$O(X)$ contient les groupes $G_1, G_2, \dots, G_n$ de transitions de loi discrète. L'état X est alors dit "instantané", et les transitions de lois continues sont **ignorées**. Dans ce cas, le prochain état que prendra l'automate sera $I(t_1 \circ t_2 \circ \dots t_k(X))$, expression dans laquelle t_i représente un choix quelconque (non déterminisme) de transition dans le groupe $G_i : t_i \in G_i$. L'ordre d'application des transitions (autrement dit le choix des indices pour les groupes) n'est pas précisé. Ceci peut produire des problèmes de cohérence, qui sont discutés plus loin.

4. Les éléments lexicaux du langage Figaro

Ce chapitre décrit les différentes règles de base à suivre pour écrire un modèle Figaro correct du point de vue lexicographique en Figaro.

4.1. L'alphabet

L'ensemble des caractères de la table ASCII (7 bits) sont acceptés. Les caractères accentués de la langue française ne sont pas autorisés par l'analyseur syntaxique (sauf dans les commentaires).

Voici quelques définitions de base :

- Les **séparateurs** de mots sont : l'espace, la tabulation, le retour chariot et le saut de page. Ils peuvent apparaître en nombre quelconque.
- Une **lettre** est un des caractères suivants : a...z A...Z _ (souligné).
- Un **chiffre** est un des caractères suivants : 0...9.
- La **fin d'une description de champ** et la **fin de description d'une règle** sont représentées par un point-virgule '; '.
- Le point-virgule est aussi utilisé comme séparateur de haut niveau pour :
 - * marquer la fin de la description du "champ" TYPE (cf. TYPE),
 - * séparer les étapes définies pour une base de connaissances (cf. ORDRE_DES_ETAPES),
 - * séparer les groupes définis pour une base de connaissances (cf. NOMS_DES_GROUPES),
 - * séparer les systèmes d'équations définis pour une base de connaissances (cf. NOMS_DES_SYSTEMES).
- Le séparateur des éléments d'une **liste** est la virgule ','. Les listes sont principalement utilisées :
 - * pour définir plusieurs actions d'une même règle (cf. Action),
 - * pour attribuer un ensemble de valeurs à la facette EDITION d'un champ (cf. EDITION),
 - * pour définir un ensemble d'identificateurs pour une même ETAPE (cf. ETAPE dans Notions avancées).
- Les séparateurs des éléments d'une **énumération** sont l'espace ou la tabulation. Les énumérations sont principalement utilisées :
 - * pour définir les valeurs d'un DOMAINE énuméré (cf. DOMAINE),
 - * pour définir l'ensemble des GROUPES auxquels appartiennent une même règle (cf. GROUPE dans Notions avancées),
 - * pour définir, dans une base de faits, l'ensemble des objets connectés à un même objet dans une interface,
 - * pour définir, dans le cas d'un héritage multivalué, l'ensemble des types dont hérite un même TYPE (cf. SORTE_DE et Héritage).
- Une **chaîne de caractères** s'écrit entre guillemets (") et peut contenir des espaces. Les chaînes de caractères sont uniquement utilisées pour définir le libellé d'un champ (cf. LIBELLE).

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- Une **valeur symbolique** (ou alphanumérique) s'écrit entre apostrophes (') et peut contenir des espaces. Les valeurs symboliques sont uniquement utilisées pour définir les différentes valeurs d'un domaine énuméré (cf. DOMAINE).

- Un **commentaire** commence par '(' et finit par '*'. Tout caractère est permis entre les deux délimiteurs. Les commentaires peuvent être imbriqués.

Exemple

```

TYPE vanne
  SORTE_DE composant; (*le ; désigne la fin du champ TYPE*)
ATTRIBUT
  etat  EDITION      VISIBLE, NON MODIFIABLE, NON OBLIGATOIRE
                        (* liste *)
  LIBELLE           "etat de la vanne"
                        (* chaîne de caractères *)
  DOMAINE           'ouvert' 'ferme' 'inconnu'
                        (* enumeration de valeurs symboliques*)
  PAR_DEFAULT      'ouvert' ;    (*le ; désigne la fin de
                                description du champ etat*)

  (* commentaire
    (* commentaire imbriqué *)
  *)

```

Remarque : Il est interdit de mettre un retour chariot dans une chaîne de caractères (entre "), dans une valeur symbolique (entre '). En revanche, le retour chariot est autorisé dans les commentaires.

Remarque : La taille maximale d'une chaîne ou d'une valeur symbolique est de 200 caractères.

4.2. Les valeurs numériques

- Les **constantes booléennes** sont : VRAI et FAUX (en majuscule).

- Les **constantes numériques** (ou nombres) sont de deux types : entier ou réel. Un entier est un nombre à plusieurs chiffres (précédé du signe '-' si besoin). Un réel a la forme suivante :

<partie entière>.<partie réelle>**e**<puissance de dix>

La puissance de dix est un entier positif ou négatif. La partie entière est un entier positif ou négatif. La partie réelle est un entier sans signe.

Exemples de constantes numériques:

```

25          (* un entier *)
-135        (* un entier négatif *)
256.012     (* un réel *)
-58.5e-5    (* un réel négatif avec une puissance de dix *)

```

4.3. Les identificateurs

Un identificateur permet de donner un nom à un type, un objet, un champ, un opérateur spécifique ou une règle.

On distingue deux possibilités pour décrire un identificateur :

- * Soit par une suite de lettres et de chiffres commençant toujours par une lettre et ne contenant pas d'espaces (on utilisera souvent le caractère _ pour assurer la lisibilité de l'identificateur).
- * Soit par une suite de lettres et de chiffres encadrée par des barres verticales '|'. Dans ce cas, le nom de l'identificateur peut commencer par un chiffre et il peut contenir des espaces et des tabulations (le retour chariot est interdit).

La taille maximale d'un identificateur est de 200 caractères.

Exemples :

```

ABCDEF (* autorisé *)
abcdef (* autorisé. Ces deux symboles sont différents*)
A1234  (* autorisé *)

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

12ABCD (* non autorisé !!! Sera lu comme 12 ABCD *)
|12 AbDEF| (* autorisé *)

Remarque : Pour assurer la lisibilité du modèle Figaro, il est préférable de ne pas utiliser des identificateurs identiques à des mots clés, et de ne pas utiliser deux identificateurs identiques pour représenter des données différentes.

Exemple à ne pas suivre :

```

TYPE composant ;
ATTRIBUT
    attribut DOMAINE ENTIER ; (* DECONSEILLE : attribut (le champ)
                                et ATTRIBUT (le mot-clé) *)

```

4.4. Les mots-clés

Les **mots-clés de Figaro** doivent être écrits en **majuscules**, sinon ils seront considérés comme étant des identificateurs de variables.

4.4.1. Les opérateurs

sens Figaro	terme Français	terme Anglais
opérateur d'assignation	<--	<--
opérateur booléen égalité assignation en Figaro 0	=	=
opérateur booléen différent	<>	<>
opérateur booléen inférieur strictement	<	<
opérateur booléen inférieur ou égal	<=	<=
opérateur booléen supérieur strictement	>	>
opérateur booléen supérieur ou égal	>=	>=
opérateur de multiplication	*	*
opérateur de division	/	/
opérateur d'addition	+	+
opérateur de soustraction	-	-
modulo (a % b désigne a modulo b)	%	%
puissance (a**b désigne a puissance b)	**	**
négation booléenne	NON	NOT
opérateur booléen ET	ET	AND
opérateur booléen OU	OU	OR

4.4.2. Les mots-clés

ALORS	EXP	MODIFIABLE	REPARATION
ATTRIBUT	EXPONENTIELLE	NOMS_DES_GROUPES	REPARE
AU_MOINS	FAIRE	NOMS_DES_SYSTEMES	RESOUDRE_SYSTEME
BOOLEEN	FAUX	NON	SI
CARDINAL	FORMULE	OBJET	SINON

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

CONDITION	GENRE	OBLIGATOIRE	SOIT
CONFIGURATION	GLOBAL	OCCURRENCE	SOMME
CONSTANTE	GROUPE	ON_A	SORTE_DE
DE	IL_EXISTE	ORDRE_DES_ETAPES	SYSTEMES_EQUATIONS
DEFAILLANCE	IL_PEUT_SE_PRODUIRE	OU	TELLE_QUE
DESCRIPTION	INCLUS_DANS	OU_BIEN	TEL_QUE
DES_TERMES	INDISPONIBILITE	PAGE	TEMPS_CONSTANT
DE_TYPE	INFINI	PANNE	TRANSITION
DOMAINE	INS	PARAMETRE_LOI	TYPE
EDITION	INSTANTANEE	PARMI	T_C
EFFET	INTERACTION	PAR_DEFAULT	UN
ENTIER	INTERFACE	POUR_TOUT	UNE
EQUATION	JUSQUA	PRODUIT	VALEURS
EST_UN	LIBELLE	PROVOQUE	VERIFIANT
EST_UNE	LINEAIRE	QQSOIT	VISIBLE
ET	LOI	REEL	VRAI
ETAPE	MARCHE	REINITIALISATION	

5. Les expressions

Dans cette partie, nous allons tout d'abord définir précisément quelles sont les variables du langage Figaro et comment y accéder. Puis nous définirons la syntaxe des expressions en commençant par les expressions simples (semblables à celles des langages de programmation classiques) et en terminant par les expressions utilisant des opérateurs complexes spécifiques au langage Figaro (les quantificateurs, les opérateurs booléens et numériques complexes).

5.1. Préliminaires

5.1.1. Champs valués et variables globales

Les expressions écrites en Figaro vont utiliser principalement deux types de variables :

- * les champs descriptifs des types et des objets (attributs, constantes, ...),
- * les variables globales : ce sont les champs descriptifs des « objets système » (cf. § 7.2).

Les champs valués sont les champs utilisés pour décrire les caractéristiques d'un type ou d'un objet particulier.

```

TYPE rectangle ;
ATTRIBUT
    longueur DOMAINE REEL;
    largeur DOMAINE REEL;
    surface DOMAINE REEL PAR_DEFAULT (longueur*largeur);

```

Dans cet exemple, l'expression *longueur*largeur* est une expression utilisant les attributs du type rectangle.

5.1.2. Définir un ensemble

5.1.2.1. Notions d'ensembles et de variables liées

Comme nous le verrons dans la suite de cette partie, Figaro permet l'utilisation de quantificateurs et d'opérateurs numériques complexes qui s'utilisent sur des **ensembles** afin d'effectuer des tests sur une série d'objets. Cette

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

possibilité est indispensable pour écrire, dans une base de connaissances, des **règles génériques** modélisant le comportement d'un objet quelconque d'un type.

Un **quantificateur logique** se définit comme un symbole qui permet d'utiliser une variable définie sur un ensemble particulier. Il existe deux types de quantificateurs en Figaro :

- * le quantificateur universel : $\forall x \in \text{ensemble } E,$
- * le quantificateur existentiel : $\exists x \in \text{ensemble } E.$

De même, les **opérateurs numériques complexes** somme, produit, minimum et maximum permettent d'effectuer des calculs sur un ensemble de variables :

- $\sum_{x \in \text{ensemble } E} f(x)$
- $\prod_{x \in \text{ensemble}} f(x)$
- même principe pour le minimum et maximum.

En Figaro, les quantificateurs et les opérateurs numériques complexes s'utilisent uniquement sur des **ensembles d'objets**. Le quantificateur (ou l'opérateur) sera utilisé avec une **variable liée** représentant un objet quelconque de l'ensemble.

Exemple

```

TYPE pompe;
ATTRIBUT
    panne DOMAINE BOOLEEN ;

TYPE pompe_normale SORTE_DE pompe;

TYPE pompe_secours SORTE_DE pompe;
ATTRIBUT
    demande_demarrage DOMAINE BOOLEEN ;
INTERFACE
    secourue GENRE pompe_normale CARDINAL 1 JUSQUA INFINI;
INTERACTION
SI QSOIT x UNE secourue ON_A panne DE x = VRAI
ALORS demande_demarrage <-- VRAI ;

```

Cette règle d'interaction se lit :

Soit o un objet quelconque de type pompe_secours,

Si $\forall x \in \{\text{pompes normales secourues par } o\}$, x est en panne, (x est une variable liée)

Alors, il y a demande de démarrage de o.

Pour utiliser des quantificateurs ou des opérateurs numériques complexes en Figaro, il est donc nécessaire de définir des ensembles d'objets. Dans la suite de cette partie, nous allons voir qu'il existe deux moyens principaux de définir un ensemble d'objets en Figaro.

5.1.2.2. Utilisation d'un champ d'interface pour définir un ensemble

Un premier moyen de définir un ensemble d'objets en relation avec un objet d'un type est d'utiliser un **champ interface** du type.

Exemple 1

```

TYPE composant ;
ATTRIBUT
    relie DOMAINE BOOLEEN;
INTERFACE
    amont GENRE composant ;

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

INTERACTION

```
SI IL_EXISTE x UN amont TEL_QUE relie DE x = VRAI
ALORS relie <-- VRAI ;
```

Si o est un objet quelconque de type composant, *amont* désigne l'ensemble des composants reliés à o par la relation amont.

Dans cet exemple, l'ensemble de définition de la variable liée au quantificateur 'IL_EXISTE' est défini par **l'identificateur d'un champ interface**. Ce moyen de définir un ensemble d'objets avec un champ interface est le moyen le plus couramment utilisé en Figaro. Les deux exemples suivants illustrent des définitions plus complexes d'ensembles.

Exemple 2

```
TYPE X;

TYPE Y;
INTERFACE
  i1 GENRE X;

TYPE Z;
INTERFACE
  i2 GENRE Y CARDINAL 1;
INTERACTION
  SI IL_EXISTE x UN i1 DE i2 ....;
```

Si o est un objet quelconque de type Z, l'expression *i1 DE i2* désigne l'ensemble des objets de type X en relation i1 avec l'**unique** objet de type Y en relation i2 avec o.

Dans cet exemple, l'ensemble de définition de la variable liée au quantificateur IL_EXISTE est défini par **'identificateur de champ interface' DE 'ensemble de cardinal 1'**.

Exemple 3

```
TYPE X;

TYPE Y;
  i1 GENRE X;

TYPE Z;
INTERFACE
  i2 GENRE Y;
INTERACTION
  SI IL_EXISTE y UN i2 TEL_QUE (IL_EXISTE x UN i1 DE y TEL_QUE...);
```

Si o est un objet quelconque de type Z :

- * l'expression i2 désigne l'ensemble des objets de type Y en relation i2 avec o,
- * l'expression i1 DE y désigne l'ensemble des objets de type X en relation i1 avec y, un élément de l'ensemble précédent.

Dans cet exemple, l'ensemble de définition de la variable liée au deuxième quantificateur IL_EXISTE est défini par **'identificateur de champ interface' DE variable liée**.

5.1.2.3. Utilisation d'un type pour définir un ensemble

Il existe un autre moyen d'accéder à un ensemble d'objets. En effet, il est parfois intéressant en Figaro de pouvoir adresser tous les objets d'un **type particulier**. C'est pourquoi Figaro permet d'utiliser un nom de type pour définir un ensemble.

Pour définir un ensemble à l'aide d'un nom de type, on utilise une syntaxe de la forme :

$\forall x$ UN OBJET DE_TYPE 'nom de type'

Exemple

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

TYPE composant ;

TYPE pompe ;

INTERACTION

SI QQS0IT x UN OBJET DE_TYPE composant ON_A ...

ALORS ... ;

L'expression *UN OBJET DE_TYPE composant* désigne l'ensemble des objets d'un système instance du type composant.

5.1.2.4. Limite de Figaro

Les ensembles d'objets manipulés en Figaro sont des **ensembles constants**.

En effet, comme nous l'avons évoqué précédemment, les liens entre les objets d'un système (champs interface des objets) n'évoluent pas au cours de la vie du système. Autrement dit, les règles Figaro ne pourront pas ajouter un objet dans une interface ou enlever un objet d'une interface.

De même, il est impossible en Figaro d'ajouter au système un objet instance d'un type (ou de supprimer un objet instance d'un type).

Ainsi, que l'on utilise des champs interface ou des noms de types pour définir un ensemble d'objets, l'ensemble sera constant au cours de la vie du système.

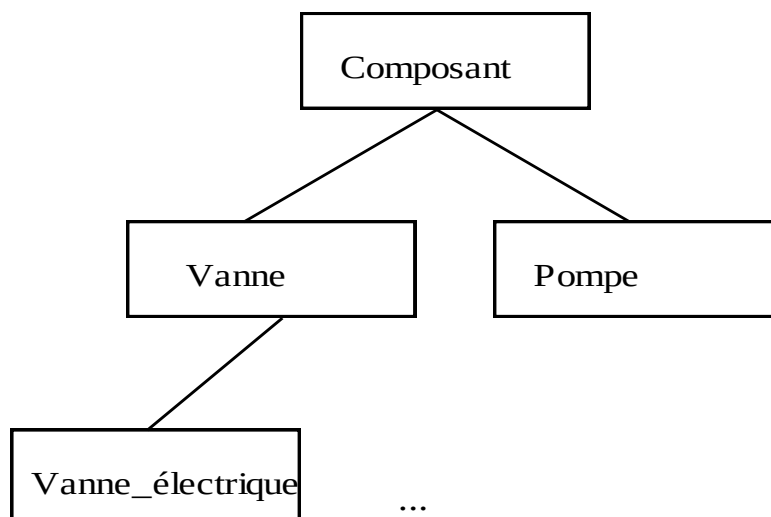
5.1.2.5. Restriction d'un ensemble

Il arrive parfois que l'on veuille **restreindre** l'ensemble des objets d'une interface ou l'ensemble des objets d'un type. Ainsi, il est possible en Figaro d'utiliser des contraintes pour réduire le nombre d'objets de l'ensemble.

Il existe deux contraintes optionnelles en Figaro introduites par les mots-clés DE_TYPE et VERIFIANT. Ces deux contraintes peuvent être utilisées ensemble ou séparément. Elles s'appliquent sur l'ensemble de définition du quantificateur ou de l'opérateur complexe.

La contrainte DE_TYPE :

Cette contrainte permet de restreindre le type des objets de l'ensemble. **Cette contrainte ne peut être utilisée que lorsque l'ensemble est défini avec un champ interface**. En effet, les objets d'une interface sont tous d'un type particulier précisé lors de la déclaration par le mot-clé GENRE. Cependant, il peut être intéressant de réduire cet ensemble d'objets à un sous-type. Prenons comme exemple, la hiérarchie de classes suivante :



Composant est un type générique hérité par les types *vanne* et *pompe*. *Vanne_électrique* est une classe qui spécifie le type *vanne*. Aussi, si le type de l'interface est *composant*, toute instance des 4 types ci-dessus peut être incluse dans l'interface. En effet, puisque ces types héritent de *composant*, ce sont des composants. L'intérêt de la contrainte DE_TYPE apparaît alors.

L'exemple suivant met en évidence l'utilisation de cette contrainte : le type *vanne* est un sous type de *composant* et les objets interfacés avec le type *pompe* sont d'un type *composant*. Dans la règle, on veut uniquement s'intéresser aux objets de type *vanne* qui sont interfacés :

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

```

TYPE composant ;

TYPE vanne SORTE_DE composant;

TYPE pompe SORTE_DE composant;
INTERFACE
    montant GENRE composant ;
INTERACTION
    SI QSOIT x UN montant DE_TYPE vanne ON_A ...
ALORS ... ;

```

Si o est un objet quelconque de type pompe l'expression *montant DE_TYPE vanne* désigne l'ensemble des objets de type vanne (sous-type de composant) en relation montant avec o.

La contrainte VERIFIANT :

Cette contrainte permet de restreindre les objets d'un ensemble d'après une condition. Le principe est simple : l'ensemble est réduit au sous-ensemble ne contenant que les objets vérifiant cette condition.

La condition suivant le mot-clé VERIFIANT doit donc être une expression booléenne (qui renvoie VRAI ou FAUX).

Pour que l'ensemble défini soit un ensemble d'objets constant, l'expression suivant le mot-clé VERIFIANT doit être une **expression constante**, c'est-à-dire ne faisant appel qu'à des champs constants.

La condition peut contenir un nombre quelconque d'expressions booléennes (y compris complexes) reliées par des opérateurs logiques (OU, ET ou NON).

Dans l'exemple suivant, on ne s'intéresse qu'aux objets de l'interface dont le débit dépasse une certaine valeur :

```

TYPE pompe ;
CONSTANTE
    debit DOMAINE REEL;

TYPE composant ;
INTERFACE
    montant GENRE pompe ;
INTERACTION
    SI QSOIT x UN montant VERIFIANT debit DE x > 3000 ON_A ...
ALORS ...

```

Si o est un objet quelconque de type composant, *montant VERIFIANT debit DE x > 3000* désigne l'ensemble des pompes reliées à o par la relation montant, dont le débit est strictement supérieur à la valeur 3000.

5.1.3. Accès à un champ de type ou d'objet

5.1.3.1. Accès aux champs du type décrit

Lorsque, pour décrire un TYPE, on a besoin d'écrire une expression utilisant un champ des objets du type, il suffit de nommer ce champ.

```

TYPE pompe;
ATTRIBUT
    etat DOMAINE 'arrêt' 'marche';
INTERACTION
    SI etat = 'arrêt'
ALORS ....;

```

Si o est un objet quelconque de type pompe, l'expression *etat = arrêt* signifie etat de o = arrêt.

5.1.3.2. Accès aux champs d'une variable liée à un ensemble d'objets

Pour accéder aux objets d'un ensemble, on utilise toujours une variable liée qui représente un objet de l'ensemble (cette variable prendra successivement toutes les valeurs de l'ensemble). Pour accéder aux champs des objets représentés par la variable liée, on utilise l'opérateur **DE**.

Syntaxe :

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

```

champ DE variable_liee
champ (variable_liee)

```

Le mécanisme de l'opérateur est simple. Pour chaque nom d'objet substitué à la variable, il renvoie la valeur du champ de l'objet. Si le champ n'existe pas, il provoque une erreur.

```

TYPE pompe;
INTERFACE
    secours GENRE composant;
ATTRIBUT
    etat DOMAINE 'marche' 'arret';
INTERACTION
    SI IL_EXISTE x UN secours TEL_QUE etat DE x = 'arret'
    ALORS .....;

```

5.1.3.3. Accès au champ d'un objet défini par un ensemble de cardinal 1

Lorsque l'on veut accéder à l'objet d'un ensemble de cardinal 1, l'utilisation d'une variable liée n'est pas nécessaire. On accède à l'objet en nommant simplement l'ensemble. Ainsi, pour accéder aux champs d'un objet appartenant à un ensemble de cardinal 1, on utilise l'opérateur **DE** avec le nom de l'ensemble.

```

TYPE pompe;
INTERFACE
    secours GENRE composant CARDINAL 1;
ATTRIBUT
    etat DOMAINE 'marche' 'arret';
INTERACTION
    SI etat DE secours = 'arret'
    ALORS .....;

```

5.1.3.4. Accès aux champs d'un objet nommé

Dans une base de faits (description du système), pour accéder aux champs d'un objet nommé on utilise l'opérateur **DE**.

```

Syntaxe :
champ DE nom_objet
champ (nom_objet)

```

Cet opérateur permet d'accéder à un champ d'un objet grâce à son nom. Le mécanisme de l'opérateur est simple :

- Il vérifie que l'objet est bien défini dans la base de faits.
- Si l'objet existe, il renvoie la valeur du champ de l'objet en question. Si le champ n'existe pas, il provoque une erreur.
- Si l'objet n'existe pas, l'opérateur provoque une erreur.

```

Exemple :
debit DE v1

```

5.1.3.1. Accès aux variables globales

C'est un cas particulier de ce qui est décrit au paragraphe précédent. La grande différence entre les champs des objets système et les autres champs d'objets est que l'on peut faire directement référence à eux dans tous les **types** du modèle.

```

TYPE global ;
ATTRIBUT
    reparable DOMAINE BOOLEEN PAR_DEFAULT VRAI;

OBJET_SYSTEME parametres_globaux EST_UN global ;

TYPE pompe;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' 'panne';

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

OCCURRENCE

```
SI etat = 'panne' ET reparable(parametres_globaux)
IL_PEUT_SE_PRODUIRE REPARATION.....;
```

5.2. Expressions simples

On entend par expressions simples, des expressions sans opérateurs complexes spécifiques au langage Figaro. Rappelons certaines définitions :

1. Une expression est une combinaison **d'expressions élémentaires**.

Exemple d'une expression :
`a = (b + d) * ((r + z) >= 3.5 ET NON q)`

2. D'une façon générale, une expression élémentaire comporte un opérateur et un ou deux opérandes.

Exemples d'expressions élémentaires :
`b + d`
`r + z`
`NON q`
`a = e`

3. Une expression élémentaire est dite numérique si l'opérateur est arithmétique. Sinon, l'expression est dite logique.

Exemples d'expressions numériques :
`b + d`
`r + z`
 Exemples d'expressions logiques :
`NON q`
`a = e`

4. On appelle *domaine* d'un opérande **l'ensemble des valeurs possibles** de l'opérande. En Figaro, il existe 4 domaines possibles : ENTIER, REEL, BOOLEEN et énuméré. Si un champ se voit affecter une valeur en dehors de son domaine, Figaro provoque une erreur.

Exemples de champs avec domaine :
 TYPE pompe ;
CONSTANTE
`a DOMAINE ENTIER`
`PAR_DEFAULT 3 ;`
`b DOMAINE REEL`
`PAR_DEFAULT -12e-3 ;`
`c DOMAINE BOOLEEN`
`PAR_DEFAULT VRAI ;`
`d DOMAINE 'marche' 'arret' (* domaine énuméré *)`
`PAR_DEFAULT 'arret' ;`

Nous allons étudier les différentes opérations disponibles en Figaro.

5.2.1. Opérations arithmétiques

Opérateurs :

Les opérateurs arithmétiques disponibles sont les opérateurs classiques :

```
+ addition
- soustraction
* multiplication
/ division
** exponentiation
% modulo
```

Opérandes :

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Ces opérateurs sont binaires, c'est-à-dire qu'ils nécessitent deux opérandes. Ces opérandes sont soit des entiers, soit des réels. L'opérateur de soustraction est unaire lorsque l'opérande à gauche est un opérateur ou qu'il n'y a pas d'opérande à gauche (par exemple, -3). Dans ce cas, il n'agit que sur l'opérande de droite.

Exemples :

```

5 - 15      (* opérateur - binaire *)
5 + -15     (* opérateur - unaire : le résultat est le même *)
1e-4 * 123
5 % 3       (* 5 modulo 3 : le résultat est 2 *)
```

Domaines :

Lorsque deux opérandes n'ont pas le même domaine dans une expression arithmétique, le résultat de l'expression est une valeur qui a pour domaine le plus "grand" domaine des deux opérandes sachant que :

```
domaine(réel) > domaine(entier)
```

Ainsi, le domaine de l'expression suivante :

```
1.8e-4 * 123
```

est réel car 1.8e-4 est un réel et 123 est un entier. Donc, l'expression est du plus grand domaine, soit réel.

Utilisation particulière de l'opérateur de multiplication * :

Il peut être intéressant d'utiliser des opérandes booléens dans une opération arithmétique avec l'opérateur multiplication (et seulement avec cet opérateur). Dans ce cas, les valeurs booléennes sont converties en valeurs entières :

```

VRAI  est remplacé par  1
FAUX  est remplacé par  0
```

Ainsi :

```

      VRAI * (5 > 6) * 3
est égal à ( à cause de *, on convertit VRAI )
      1   *   FAUX   * 3
est égal à ( à cause de *, on convertit FAUX )
      1   *    0    * 3
```

C'est l'opérateur arithmétique de multiplication qui force Figaro à transformer les booléens en entier sachant que :

```
domaine(réel) > domaine(entier) > domaine(booléen)
```

Donc un booléen sera d'abord converti en entier et si cela ne suffit pas, il sera converti en réel.

Par exemple, l'expression suivante permet de déterminer le montant de l'amende à payer suivant que l'on a commis une petite_infraction (17€), une infraction_moyenne (100€), ou une infraction ni petite ni moyenne :

```

17 * petite_infraction + 100 * infraction_moyenne +
1000 * (NON (petite_infraction OU infraction_moyenne))
```

5.2.2. Opérations booléennes

Les opérateurs booléens (c'est-à-dire qui retournent un résultat booléen VRAI ou FAUX) sont divisés en deux catégories : les opérateurs de comparaison et les opérateurs logiques.

Opérateurs de comparaison :

```

=   égal
<> différent
```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

< inférieur
 <= inférieur ou égal
 > supérieur
 >= supérieur ou égal

Opérandes :

Les opérateurs de comparaison sont binaires. Leurs opérandes sont soit des entiers, soit des réels, soit uniquement pour les opérateurs = et <>, des valeurs symboliques (entre ') ou des booléens.

Exemples :

```

5 >= 6      (* toujours FAUX *)
a < 10      (* a ne peut être qu'entier ou réel *)
etat = 'arrêt' (* VRAI ou FAUX selon la valeur de la variable etat*)

```

Domaines :

Le résultat est toujours booléen. Si les opérandes ont des domaines différents, le domaine de l'opérande de plus petit domaine sera transformé en domaine de l'opérande de plus grand domaine uniquement si les domaines sont ENTIER ou REEL. Sinon, Figaro provoquera une erreur.

Exemple :

```

5 >= 6.2e-3      (* 5 est transformé en réel *)

```

La comparaison entre des opérandes symboliques et numériques est interdite. Aucune conversion ne peut être effectuée.

Exemples :

```

'arrêt' < 'marche' (* interdit *)
'arrêt' < 5        (* interdit *)

```

Opérateurs logiques :

OU
NON
ET

Opérandes :

Seul l'opérateur de négation est unaire (il ne prend qu'une seule opérande à droite). Les autres sont binaires. Les opérandes ne peuvent être que des booléens. Chacun de ces opérateurs renvoie un booléen.

Exemples :

```

rupture OU court_circuit
(* les variables doivent être booléennes *)
NON ouvert

```

Domaines :

Le calcul des domaines ne pose aucun problème car tout est booléen. Il est donc clair qu'une opération logique entre des opérandes non booléennes provoquera une erreur. En effet, Figaro ne peut effectuer de conversion d'un non booléen vers un booléen.

Exemples :

```

a < 10 OU 123 (* interdit *)
a < 10 ET b > 4 (* OK *)
rupture OU NON ( a < 13)
(* OK si rupture est une variable booléenne
et a une variable entière ou réelle *)

```

5.2.3. Priorités entre les opérateurs

Le tableau suivant présente les priorités d'évaluation qui s'appliquent sur les opérateurs. Un calcul d'expression en Figaro est effectué dans l'ordre des priorités des opérateurs. C'est à dire que Figaro commencera par évaluer

les expressions élémentaires avec des opérateurs plus prioritaires jusqu'à évaluer les expressions avec les opérateurs moins prioritaires. En cas d'égalité sur les priorités, la priorité est à l'opérateur situé le plus à gauche.

**	Plus prioritaire
* /	
+ -	.
= > >= < <= <>	.
NON	.
ET	.
OU	Moins prioritaire

L'utilisation des parenthèses permet de gérer l'ordre d'évaluation de l'expression. Mieux vaut en mettre trop que pas assez et se tromper sur la signification d'une expression !

Exemples :

```
10 * a - 3 / 2
(* est évaluée comme (10*a) - (3/2) *)
NON p OU a <= 3 * 2
(* est évaluée comme (NON p) OU (a <= (3*2)) *)
2/3*4
(* est évalué comme (2/3)*4 *)
2-3+4
(* est évalué comme (2-3)+4 *)
```

5.3. Les quantificateurs

Comme nous l'avons déjà vu, un **quantificateur logique** se définit comme un symbole qui permet d'utiliser une variable définie sur un ensemble particulier. Il existe deux types de quantificateurs en Figaro :

- * le quantificateur universel : $\forall x \in \text{ensemble } E,$
- * le quantificateur existentiel : $\exists x \in \text{ensemble } E.$

Comme il existe plusieurs façons d'utiliser ces deux types de quantificateurs en Figaro, chacun de ces deux types correspond à plusieurs quantificateurs Figaro.

- * les quantificateurs Figaro IL_EXISTE, IL_EXISTE AU_MOINS et SOIT correspondent à $\exists$,
- * les quantificateurs Figaro QQSOIT et POUR_TOUT correspondent à $\forall$.

5.3.1. Le quantificateur IL_EXISTE

Syntaxe :

```
IL_EXISTE variable
  UN | UNE      nom_d_ensemble
  [ DE_TYPE     type ]
  [ VERIFIANT   condition ]
  TEL_QUE | TELLE_QUE expression_bouleanne
```

Cet opérateur correspond au quantificateur $\exists$ de la logique des prédicats. Il permet de tester si **au moins un** objet de l'ensemble vérifie une condition particulière (facette TEL_QUE ou TELLE_QUE) qui est représentée par une expression booléenne :

```
IL_EXISTE x UN <ensemble d'objets> TEL_QUE <condition>
```

Le mécanisme de cet opérateur est le suivant :

- Si l'ensemble est non vide, on évalue l'expression booléenne de la condition TEL_QUE pour chaque objet de l'ensemble. Si au moins un résultat est VRAI (OU logique entre les résultats), le résultat de l'opérateur est VRAI. Sinon (tous les résultats sont FAUX), le résultat de l'opérateur est FAUX.
- Si l'ensemble est vide, le résultat est FAUX.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

L'expression booléenne peut être une combinaison d'expressions booléennes simples ou complexes.

Remarque : La variable d'ensemble est liée à l'opérateur. Elle peut donc être utilisée uniquement :

- dans les contraintes pour restreindre l'ensemble de définition du quantificateur,
 - dans la condition TEL_QUE.
- mais pas dans le reste de la règle.

Exemple :

```

TYPE vanne ;
CONSTANTE
    debit DOMAINE REEL ;
ATTRIBUT
    position DOMAINE 'ouvert' 'ferme' PAR_DEFAULT 'ouvert' ;

TYPE pompe ;
INTERFACE
    montant GENRE vanne ;
INTERACTION
SI IL_EXISTE x UN montant
        TEL_QUE (debit DE x > 1000 ET position DE x = 'ouvert')
ALORS ...

```

L'exemple précédent peut s'écrire différemment grâce à la contrainte VERIFIANT. En effet, le champ débit du type vanne est une constante (si c'était un attribut, ce serait refusé par le *parser*).

Exemple :

```

TYPE vanne ;
CONSTANTE
    debit DOMAINE REEL ;
ATTRIBUT
    position DOMAINE 'ouvert' 'ferme' PAR_DEFAULT 'ouvert' ;

TYPE pompe ;
INTERFACE
    montant GENRE vanne ;
INTERACTION
SI IL_EXISTE x UN montant
        VERIFIANT debit DE x > 1000
        (* utilisation de la contrainte VERIFIANT *)
        TEL_QUE (position DE x = 'ouvert')
ALORS ...

```

5.3.2. Le quantificateur QQSOIT

Syntaxe :

```

QQSOIT variable
    UN | UNE      nom_d_ensemble
    [ DE_TYPE      type ]
    [ VERIFIANT    condition ]
    ON_A expression_booléenne

```

Cet opérateur correspond au quantificateur $\forall$ de la logique des prédicats. Il permet de tester si **tous** les objets de l'ensemble vérifient une condition particulière (facette ON_A) qui est représentée par une expression booléenne :

```

QQSOIT x UN <ensemble d'objets> ON_A <condition>

```

Le mécanisme de cet opérateur est le suivant :

- Si l'ensemble est non vide, on évalue l'expression booléenne de la condition ON_A pour chaque objet de l'ensemble. Si tous les résultats sont VRAIs (ET logique entre les résultats), le résultat de l'opérateur est VRAI. Sinon (au moins un des résultats est FAUX), le résultat de l'opérateur est FAUX.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- Si l'ensemble est vide, le résultat est VRAI. Attention à ce cas particulier, si on l'oublie, cela peut être la source d'une erreur !

Exemple :

```

TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'arret' 'marche' PAR_DEFAULT 'marche' ;

TYPE pompe_secours ;
INTERFACE
    pompe_secourue GENRE pompe;
INTERACTION
SI QQSOIT x UNE pompe_secourue
    ON_A etat DE x = 'arret'
ALORS ...

```

5.3.3. Le quantificateur IL_EXISTE AU MOINS

Syntaxe :

```

IL_EXISTE AU MOINS      expression_entière
                           variable
                           INCLUS_DANS nom_d_ensemble
                           [ DE TYPE type ]
                           [ VERIFIANT condition ]
                           TEL_QUE | TELLE_QUE expression_booléenne

```

Cet opérateur correspond au quantificateur k sur n . Il permet de tester si **au moins** k (exprimé par l'expression entière) objets de l'ensemble vérifient une condition particulière (facette TEL_QUE ou TELLE_QUE) qui est représentée par une expression booléenne :

```
IL_EXISTE AU MOINS k x INCLUS_DANS <ensemble d'objets> TEL_QUE <condition>
```

Le mécanisme de cet opérateur est le suivant :

- Si l'ensemble est non vide (c'est-à-dire $n \neq 0$), on évalue l'expression entière représentant la valeur k .

* Si $k = 0$, le résultat est VRAI.

* Si $k > n$, le résultat est FAUX.

* Si $0 < k \leq n$, on évalue l'expression booléenne de la condition TEL_QUE pour chaque objet de l'ensemble. Si au moins k expressions sont évaluées à VRAI, le résultat est VRAI. Sinon (au moins $n - k$ expressions sont évaluées à FAUX) le résultat est FAUX.

- Si l'ensemble est vide (c'est-à-dire $n = 0$), le résultat est FAUX.

Exemple :

```

TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'arret' 'marche' PAR_DEFAULT 'marche' ;

TYPE pompe_secours ;
ATTRIBUT
    K      DOMAINE ENTIER
           PAR_DEFAULT 2 ;
INTERFACE
    pompe_secourue GENRE pompe ;
INTERACTION
(* La pompe de secours ne se déclenche que si AU_MOINS K pompes
   sont à l'arret *)
SI IL_EXISTE AU MOINS K x      INCLUS_DANS pompe_secourue
    TELLE_QUE etat DE x = 'arret'

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

ALORS ...

5.3.4. Le quantificateur POUR_TOUT

Ce quantificateur sera présenté dans le paragraphe consacré aux actions des règles Figaro (cf. POUR_TOUT).

5.3.5. Le quantificateur SOIT

Ce quantificateur peut être assimilé au quantificateur universel $\exists$, mais il aura un rôle différent du quantificateur Figaro IL_EXISTE. Il sera présenté dans le paragraphe consacré aux règles d'interaction Figaro (cf. SOIT).

5.3.6. Combinaison de quantificateurs

Lors de l'utilisation d'opérateurs complexes imbriqués, il n'est pas évident de se rendre compte du résultat que Figaro produira. Aussi, il est possible de considérer que tout quantificateur correspond à une succession de conditions liées par un opérateur logique. En effet, les résultats des évaluations des conditions des quantificateurs sont liés :

- par un OU pour le quantificateur IL_EXISTE
- par un ET pour le quantificateur QQSOIT.

Lors d'imbrications de quantificateurs, il peut être intéressant de substituer les quantificateurs par leur combinaison d'expressions sur un exemple d'objets pour se faire une idée du résultat (cela revient à faire mentalement une instanciation en Figaro d'ordre 0).

Par exemple, si on considère les ensembles d'objets suivants :

```
X = { x1 x2 }
Y = { y1 y2 y3 }
```

et la règle suivante :

```
SI  $\exists$  x UN X / ( $\forall$  y UN Y ON_A x <> y) ALORS ...
```

cette règle équivaut à :

```
SI ( $\forall$  y UN Y ON_A x1 <> y) OU ( $\forall$  y UN Y ON_A x2 <> y) ALORS ...
```

qui est équivalent à

```
SI (x1 <> y1 ET x1 <> y2 ET x1 <> y3) OU
(x2 <> y1 ET x2 <> y2 ET x2 <> y3) ALORS ...
```

Grâce à cette substitution, il est facile de se faire une idée du résultat.

Il faut particulièrement faire attention à la portée des variables liées lors des imbrications d'opérateurs complexes. En effet, la portée d'une variable liée est réduite à l'intérieur du quantificateur qui définit l'ensemble auquel appartient cette variable. L'exemple suivant provoque une erreur car la variable x (définie dans l'opérateur IL_EXISTE) n'est pas définie dans le quantificateur QQSOIT :

```
Exemple :
SI IL_EXISTE x UN X (* fin de l'opérateur IL_EXISTE *)
  ET QQSOIT y UN Y ON_A x <> y (* ERREUR : x n'est pas défini ! *)
```

Il faut écrire :

```
SI IL_EXISTE x UN X TEL_QUE
  QQSOIT y UN Y ON_A x <> y (* x est défini dans IL_EXISTE *)
```

5.4. Les opérateurs booléens

Ils sont au nombre de quatre : un opérateur de gestion d'ensemble (INCLUS_DANS), un opérateur de gestion d'expression (AU_MOINS ... PARMI) et deux opérateurs de test de panne (MARCHE et PANNE).

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

5.4.1. L'opérateur INCLUS_DANS

Syntaxe :
variable **INCLUS_DANS** nom_d_ensemble

Cet opérateur permet de tester l'appartenance d'un objet à un ensemble. Pour cela, on précise à gauche de l'opérateur la variable objet et, à droite, un ensemble d'objets représenté par une variable d'interface ou un nom de type. Dans le cas d'un nom de type, on teste simplement si l'objet est du type précisé ou bien si l'objet est d'un type qui hérite du type spécifié.

Si l'objet apparaît dans l'ensemble des objets, le résultat sera VRAI. Sinon il sera FAUX. Si l'ensemble est vide, le résultat est FAUX.

Dans l'exemple suivant, on teste si toutes les pompes du système sont incluses dans l'interface de l'objet.

```
Exemple :
TYPE composant ;
INTERFACE
    montant GENRE pompe ;
INTERACTION
SI QSOIT x UN_OBJET DE_TYPE pompe
            ON_A x INCLUS_DANS montant
ALORS ...
```

Remarque : Il n'est pas possible d'imposer des contraintes d'ensemble sur l'opérateur INCLUS_DANS.

5.4.2. L'opérateur MARCHE

Syntaxe :
MARCHE [**DE** objet]
MARCHE [(objet)]

L'opérateur MARCHE teste si un objet est en marche, c'est-à-dire si toutes les variables PANNE de l'objet sont à FAUX (cf. PANNE (variable d'état)). L'opérateur MARCHE permet d'accéder à un objet de la même façon que l'opérateur DE :

- Si aucun objet n'est précisé, l'opérateur considère qu'il s'agit de l'objet dans lequel l'opérateur est utilisé.
- L'objet sur lequel doit agir l'opérateur peut être identifié soit par une variable liée, soit par un nom d'ensemble de cardinal 1, soit par un nom d'objet particulier.

L'opérateur effectue les opérations suivantes :

- Il récupère tous les champs PANNE de l'objet.
- Il teste si tous ces champs (booléens) sont à FAUX. Si c'est le cas, le résultat de l'opération est VRAI. Sinon (au moins un des champs PANNE est VRAI), le résultat est FAUX.
- Si l'objet n'a aucun champ PANNE, le résultat est VRAI.

L'exemple suivant teste si une pompe est alimentée par au moins une source et ce, uniquement si la pompe fonctionne.

```
Exemple :

TYPE pompe;
INTERFACE
    montant GENRE source ;
PANNE
    panne_pompe ;
EFFET alim ;
INTERACTION
    SI MARCHE ET
        IL_EXISTE x UN montant TEL_QUE alim DE x
    ALORS
        alim ;
```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

5.4.3. L'opérateur PANNE

Syntaxe :

PANNE [**DE** objet]

PANNE [(objet)]

L'opérateur PANNE est le complémentaire de l'opérateur MARCHE. Il teste si au moins un des attributs PANNE (cf. PANNE (variable d'état)) de l'objet est à VRAI. Dans ce cas, il renvoie VRAI. L'opérateur PANNE permet d'accéder à un objet de la même façon que l'opérateur DE :

- Si aucun objet n'est précisé, l'opérateur considère qu'il s'agit de l'objet dans lequel l'opérateur est utilisé.
- L'objet sur lequel doit agir l'opérateur peut être identifié soit par une variable liée, soit par un nom d'ensemble de cardinal 1, soit par un nom d'objet particulier.

L'opérateur effectue les opérations suivantes :

- Il récupère le nom de tous les champs PANNE de l'objet
- Il teste si au moins un des champs (booléens) est à VRAI. Si c'est le cas, le résultat de l'opération est VRAI. Sinon (tous les champs PANNE sont FAUX), le résultat est FAUX.
- Si l'objet n'a aucun champ PANNE, le résultat est FAUX.

L'exemple suivant concerne une pompe de secours qui ne démarre que si la pompe secourue est en panne.

Exemple :

```

TYPE pompe_secours;
INTERFACE
    pompe_secourue GENRE pompe CARDINAL 1 ;
ATTRIBUT
    etat DOMAINE 'arret' 'marche' 'demande_demarrage'
        PAR_DEFAULT 'arret' ;
INTERACTION
    (* On essaie de démarrer si la pompe secourue est en panne *)
    SI MARCHE ET etat = 'arret'
        ET PANNE( pompe_secourue )
    ALORS
        etat <-- 'demande_demarrage' ;

```

5.4.4. L'opérateur AU_MOINS ... PARMI

Syntaxe :

AU_MOINS expression_entière **PARMI** (liste_expressions_booléennes)

Cet opérateur permet de tester si au moins k expressions de la liste sont vérifiées. Pour cela, on précise la valeur de k par une expression entière et la liste des expressions booléennes à tester (les expressions sont séparées par des virgules). Si au moins k expressions sont évaluées à VRAI, le résultat sera VRAI. Sinon il sera FAUX. Si la valeur de k est égale à 0, le résultat est VRAI. Si la valeur de k est supérieure au nombre d'expressions, le résultat est FAUX.

Dans l'exemple suivant, on teste si au moins 2 des composants en interface sont en panne.

Exemple :

```

TYPE composant ;
INTERFACE
    composant1 GENRE composant CARDINAL 1;
    composant2 GENRE composant CARDINAL 1;
    composant3 GENRE composant CARDINAL 1;
INTERACTION
    SI AU_MOINS 2 PARMI (    PANNE(composant1),
                                PANNE(composant2),
                                PANNE(composant3) )
    ALORS ...

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

5.5. Opérateurs numériques complexes

Figaro dispose aussi d'opérateurs numériques complexes agissant sur des ensembles, c'est-à-dire des champs d'interface ou des noms de type. Ces opérateurs sont principalement utilisés pour écrire des équations en Figaro (cf. Chapitre Equations).

Remarque : Comme pour les opérateurs booléens complexes, il est possible de préciser des contraintes sur l'ensemble d'objets traités par l'opérateur (DE_TYPE et VERIFIANT) pour ne traiter qu'un sous-ensemble d'objets.

Les opérateurs numériques complexes sont au nombre de cinq : CARDINAL, SOMME, PRODUIT, MINIMUM, MAXIMUM.

5.5.1. L'opérateur CARDINAL

Syntaxe :

```
CARDINAL  DE champ_d_interface
CARDINAL  ( champ_d_interface )
```

L'opérateur CARDINAL retourne le nombre d'éléments d'un ensemble. Appliqué à une interface, il renvoie le nombre d'objets que contient le champ interface.

- Si l'interface est monovaluée, le résultat est 1.
- Si l'interface ne contient pas d'objets, le résultat est 0.

Il est possible de récupérer le CARDINAL d'une interface d'un objet voisin grâce à l'opérateur DE ou ().

Dans l'exemple suivant, on modélise le comportement d'une porte logique ET. On vérifie que la porte logique ET est reliée à au moins deux composants en *amont*. On vérifie aussi que tous les objets de l'interface *amont* de type *porte_logique*, ont une interface *aval* à un seul élément.

Exemple :

```
TYPE composant ;

TYPE porte_logique SORTE_DE composant ;
INTERFACE
    aval GENRE composant;

TYPE porte_ET SORTE_DE porte_logique;
INTERFACE
    amont GENRE composant ;
ATTRIBUT
    etat DOMAINE 'valide' 'pas valide' PAR_DEFAULT 'valide';
INTERACTION
    SI ( CARDINAL DE amont < 2 )
    OU ( IL_EXISTE x UN amont DE_TYPE porte_logique
        TEL_QUE CARDINAL DE aval DE x <> 1 )
    ALORS etat <-- 'pas valide' ;
```

Remarque : Cet opérateur est à différencier de la facette CARDINAL accessible pour la description de l'interface d'un objet.

5.5.2. L'opérateur SOMME

Syntaxe :

```
SOMME POUR_TOUT variable UN | UNE nom_d_ensemble
    [ DE_TYPE type ]
    [ VERIFIANT condition ]
DES_TERMES expression_numérique
```

L'opérateur SOMME permet d'effectuer une somme d'expressions sur un ensemble d'objets.

Il utilise une variable liée définie sur un ensemble d'objets (facette POUR_TOUT) et il additionne successivement, pour chaque objet de l'ensemble, la valeur de l'expression numérique (facette DES_TERMES) pour cet objet :

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

SOMME POUR_TOUT x **UN** <ensemble d'objets> **DES_TERMES** <expression numérique>

Exemple :

```

TYPE vanne ;
CONSTANTE
    debit_max DOMAINE ENTIER PAR_DEFAULT 2500 ;
INTERFACE
    amont GENRE tuyau ;
INTERACTION
    SI ( SOMME POUR_TOUT x UN amont DES_TERMES debit DE x ) > debit_max
    ALORS ... ;

```

Le mécanisme de l'opérateur est le suivant :

- Si l'ensemble est non vide, on évalue l'expression numérique **DES_TERMES** pour chaque objet de l'ensemble. Le résultat de l'opérateur est la somme des résultats des expressions.
- Si l'ensemble est vide, le résultat est 0. Attention à ne pas oublier ce cas particulier !

La facette **DES_TERMES** permet de préciser quels sont les calculs que nous voulons effectuer et additionner. L'expression doit être numérique pour pouvoir être additionnée.

5.5.3. L'opérateur PRODUIT

Syntaxe :

```

PRODUIT POUR_TOUT variable UN | UNE nom_d_ensemble
                                [ DE_TYPE type ]
                                [ VERIFIANT condition ]
DES_TERMES expression_numérique

```

L'opérateur **PRODUIT** permet d'effectuer un produit d'expressions sur un ensemble d'objets.

Il utilise une variable liée définie sur un ensemble d'objets (facette **POUR_TOUT**) et il calcule le produit, pour tous les objets de l'ensemble, des valeurs de l'expression numérique donnée dans la facette **DES_TERMES** :

PRODUIT POUR_TOUT x **UN** <ensemble d'objets> **DES_TERMES** <expression numérique>

L'exemple suivant montre une utilisation de cet opérateur dans l'initialisation par défaut d'un attribut.

Exemple :

```

TYPE calcul ;
INTERFACE
    amont GENRE composant ;
ATTRIBUT
    valeur DOMAINE ENTIER
            PAR_DEFAULT PRODUIT POUR_TOUT x UN amont
            DES_TERMES ( a DE x ) + 5

```

Le mécanisme de l'opérateur est le suivant :

- Si l'ensemble est non vide, on évalue l'expression numérique **DES_TERMES** pour chaque objet de l'ensemble. Le résultat de l'opérateur est le produit des résultats des expressions.
- Si l'ensemble est vide, le résultat est 1. Attention à ne pas oublier ce cas particulier !

La facette **DES_TERMES** permet de préciser quels sont les calculs que nous voulons effectuer et multiplier. L'expression doit être numérique pour pouvoir être multipliée. Elle peut aussi être booléenne car l'opérateur utilisé pour lier les résultats est l'opérateur de multiplication qui peut effectuer une conversion des booléens en entier.

5.6. Opérateurs spécifiques

Il est possible dans l'Atelier Figaro de définir des opérateurs spécifiques, afin de gérer des problèmes particuliers associés à une base de connaissances (exemples : exp, sin, cos, tan, max, ...).

La liste des opérateurs spécifiques s'allonge au gré des besoins. Cette liste est actualisée dans la note définissant la syntaxe Figaro.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

L'utilisation de tels opérateurs suit les règles suivantes :

```
opérateur ( expression )
ou
opérateur ( liste_expressions )
```

Remarque : Dans une liste d'expressions, les expressions sont séparées par des virgules.

Exemples :

```
EXP ( 2 - ( a * 3 ) )
MAX ( a, b, c )
```

Ces opérateurs sont définis avec une arité particulière (de 1 à infini).

5.7. Portée d'une variable

Comme dans tout langage de programmation, les variables en Figaro ont une portée. Cela signifie qu'elles sont accessibles jusqu'à un certain point.

Suivant sa nature (variable d'OBJET_SYSTÈME, champ d'objet, variable liée), chaque variable a une portée différente et une priorité d'évaluation différente (c'est-à-dire l'ordre dans lequel la valeur de la variable est recherchée dans chacun des contextes : global, d'objet ou local à un opérateur complexe).

La portée d'une variable d'OBJET_SYSTÈME est globale : tous les types de la base de connaissances pourront y accéder.

Un champ de type est associé à un type. Sa portée est locale au type, c'est-à-dire qu'elle est inconnue et inaccessible par les autres types autrement que via une interface ou une chaîne d'interfaces du genre `a_gauche(en_haut(a_droite))`.

Enfin, une variable liée est une variable définie dans un opérateur complexe comme QQSOIT, IL_EXISTE, IL_EXISTE AU_MOINS, SOMME, PRODUIT, MINIMUM, MAXIMUM, POUR_TOUT et SOIT. En effet, ces opérateurs permettent de définir une variable qui prend successivement certaines valeurs afin d'effectuer certains calculs. Cette variable est liée à l'opérateur, c'est-à-dire que sa portée est interne à l'opérateur. En dehors de celui-ci, la variable n'est plus connue.

Maintenant que la notion de portée est plus précise, il nous faut définir l'ordre dans lequel Figaro recherche la valeur d'une variable. En effet, il est possible de surcharger le nom d'une variable. C'est à dire que l'on peut nommer une variable liée comme le champ d'un type. Dans ces cas, la priorité est donnée à la variable qui a la portée la plus réduite. Ainsi, Figaro recherchera d'abord (grâce au nom de la variable) la valeur de la variable dans le contexte des variables liées. Ensuite, s'il échoue, il passera au contexte des champs du type (contexte local).

Exemple :

```
TYPE vanne
ATTRIBUT
  x DOMAINE BOOLEEN PAR_DEFAULT VRAI;
INTERACTION
  SI QQSOIT x UN composant
    ON_A propriété(x)
  ALORS x <-- VRAI ;
```

La variable `x` est liée à l'instruction QQSOIT. Dans l'instruction propriété de `x`, le `x` trouvé est celui du QQSOIT. Dans le ALORS, il s'agit du champ de l'objet.

Remarque : Il est conseillé de ne pas déclarer de variables qui ont le même nom qu'un type pour faciliter la lecture de la base de connaissances.

6. Description d'un type Figaro

Comme nous l'avons vu au chapitre 2, une base de connaissances est un ensemble de définitions de TYPE. Dans ce chapitre, nous présentons de façon détaillée comment décrire un TYPE en Figaro.

Remarque : on trouvera en Annexe 2 un schéma de principes représentant la structure générale d'une base de connaissances.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

6.1. Structure générale d'une définition de classe

6.1.1. Déclaration d'un TYPE

Syntaxe :

```
TYPE nom_type
  [SORTE_DE énumération_de_types];
```

Un type Figaro représente une famille d'objets possédant des caractéristiques communes.

La description d'une classe commence par le mot-clé **TYPE** suivi du nom (de l'identifiant) de la classe.

```
TYPE moto_pompe ;
```

L'héritage est spécifié par une facette **SORTE_DE** qui peut être multivaluée (héritage multiple), et dans laquelle on énumère les surclasses de la classe décrite.

```
TYPE moto_pompe
  SORTE_DE composant pompe ;
```

Contrairement aux types, un objet ne peut hériter que d'un seul type.

6.1.2. Structure générale de description d'un TYPE

Les caractéristiques associées à un TYPE se décomposent en deux grands groupes :

- Les champs valués représentent les caractéristiques **statiques** d'un TYPE.

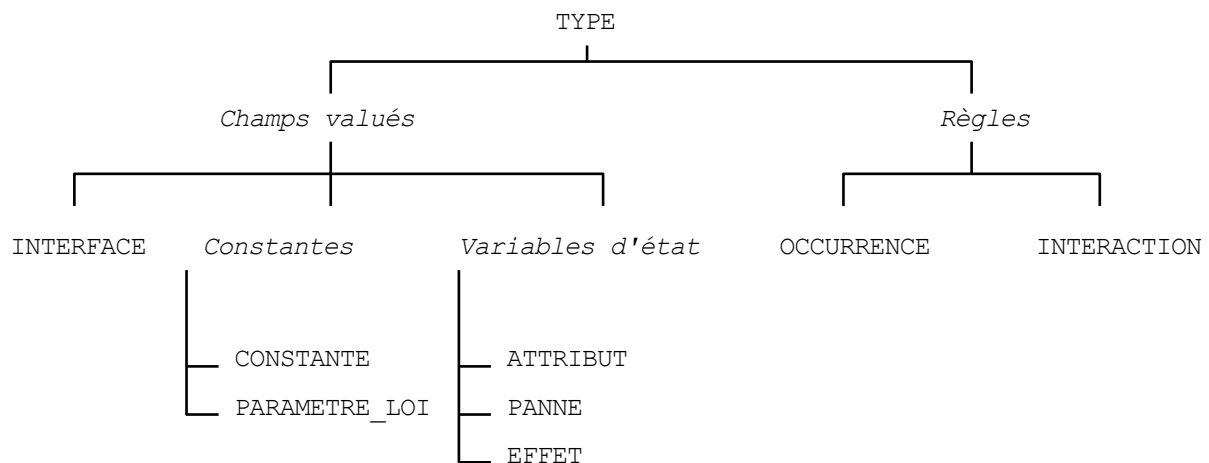
Exemple :

L'interface (classes en interface)

Variables d'état du composant

- Les règles de fonctionnement (règles d'occurrence et d'interaction) permettant de décrire le comportement **dynamique** d'un TYPE, c'est-à-dire :
 - Les événements pouvant survenir pour un objet d'un type (changement d'état, défaillance, réparation, ...)
 - Les conditions d'occurrence de ces événements.
 - Les conséquences de ces événements sur le système.

Le schéma suivant présente l'organisation des caractéristiques associées à un TYPE.



EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

6.2. Les champs valués

On distingue trois catégories de champs valués pour un type :

- * Les **champs constants** représentent les caractéristiques des objets qui n'évoluent pas au cours de la vie du système (longueur d'un câble de connection par exemple). Ces champs sont univalués.
- * Les **champs d'interface** sont des champs objets contenant des noms d'objets. La valeur de ces champs n'évoluent pas au cours de la vie du système (les relations entre les objets d'un système sont définies lors de la description du système et ne peuvent évoluer). Ces champs peuvent être multivalués.
- * Les **variables d'état** caractérisent l'état des objets et sont susceptibles d'évoluer au cours de la vie du système. Ces champs sont univalués.

En Figaro, tout champ appartient à un type de champ identifié par un mot-clé (CONSTANTE et PARAMETRE_LOI pour les champs constants, INTERFACE pour les champs d'interface, ATTRIBUT, PANNE et EFFET pour les variables d'état). Chaque champ est décrit par son nom puis par des facettes de description. On peut résumer la structure générale de la description d'un champ de type comme suit :

```
Syntaxe :
<type_de_champs>
  champ1 <nom_de_facette>    <liste_de_valeurs>
    ...
    <nom_de_facette>    <liste_de_valeurs> ;

  champ2 <nom_de_facette>    <liste_de_valeurs>
    ...
    <nom_de_facette>    <liste_de_valeurs> ;

  ...
<type_de_champs>    ...
```

Exemples de déclaration de champ :

```
ATTRIBUT
  fonc DOMAINE 'arret' 'demande_demarrage'
      'refus_demarrage' 'marche'
  PAR_DEFAULT 'arret' ;

CONSTANTE
  poids DOMAINE ENTIER ;
```

Avant de décrire plus en détail chaque type de champs possible, il nous faut préciser certaines notions communes à toutes les catégories de champs.

6.2.1. Préliminaires

6.2.1.1. Etat initial d'un modèle Figaro

Lors de l'écriture d'une base de connaissances, l'utilisateur décrit des types génériques et réutilisables pour différentes études. Pour étudier un système particulier, ces types sont utilisés pour décrire le système. La description complète d'un système consiste à créer les objets du système et à affecter des valeurs à chaque champ caractéristique du type de l'objet. Ces valeurs sont :

- soit une valeur initiale pour les variables d'état du modèle,
- soit une valeur pour les constantes du modèle.

```
Exemple :
TYPE vanne ;
CONSTANTE
  position DOMAINE 'ouvert' 'ferme';

(* base de faits *)
OBJET v1 EST_UNE vanne ;
  position = 'ouvert ;
```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

L'ensemble des valeurs attribuées aux champs des objets lors de la description du système constitue l'état initial incomplet du modèle. L'état initial complet du modèle sera calculé en appliquant les règles d'interaction sur cet état initial incomplet.

Remarque : l'attribution d'une valeur initiale à chaque variable d'état du modèle n'est pas obligatoire. Avant d'appliquer les règles d'interaction, le traitement de calcul de l'état initial d'un modèle attribue une valeur par défaut à chaque variable non initialisée.

6.2.1.2. Valeur par défaut

Il est parfois intéressant d'indiquer dans la base de connaissances une **valeur par défaut** pour les champs génériques caractérisant les types. Cela permet d'alléger la phase de description d'un système. En effet, les valeurs par défaut attribuées aux champs d'un type seront automatiquement attribuées aux champs de chaque objet créé pour ce type. Ces valeurs pourront bien sûr être modifiées par l'utilisateur.

Une valeur par défaut représente :

- soit une valeur initiale par défaut pour les champs "variables d'état",
- soit une valeur par défaut pour les champs "constantes".

Pour un champ donné d'un type, une valeur par défaut est créée en utilisant la facette PAR_DEFAULT. **L'attribution d'une valeur par défaut à un champ de type n'est jamais obligatoire.**

L'exemple suivant met en évidence cette notion en définissant un type *vanne* avec un attribut *position* qui aura pour valeur par défaut 'ouvert'. Deux objets de ce type seront alors créés : un utilisant la valeur par défaut, l'autre la redéfinissant.

```
Exemple :
TYPE vanne ;
ATTRIBUT
    position DOMAINE 'ouvert' 'ferme' PAR_DEFAULT 'ouvert' ;

(* base de faits *)
OBJET v1 EST_UNE vanne ;
(* la valeur de position de v1 est 'ouvert' *)

OBJET v2 EST_UNE vanne ;
ATTRIBUT
    position = 'ferme' ;
(* la valeur de position de v2 est 'ferme' *)
```

Remarque : Pour un champ donné d'un objet, une valeur par défaut est créée en utilisant la facette "=". Cette facette n'est pas autorisée pour les caractéristiques des types. Réciproquement la facette PAR_DEFAULT est interdite pour les caractéristiques des objets.

La valeur par défaut utilisée avec le mécanisme d'héritage de Figaro permet de définir plus précisément un élément d'un type au niveau de ses sous-types. Dans notre cas, on pourra par exemple définir dans un type le domaine d'une variable d'état, et donner dans les sous-types la valeur par défaut de cette variable.

Dans l'exemple suivant, on divise les composants électriques en deux familles : les sources de courant (lignes, diesels, ...) et les éléments passifs (traversés par un courant comme les disjoncteurs et les tableaux). Par défaut, on dit qu'une source est alimentée, et qu'un élément passif ne l'est pas.

```
Exemple :
TYPE composant_electrique ;
ATTRIBUT
    alimente DOMAINE 'oui' 'non' ;

TYPE source_de_courant SORTE_DE composant_electrique;
ATTRIBUT
    alimente PAR_DEFAULT 'oui' ;

TYPE element_passif SORTE_DE composant_electrique ;
ATTRIBUT
    alimente PAR_DEFAULT 'non' ;
```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

6.2.1.3. Notion d'édition

En principe, lorsqu'un utilisateur décrit un système en utilisant une base de connaissances, chaque fois qu'un objet est créé, il a la possibilité d'éditer l'ensemble des champs caractéristiques du type de l'objet ainsi que leur valeur par défaut qu'il peut modifier.

Cependant, le langage Figaro permet au concepteur de base de connaissances :

- de restreindre l'ensemble des champs visualisables et éditables par l'utilisateur chargé de décrire un système (lors de l'écriture d'une base de connaissances, il peut être utile de définir des variables utiles à la spécification du modèle, mais inutiles à sa compréhension),
- de contrôler l'édition des champs de la base (rendre obligatoire l'initialisation de certains champs, rendre non modifiables les valeurs par défaut de certains champs, etc).

La facette EDITION (facultative) permet de préciser les caractéristiques d'édition d'un champ d'un type dans la base de connaissances. Les options d'éditions sont les suivantes. Elles peuvent être regroupées dans la même facette d'EDITION si elles sont liées par des virgules (liste de valeurs) .

VISIBLE NON VISIBLE
MODIFIABLE NON MODIFIABLE
OBLIGATOIRE NON OBLIGATOIRE

*Remarque : La négation d'une option est réalisée par l'ajout du mot-clé NON à cette option. Exemple : Négation de l'option VISIBLE : **NON VISIBLE**.*

Les options en **gras** sont les options par défaut d'un champ quelconque. Voici le détail des options :

Option **VISIBLE**

Le champ sera visualisé par l'utilisateur lors de la création d'un objet du type.

Option **NON VISIBLE**

Le champ ne sera pas visualisé par l'utilisateur lors de la création d'un objet du type.

Option **MODIFIABLE**

La valeur par défaut attribuée au champ dans la base de connaissances pourra être modifiée par l'utilisateur lors de la description d'un système particulier.

Option **NON MODIFIABLE**

La valeur par défaut attribuée au champ dans la base de connaissances ne pourra pas être modifiée par l'utilisateur lors de la description d'un système particulier.

Option **OBLIGATOIRE**

L'utilisateur doit obligatoirement renseigner une valeur initiale pour le champ (cette option n'est pas pertinente pour les champs constants qui doivent être obligatoirement renseignés).

Option **NON OBLIGATOIRE**

La valeur initiale de la variable n'est pas obligatoire (elle pourra par exemple être calculée par application des règles d'interaction pour le calcul de l'état initial complet).

Remarque : Les options s'appliquent dans l'ordre de priorité suivant : 1. Visible (non visible), 2. Modifiable (non modifiable), 3. Obligatoire (non obligatoire). Ainsi, un champ déclaré non visible et ne pourra être modifié car rendu non accessible à l'utilisateur (non visible).

Exemples :
 ATTRIBUT
 champ1 DOMAINE ENTIER
 PAR_DEFAULT 12

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

EDITION NON VISIBLE ;
 (* ce champ ne pourra être modifié lors de la création d'un objet du type *)

champ2 DOMAINE REEL
 EDITION VISIBLE, MODIFIABLE, OBLIGATOIRE ;
 (* ce champ devra être affecté lors de la création d'un objet du type *)

Remarque : Seuls les types peuvent définir des champs avec une facette EDITION. Cette facette n'est en aucun cas reconnue dans les bases de faits, elle ne doit donc pas être utilisée dans les objets.

6.2.1.4. Remarque sur le calcul de l'état initial d'un modèle Figaro

Avant toute exploitation d'un modèle Figaro pour générer un modèle de fiabilité (Arbre de défaillances, Graphe d'états, ...), le traitement devra calculer l'état initial complet du modèle.

Le concepteur de base de connaissances devra donc définir, pour chaque variable d'état de la base de connaissances, la méthode d'initialisation retenue :

- attribution d'une valeur initiale par défaut dans la base de connaissances (facette PAR_DEFAULT) non modifiable (facette EDITION) par l'utilisateur (la valeur initiale pour ce champ est fixée pour tous les systèmes de la classe),
- attribution obligatoire (facette EDITION) d'une valeur initiale lors de la description d'un système,
- calcul de la valeur initiale par les règles d'interaction.

6.2.2. INTERFACE

Syntaxe :

```
INTERFACE nom_interface
  [GENRE nom_type]
  [CARDINAL nombre_entier]
  | [CARDINAL nombre_entier JUSQUA nombre_entier | INFINI ]
```

Comme nous l'avons vu au chapitre 2, les champs interface permettent de représenter les relations entre les objets des classes. Pour un type donné, les champs introduits par le mot clé INTERFACE permettent d'indiquer les types avec lesquels le type décrit aura une relation.

- Un champ interface est identifié par le nom de l'interface (nom de la relation)
- La facette GENRE permet de préciser le nom du type en relation pour cette interface avec le type décrit (elle est bien sûr obligatoire). L'utilisation de la facette GENRE est interdite dans les objets.

```
TYPE tableau_electrique ;
```

```
TYPE pompe ;
INTERFACE alim GENRE tableau_electrique ;
```

- La facette CARDINAL permet de spécifier des contraintes sur la cardinalité de la relation définie par le champ interface. Cette facette permet notamment de spécifier si la relation est obligatoire ou non.

- * Facette CARDINAL non définie

Si la facette CARDINAL n'est pas définie (*cette facette n'est pas obligatoire*), cela signifie qu'un objet quelconque du type décrit peut être interfacé avec 0 ou plusieurs objets du type en interface (sans limitation de nombre). Dans ce cas, la relation n'est pas obligatoire.

- * **CARDINAL** nombre_entier

Si la facette CARDINAL est égale à un nombre entier n, cela signifie que tout objet du type décrit sera **obligatoirement** interfacé avec n objets du type en interface (le cardinal de l'ensemble des objets en interface est fixé à n). Si n=1, on dit que l'interface est monovaluée.

- * **CARDINAL** nombre_entier **JUSQUA** nombre_entier | **INFINI**

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Si la facette **CARDINAL** est un intervalle p,q, cela signifie que tout objet du type décrit sera relié à entre p et q objets du type en interface (le cardinal de l'ensemble des objets en interface est inclus au sens large dans l'intervalle défini). Deux cas particuliers peuvent se présenter :

- Si p=0, cela signifie que la relation n'est pas obligatoire,
- Si q = INFINI , cela signifie que tout objet du type décrit peut être relié à plusieurs objets du type en interface, sans limitation de nombre).

Exemple :

```
TYPE vanne_electrique;
  INTERFACE      amont GENRE composant;
                   alim_220 GENRE tableau_electrique CARDINAL 1 ;
                   voie_de_commande GENRE commande CARDINAL 1 JUSQU' A 2 ;
```

Remarque :

Remarque : L'utilisation de la facette **CARDINAL** est interdite dans la base de faits, donc dans les objets.

6.2.3. Les constantes

Les constantes sont des champs dont la valeur ne varie pas au cours de la vie du système. De par leur nature, ces champs ne pourront donc jamais être modifiés au cours du fonctionnement du système.

On distingue deux types de valeurs constantes : les paramètres loi utilisés par les lois de probabilité (type de champ **PARAMETRE_LOI**), et les constantes proprement dites (type de champ **CONSTANTE**).

6.2.3.1. CONSTANTE

Syntaxe :

```
CONSTANTE      nom_constante
[ LIBELLE      "description" ]
[ DOMAINE      ENTIER | REEL | BOOLEEN |
  énumération_de_symboles
[ PAR_DEFAULT valeur | expression ]
[ EDITION      option_edition ]
```

Les constantes sont les caractéristiques des objets d'une classe qui n'évoluent pas au cours de la vie d'un système. Autrement dit, les événements survenant sur le système n'ont pas de conséquences (ne modifient pas) sur ces caractéristiques. Les constantes sont donc principalement utilisées pour représenter les caractéristiques fixes d'un système (longueur d'un câble, impédance d'une ligne, ...).

Exemple :

```
TYPE cable ;
CONSTANTE
  longueur      DOMAINE REEL
                LIBELLE "longueur de l'arc"
                PAR_DEFAULT 25.0
                EDITION OBLIGATOIRE ;
  impédance     DOMAINE REEL ;
```

Plusieurs facettes permettent de définir précisément le type et la valeur d'une constante. Voici la description de chacune d'entre elles :

Facette **LIBELLE**(facultative)

Permet de renseigner entre guillemets une description plus complète de la constante. Elle rend la base de connaissances et la description du système plus lisibles. Cette facette est autorisée pour les types et les objets.

Facette **DOMAINE**(obligatoire)

Précise l'ensemble des valeurs possibles pour la constante.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Il existe trois types prédéfinis de domaines en Figaro :

- **BOOLEEN** : contient uniquement les valeurs VRAI et FAUX.
- **ENTIER** : contient toutes les valeurs entières.
- **REEL** : contient toutes les valeurs réelles.

Il est possible de définir son propre domaine de valeurs. Pour cela, nous disposons du type **ENUMERE** :

- **ENUMERE** : un domaine énuméré définit un ensemble de valeurs symboliques possibles. Ces valeurs doivent être des valeurs constantes (il n'est pas possible d'utiliser des variables dans l'énumération). Chaque valeur est un *symbole* matérialisé par une chaîne de caractères entre quotes ('chaîne'). La déclaration d'un domaine énuméré permet de préciser quelles sont les valeurs que l'on peut donner au champ sans introduire d'incohérence dans le modèle.

Exemple :

```
TYPE transformateur ;
CONSTANTE
    tension_primaire    DOMAINE '400kv' '220kv'
                        PAR_DEFAULT '400kv'
                        EDITION OBLIGATOIRE ;
```

Facette PAR_DEFAULT (facultative)

Cette facette accepte une valeur précise appartenant au domaine de définition ou une expression constante, c'est-à-dire dans laquelle ne sont utilisés que des champs constants (l'ensemble des valeurs possibles pour l'expression doit être inclus dans le domaine de définition de la constante).

Remarque sur l'affectation d'une valeur par défaut sous forme d'expression :

Lors de l'exploitation d'un modèle Figaro, le traitement de calcul de l'état initial détectera une erreur si il trouve dans le modèle une constante dont la valeur n'est pas définie (toutes les constantes doivent être résolues au moment du calcul de l'état initial). Ainsi, lorsque dans une base de connaissances, on définit une valeur par défaut sous forme d'expression pour une constante, il faudra veiller à ce que l'utilisation de cette base rende obligatoire la saisie d'une valeur pour les champs constants intervenant dans l'expression (en utilisant l'option **OBLIGATOIRE** pour la facette **EDITION** par exemple). En effet, si un champ de l'expression n'a pas de valeur, le traitement ne pourra évaluer l'expression.

Facette EDITION (facultative)

(cf. 5.2.1)

6.2.3.2. PARAMETRE_LOI

Syntaxe :

```
PARAMETRE_LOI nom_parametre_loi
[ LIBELLE "description" ]
[ PAR_DEFAULT valeur | expression ]
[ EDITION option_edition ]
```

Comme nous l'avons vu précédemment, l'occurrence d'un événement sur un objet est fonction d'une loi de probabilité, qui s'exprime en fonction d'un certain nombre de paramètres.

Exemple : L'occurrence de la défaillance en fonctionnement d'une pompe est régie par une loi de probabilité exponentielle de paramètre `lambda_pompe`.

La catégorie de champs **PARAMETRE_LOI** d'une classe permet de regrouper tous les paramètres qui sont utilisables pour exprimer les lois de probabilité des événements pouvant survenir sur les objets de la classe.

Exemple :

```
TYPE pompe ;
PARAMETRE_LOI
```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

lambda_pompe ;

Les paramètres loi sont des constantes qui seront **toujours des réels**, ce qui nous dispense de déclarer leur domaine de définition.

Les facettes associées à la déclaration d'un champ PARAMETRE_LOI sont :

Facette LIBELLE (facultative)

(cf. CONSTANCE)

Facette PAR_DEFAULT (facultative)

Cette facette accepte une valeur réelle ou une expression constante, c'est-à-dire dans laquelle ne sont utilisés que des champs constants (l'ensemble des valeurs possibles pour l'expression doit être inclus dans l'ensemble des réels).

Cf. CONSTANCE pour la remarque sur l'affectation d'une valeur par défaut sous forme d'expression.

Facette EDITION (facultative)

(cf. 5.2.1)

Exemple :

```
TYPE pompe ;
PARAMETRE_LOI
    lambda      PAR_DEFAULT 1.e-8 ;
    beta_gamma  PAR_DEFAULT 1.e-7 ;
```

6.2.4. Les variables d'état

A la définition d'un type de composant, on sait que certaines caractéristiques, contrairement à celles qui sont fixes et que l'on déclarera en constantes, pourront très bien changer au cours du temps en fonction des événements survenant sur le système (par exemple, la position ouvert ou fermé d'un disjoncteur, la présence ou non d'une panne).

Pour cela, toutes les caractéristiques susceptibles de changer sont décrites dans des **variables d'état**. Ces caractéristiques sont représentées en interne par des variables, **toujours univaluées**, et l'état du système à un moment donné sera défini par une instantiation possible de ces variables.

6.2.4.1. Notion de variable réinitialisée

Pour rendre plus transparent la modélisation d'une classe de systèmes en Figaro, il est souvent utile de distinguer deux types de variables :

- les variables caractéristiques de l'état d'un système (variables non réinitialisées),
- les variables dont la valeur, **pour un état du système**, est **déduite** de la valeur des autres variables dans cet état par application des règles d'interaction (variables réinitialisées).

6.2.4.1.1. Variables non réinitialisées

Ce sont des variables dont la valeur évolue simplement en fonction de l'occurrence des événements.

Par exemple, au TYPE pompe on peut associer la variable d'état booléenne "défaillant" qui est FAUX initialement et deviendra VRAI si l'un des deux événements "défaillance en fonctionnement" ou "défaillance à la sollicitation" se produit.

6.2.4.1.2. Variables réinitialisées

Ce sont des variables dont la valeur sera réinitialisée à une valeur donnée (valeur de réinitialisation) avant chaque passage dans les règles d'interaction. Autrement dit, pour un état du système donné, la valeur de ces variables sera toujours **déduite** de la valeur de réinitialisation et de la valeur des autres variables du modèle.

Prenons l'exemple d'un réseau maillé composé de sources, d'arcs et de noeuds. Cette classe de systèmes est régie par les lois suivantes :

- Une source est toujours alimentée.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- Un événement "défaillance arc" peut se produire sur chaque objet du type arc.
- Un noeud est alimenté si et seulement si il existe un noeud voisin alimenté tel que l'arc reliant les deux noeuds soit non défaillant.

Pour modéliser une telle classe de systèmes, deux variables d'état sont particulièrement intéressantes. La variable booléenne "alimenté" associée aux objets de type "noeud". La variable booléenne "défaillant" associée aux objets du type "arc".

La variable "défaillant" évolue pour un arc en fonction de l'occurrence de l'événement "défaillance arc" sur cet arc. En revanche, pour un noeud donné, la variable "alimenté" doit être recalculée, après chaque occurrence d'événement, en fonction des noeuds voisins et du nouvel état des arcs du réseau (elle dépend uniquement des connexions du réseau). Cette variable est donc déduite par application des règles d'interaction. On la déclarera donc en Figaro comme une variable réinitialisée, la valeur de réinitialisation étant par exemple FAUX.

La notion de variable d'état réinitialisable présente un intérêt important du point de vue de l'exploitation des modèles Figaro. En effet, la valeur d'une variable d'état réinitialisée dépendant de la valeur des autres variables du modèle, il n'est pas nécessaire de stocker sa valeur lorsque l'on veut mémoriser un état du système.

La valeur de réinitialisation d'une variable doit être, obligatoirement, une constante.

6.2.4.2. Catégories de variables d'état en Figaro

On distingue trois formes de variables d'état en Figaro : les ATTRIBUTS, les PANNES et les EFFETS.

Les **attributs** regroupent toutes les variables d'état qui ne sont ni des pannes ni des effets. On peut définir des attributs simples (non réinitialisés) ou des attributs réinitialisés (facette REINITIALISATION).

Les **pannes** sont des variables d'état booléennes non réinitialisées qui seront utilisées pour détecter la présence ou l'absence d'une défaillance sur un objet. Ces variables d'état sont associées aux événements déclarés dans les règles d'occurrence.

Les **effets** sont des variables d'état booléennes réinitialisées à FAUX. Ils sont utilisés pour simplifier la déclaration de telles variables. Par ailleurs, ils permettent de simplifier l'écriture des bases de connaissances destinées à être exploitées pour générer des arbres de défaillances (ces variables d'état seront traitées de façon particulière par le traitement de transformation d'un Arbre des causes en Arbre de défaillances).

6.2.4.3. ATTRIBUT

Syntaxe:

```

ATTRIBUT      nom_attribut
[ LIBELLE      "description" ]
[ DOMAINE      ENTIER | REEL | BOOLEEN |
                               énumération_de_symboles
[ PAR_DEFAULT valeur | expression ]
[ REINITIALISATION valeur | expression ]
[ EDITION      option_edition ]

```

Les attributs sont des variables d'état caractéristiques de la classe. Chaque attribut est associé à un domaine de définition et peut être associé à une valeur initiale par défaut.

Exemple :

```

TYPE disjoncteur ;
ATTRIBUT
  position DOMAINE 'ouvert' 'ferme'
           PAR_DEFAULT 'ferme' ;

```

A chaque attribut peuvent être associées plusieurs facettes :

Facette DOMAINE (obligatoire)

Permet de définir l'ensemble des valeurs possibles pour l'attribut. Cette facette est identique à celle des CONSTANTES.

Exemple :

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

```

TYPE vanne SORTE_DE composant_thermohydraulique;
ATTRIBUT
    debit      DOMAINE REEL ;
    pression DOMAINE 'faible' 'moyenne' 'forte' ;

```

Facette LIBELLE (facultative)

(cf. CONSTANCE)

Facette PAR_DEFAULT (facultative)

Cette facette accepte une valeur précise appartenant au domaine de définition de l'attribut, ou une expression (l'ensemble des valeurs possibles pour l'expression doit être inclus dans le domaine de définition de l'attribut).

Remarque sur l'affectation d'une valeur initiale par défaut sous forme d'expression :

Lorsque dans une base de connaissances, on définit une valeur initiale par défaut sous forme d'expression pour un attribut, il faudra veiller à ce que l'utilisation de cette base rende obligatoire la saisie d'une valeur pour les champs intervenant dans l'expression (en utilisant l'option OBLIGATOIRE pour la facette EDITION par exemple). En effet, si un champ de l'expression n'a pas de valeur initiale, le traitement ne pourra évaluer l'expression.

Facette REINITIALISATION (facultative)

Cette facette permet de déclarer qu'un attribut est une variable réinitialisée et de lui affecter une valeur de réinitialisation (cf. Paragraphe "Variables réinitialisées" de ce chapitre). Cette facette n'est autorisée que pour les variables déclarées comme attributs dans la base de connaissance.

La valeur de réinitialisation doit obligatoirement être une constante.

6.2.4.4. PANNE

Syntaxe:

```

PANNE nom_panne
    [ LIBELLE      "description" ]
    [ EDITION      option_edition ]

```

Lorsque l'on réalise l'étude de sûreté de fonctionnement d'un système, on s'intéresse principalement aux conséquences des dysfonctionnements des composants sur le fonctionnement du système. C'est pourquoi de nombreux événements modélisés dans une classe d'objets sont des événements représentatifs des pannes pouvant survenir sur les objets de cette classe.

Exemple : l'événement "défaillance en fonctionnement" peut survenir sur les objets de type pompe.

Pour décrire les conséquences d'un "événement panne" sur le système, il est souvent utile d'associer aux classes d'objets des variables d'état booléennes représentatives de l'absence ou de la présence d'une panne (de l'occurrence ou non de "l'événement panne").

Le langage Figaro permet de regrouper ces variables particulières sous la catégorie de champs PANNE.

Exemple :

```

TYPE pompe ;
PANNE
    defaillance_en_fonctionnement ;

```

Remarque : Il ne faut pas confondre "l'événement panne", qui sera modélisé dans les règles d'occurrence, avec les "variables panne" qui mémorise si cet événement s'est produit ou non.

Les variables d'état de la catégorie PANNE sont toujours des variables booléennes (VRAI si la panne s'est produite, FAUX sinon). Leur valeur initiale est FAUX.

Les seules facettes disponibles pour une PANNE sont les facettes optionnelles EDITION et LIBELLE (cf. CONSTANCE).

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

6.2.4.5. EFFET

Syntaxe:

```

EFFET nom_effet
    [ LIBELLE      "description" ]
    [ EDITION      option_edition ]

```

Les effets **sont équivalents à des variables booléennes réinitialisées à FAUX**, mais étant donné leur emploi très fréquent (surtout dans les modèles destinés à la construction d'arbres de défaillances), il est utile de les distinguer, car cela allège beaucoup leur déclaration.

Lorsqu'une variable est une variable réinitialisée à FAUX, cela signifie que, si aucune règle d'interaction ne permet de **déduire** la valeur de cette variable, celle-ci est FAUSSE. C'est pourquoi un effet correspond à un comportement d'un composant, généralement de nature fonctionnelle (obstruction, fuite, etc.), observé à la suite de la réalisation d'un ensemble de causes. Un effet se produit donc de manière **logique**. Il disparaît dès que ses causes disparaissent : on dit qu'un *effet est non rémanent*. Les effets sont à rapprocher de la notion d'événement intermédiaire dans les arbres de défaillances.

Exemple :

```

TYPE fonction ;
    EFFET echec_fonction;

    INTERACTION
    SI ( perte DE chemin_1 ) ET ( perte DE chemin_2)
    ALORS echec_fonction;

```

Pour déclarer un effet il suffit de donner son nom. Ici, echec_fonction est une conséquence (un effet) de perte de chemin_1 et de perte de chemin_2 : il y a echec_fonction (conséquent de la règle d'interaction) s'il y a perte de chemin_1 et de chemin_2 (prémisse de la règle). Si echec_fonction (pas de transmission de flux) se produit, la réparation de sa cause va la faire disparaître : l'echec_fonction est un effet *non rémanent*.

Utilisation des effets par le générateur d'arbres de défaillances

Pour mieux comprendre l'utilisation des EFFETS dans une base de connaissances destinée à être utilisée pour générer des arbres de défaillances, il est important de comprendre comment ces variables sont utilisées par le traitement de génération d'arbres.

Comme nous l'avons vu au chapitre 2, la génération d'un arbre de défaillances à partir d'un modèle Figaro et d'un événement indésirable s'effectue en deux étapes :

- Construction d'un arbre des causes expliquant l'événement indésirable par chaînage arrière sur les règles d'interaction.
- Transformation de l'arbre des causes en arbre de défaillances.

C'est dans cette deuxième étape que les effets seront utilisés. Supposons que l'arbre des causes contienne une feuille associée à une expression booléenne du type "effet(objet) = VRAI". Supposons encore qu'aucune règle d'occurrence ne provoque la mise à VRAI de cet effet. Dans ce cas, la feuille est considérée comme FAUSSE (l'effet n'a pu se produire car aucune cause ne s'est produite) et l'arbre est simplifié en conséquence.

6.3. Les règles

Comme nous l'avons vu au chapitre 2, les règles d'un type permettent de définir le comportement dynamique des objets d'un type. Les règles décrivent les événements pouvant survenir sur les différents types modélisés et leurs conséquences sur le système. Il existe deux types de règles en Figaro.

Les **règles d'occurrence** permettent de modéliser l'occurrence des événements qui peuvent se produire sur un objet. Elles sont de la forme :

SI <condition>

IL_PEUT_SE_PRODUIRE <événement>

PROVOQUE <actions>

- * La condition exprime la condition que doit vérifier l'état du système pour qu'un événement se déclenche sur un objet d'une classe.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- * Les actions décrivent les conséquences de l'événement sur les variables d'état de la classe.

Les **règles d'interaction** modélisent l'influence de l'état des objets d'une classe sur les objets d'autres classes. Elles sont de la forme :

SI <condition>

ALORS <actions>

SINON <actions>

Pour ces règles, la condition (SI) permet par sa réalisation (ALORS) ou par sa non réalisation (SINON) le déclenchement d'un ensemble d'actions affectant les variables des autres classes.

Exploiter un modèle Figaro (le faire tourner), c'est appliquer les règles de la base de connaissances à la base de faits décrivant le système à étudier. Les règles d'un modèle Figaro peuvent être exploitées en chaînage avant (pour générer le graphe d'états du système par exemple) ou en chaînage arrière (pour générer des arbres de défaillances par exemple).

6.3.1. Les conditions

Les conditions des règles Figaro sont des expressions booléennes complexes ou non. Elles correspondent à des tests sur l'état du système qui vont être évalués par les traitements (chaînage avant ou arrière) pour tester si la règle est applicable. Si la condition d'une règle d'un objet est évaluée à VRAI pour les valeurs des variables dans l'état courant du système, alors la règle est déclenchable et les actions peuvent être appliquées.

La syntaxe des expressions utilisables pour définir les conditions des règles a été complètement définie précédemment. Rappelons simplement que, pour un type donné, on peut utiliser, dans l'expression associée à la condition d'une règle :

- les champs des objets du type,
- les champs des objets en interface en définissant un ensemble d'objets par une interface et en utilisant les opérateurs IL_EXISTE, IL_EXISTE AU MOINS, QQSOIT ou SOIT (le comportement des objets d'un type dépend principalement des objets en interface),
- les champs des objets d'un type particulier du modèle en définissant l'ensemble d'objets par le nom du type et en utilisant les opérateurs IL_EXISTE, IL_EXISTE AU MOINS, QQSOIT ou SOIT,
- les champs du paragraphe GLOBAL (variables globales).

L'exemple suivant décrit une règle d'interaction pour une pompe de secours qui se déclenche (*etat* passe à *demande_demarrage*) uniquement si la pompe secourue est en panne.

```
Exemple :
TYPE pompe ;
PANNE
    def_soll ;
    def_fonc ;

TYPE pompe_secours SORTE_DE pompe ;
ATTRIBUT
    etat DOMAINE 'arret' 'demande_demarrage' 'marche'
        PAR_DEFAULT 'arret' ;
INTERFACE
    pompe_secourue GENRE pompe CARDINAL 1;
INTERACTION
    SI PANNE( pompe_secourue ) ET MARCHE
        ET etat = 'arret'
    ALORS
        etat <-- 'demande_demarrage' ;
```

6.3.2. Les actions

Les actions sont les opérations qui permettent de modifier les variables d'état du modèle, faisant ainsi évoluer l'état du système. Une action peut donc modifier uniquement des attributs, des pannes ou des effets. Pour un

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

type donné, on peut modifier par une action les **variables d'état** :

- des objets du type,
- des objets en interface en définissant un ensemble d'objets par une interface et en utilisant les opérateurs POUR_TOUT ou SOIT (lorsque le comportement des objets d'un type dépend principalement des objets en interface),
- des objets d'un type particulier du modèle en définissant l'ensemble d'objets par le nom du type et en utilisant les opérateurs POUR_TOUT ou SOIT,
- du paragraphe GLOBAL (variables globales).

Une règle peut provoquer plusieurs **actions élémentaires** (liste d'actions). Dans ce cas, il faut les lier par des virgules. Il existe trois sortes d'actions élémentaires.

6.3.2.1. Affectation d'une valeur à une variable

L'affectation d'une nouvelle valeur à un attribut, une panne ou un effet est réalisée grâce à la facette <--.

```
<variable> <-- <nouvelle valeur>
```

La nouvelle valeur doit bien sûr appartenir au domaine de valeurs possibles spécifié pour la variable.

Afin d'alléger certaines bases de connaissances, la gestion des variables booléennes et notamment des variables PANNE et EFFET est simplifiée. Ainsi, **pour une variable booléenne** :

```
<variable> <-- VRAI      est équivalent à      <variable>
<variable> <-- FAUX      est équivalent à      NON <variable>
```

Ces deux notations sont strictement équivalentes.

Exemples :

```
... ALORS position DE x <-- 'ouvert' ,
      valeur DE y <-- valeur DE y + poids DE y ;
      (* la règle execute deux actions élémentaires *)

... ALORS refus_demarrage      ;
      (* refus_demarrage est soit un attribut booléen, une panne ou un effet *)

... ALORS NON alim ;
      (* alim est soit un attribut booléen, une panne ou un effet *)
```

6.3.2.2. Action sur un ensemble d'objets

L'opérateur POUR_TOUT permet de modifier successivement les attributs d'un ensemble d'objets. Il agit comme les opérateurs complexes IL_EXISTE, IL_EXISTE AU_MOINS, QQSOIT, SOMME et PRODUIT (cf. Expressions et Notions Avancées).

```
POUR_TOUT x UN <ensemble d'objets>
```

Syntaxe :

```
POUR_TOUT variable
      UN | UNE      nom_d_ensemble
      [ DE_TYPE     type ]
      [ VERIFIANT    condition ]
      FAIRE action
```

POUR_TOUT définit une variable **liée** à l'opérateur qui prendra successivement toutes les valeurs de l'ensemble. L'ensemble est soit un nom d'interface (multivaluée) soit un nom de type. Comme pour les opérateurs complexes, il est possible de préciser des contraintes optionnelles (DE_TYPE et VERIFIANT) pour effectuer l'action (facette FAIRE) uniquement sur un sous-ensemble d'objets.

Le mécanisme de cet opérateur est le suivant :

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- Si l'ensemble est non vide, on effectue pour chaque objet de l'ensemble, l'action précisée par la facette FAIRE.

- Si l'ensemble est vide, aucune action n'est effectuée.

Exemple :

```
POUR_TOUT x UN <ensemble d'objets>
FAIRE <affectation>
```

```
ALORS POUR_TOUT x UNE commande
FAIRE position DE x <-- 'ouvert' ;
```

```
(* Imbrication d'opérateurs POUR_TOUT *)
ALORS POUR_TOUT x UN amont
FAIRE ( POUR_TOUT y UN aval DE x
        VERIFIANT x <> y
        FAIRE valeur DE y <-- valeur DE x ) ;
```

Remarque : La portée de la variable liée est réduite à l'opérateur POUR_TOUT. Elle n'est donc accessible que dans l'action précisée par FAIRE. Pour combiner plusieurs actions sous une même facette FAIRE, il faut utiliser des parenthèses et non des virgules. En effet, la virgule a une priorité plus élevée que le mot-clé FAIRE. Les exemples suivants éclairent cet aspect :

Exemples :

```
ALORS POUR_TOUT x UN amont
FAIRE etat DE x <-- 'arrêt', position DE x <-- 'ouvert' ;
```

Cet exemple provoque une erreur car Figaro assimile cette phrase comme deux actions successives après le mot-clé ALORS et non pas après le mot-clé FAIRE :

```
(* Voici comment Figaro lit la phrase précédente *)
Exemples :
ALORS POUR_TOUT x UN amont
FAIRE etat DE x <-- 'arrêt',
position DE x <-- 'ouvert' ; (* x est inconnu !!! *)
```

Il aurait fallu écrire :

```
ALORS POUR_TOUT x UN amont
FAIRE ( etat DE x <-- 'arrêt', position DE x <-- 'ouvert' );
(* x est connu : les parenthèses sont nécessaires *)
```

6.3.2.3. Résolution de système d'équations

La facette RESOUDRE_SYSTEME permet de préciser le système d'équations à résoudre. Le mécanisme de résolution est le suivant :

- Si la règle est déclenchable, Figaro résout les systèmes d'équations.
- Les valeurs des inconnues sont alors calculées.
- Comme ces inconnues représentent des attributs, Figaro leur affecte leur nouvelle valeur.

La valeur des attributs/inconnues a donc été modifiée. L'état du système est modifié.

6.3.3. Les règles d'occurrence

Les règles d'occurrence d'un type modélisent l'occurrence des événements possibles et leurs conséquences sur ce type.

L'occurrence d'un événement est fonction d'une certaine loi de probabilité. On distingue plusieurs types d'événements possibles, en fonction de la loi de probabilité associée à l'événement :

- Les événements associés à une loi instantanée : ces événements se produisent instantanément, dès lors que leur condition d'occurrence est vérifiée. Le paramètre loi associé est un taux d'occurrence sans dimension.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- Les événements dont l'occurrence est différée dans le temps (par rapport au moment où leurs conditions d'occurrence sont vérifiées). On distingue :

* Les événements associés à une loi exponentielle : ces événements peuvent se produire au bout d'un temps aléatoire. Le paramètre loi associé est un taux d'occurrence horaire.

* Les événements associés à une loi temps constant : ces événements se produisent de façon certaine au bout d'un temps défini T, si les conditions d'occurrence de l'événement sont maintenues pendant toute la durée de T.

Une règle d'occurrence peut être associée à un ou plusieurs événements. Tous les événements associés à une même règle doivent suivre le même type de loi. Ainsi, on distingue trois types de règles d'occurrence :

- Les règles d'occurrence de type exponentiel qui déclenchent **un et un seul** événement associé à une loi exponentielle.
- Les règles d'occurrence de type temps constant qui déclenchent **un et un seul** événement associé à une loi temps constant.
- Les règles d'occurrence de type instantané qui déclenchent **plusieurs** événements associés à des lois instantanées.

Règles d'occurrence instantanées

Lorsqu'un système est dans un état donné, il peut arriver qu'il ait le choix entre plusieurs évolutions incompatibles. C'est le cas par exemple lorsqu'un composant sollicité peut démarrer ou refuser de démarrer. Le système a le choix entre ces deux évolutions, mais l'une des deux **doit forcément** se produire. Ainsi, les règles d'occurrence instantanées permettent de modéliser l'occurrence d'événements :

- incompatibles entre eux (exclusifs),
- dont l'un au moins doit se produire instantanément.

Une règle d'occurrence instantanée aura donc une syntaxe de la forme :

SI <condition>

```
IL_PEUT_SE_PRODUIRE      <événement>
                           PROVOQUE <actions>
                           OU_BIEN  <événement>
                           PROVOQUE <actions>
                           OU_BIEN ...
```

Chaque événement est associé à une loi instantanée. **La somme des paramètres loi de chaque événement doit obligatoirement être égale à 1** (un des événements doit obligatoirement se produire).

*Remarque : pour l'événement déclenché par la **dernière** facette OU_BIEN, il n'est pas obligatoire de renseigner le paramètre loi associé à l'événement. Si celui-ci n'est pas renseigné, Figaro rajoutera automatiquement un paramètre loi instantanée de valeur égale à :*

1 - $\sum$ (paramètres loi des autres événements déclenchés par la règle)

Ordre d'application des règles d'occurrence en chaînage avant

Dans le chapitre 2, nous avons étudié le principe d'exploitation des règles d'un modèle Figaro en chaînage avant. Nous avons vu que, lorsque le système est dans un état donné, le traitement trouve les règles d'occurrence des objets applicables (règles des objets dont la condition est vérifiée), choisit une de ces règles et l'applique pour déclencher un événement.

Ce processus de choix et d'application des règles d'occurrence est en fait un peu plus complexe.

Lorsque dans un état du système, le traitement trouve des règles d'occurrence instantanées applicables, cela signifie qu'au moins un événement de chaque règle doit être déclenché instantanément. Ainsi, le traitement ne choisit pas **un** événement mais une **combinaison** d'événements formée d'un événement de chaque règle applicable.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Ensuite, le traitement applique successivement chaque événement de la combinaison en déclenchant les actions provoquées par les événements. Il est important de noter que l'ordre d'application des événements de la combinaison est indifférent. Tout se passe comme si les événements étaient appliqués en parallèle (les prémisses des règles sont toutes évaluées avant que les actions ne soient exécutées). Ainsi, le concepteur de base de connaissances devra veiller à ce que les règles d'occurrence instantanées du système susceptibles d'être applicables simultanément ne concluent pas sur des faits incompatibles.

Si, dans un état du système, aucune règle instantanée n'est applicable, le système recherche les règles exponentielles ou temps constant applicables (un même modèle Figaro ne peut contenir à la fois des règles exponentielles et temps constant).

Si plusieurs règles exponentielles sont applicables, le traitement choisit simplement l'une d'elles et applique l'événement associé.

Si plusieurs règles temps constant sont applicables (leurs conditions ont été vérifiées pendant toute la durée associée au paramètre temps), le traitement choisit les règles à déclencher en priorité et applique les événements associés. Comme dans le cas des règles instantanées, l'ordre d'application des événements est indifférent.

Variables Panne et Evénements

De façon qualitative, on peut classer les événements susceptibles de survenir sur un système en quatre catégories :

- Les **transitions** sont des événements décrivant le comportement normal des composants. Exemple : démarrage d'une pompe après sollicitation.
- Les **défaillances** sont des événements décrivant les défaillances des composants. Exemple : défaillance à la sollicitation d'une pompe.
- Les **indisponibilités** représentent un type particulier de défaillance à la sollicitation. Ce sont le plus souvent des événements qui se sont produits dans le passé du système et qui se révèlent à la première sollicitation des composants. (Exemple : oubli fermé d'une vanne dont la position est normalement ouverte). Le traitement des événements de type INDISPONIBILITE sera exactement le même que le traitement des défaillances à la sollicitation. Autrement dit, l'information INDISPONIBILITE est une information purement qualitative par rapport à une simple défaillance à la sollicitation.
- Les **réparations** représentent des changements d'états dus aux opérations de maintenance sur un système. Exemple : réparation d'une pompe suite à défaillance à la sollicitation ou en fonctionnement de la pompe.

Ainsi, pour décrire un événement dans une règle d'occurrence on pourra écrire :

IL_PEUT_SE_PRODUIRE **TRANSITION** <événement>

ou

IL_PEUT_SE_PRODUIRE **DEFAILLANCE** <événement>

ou

IL_PEUT_SE_PRODUIRE **INDISPONIBILITE** <événement>

ou

IL_PEUT_SE_PRODUIRE **REPARATION** <événement>

Comme nous l'avons vu précédemment, on peut définir en Figaro dans le champ PANNE des variables d'état booléennes particulières, qui indique si un "événement panne" s'est produit.

- Un événement du type DEFAILLANCE ou INDISPONIBILITE peut ou non être associé à une variable PANNE.

- * Si l'événement est associé à une variable PANNE, c'est-à-dire si l'identificateur d'une variable PANNE est le même que l'identificateur de l'événement, la règle décrivant l'occurrence de la défaillance ou de l'indisponibilité pourra s'écrire de façon plus concise en utilisant des informations implicites. Il ne sera pas nécessaire, par exemple, de décrire que l'occurrence d'une défaillance provoque la mise à VRAI de la variable PANNE associée.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- * En revanche, si l'événement n'est pas associé à une variable PANNE, l'utilisation des mots clés DEFAILLANCE ou INDISPONIBILITE permettra uniquement de fournir une information qualitative sur l'événement.

- De même, un événement du type REPARATION peut ou non être associé à une ou plusieurs variables PANNE.

- * Si l'événement est associé à une ou plusieurs variables PANNE, c'est-à-dire si on décrit explicitement que l'événement REPAIRE une ou plusieurs variables PANNE, la règle décrivant l'occurrence de la réparation pourra s'écrire de façon plus concise en utilisant des informations implicites. Il ne sera pas nécessaire, par exemple, de décrire que l'occurrence de la réparation provoque la mise à FAUX des variables PANNE associées.
- * En revanche, si l'événement n'est associé à aucune variable PANNE, l'utilisation du mot clé REPARATION permettra uniquement de fournir une information qualitative sur l'événement.

6.3.3.1. Syntaxe d'une règle d'occurrence

```

OCCURRENCE [nom_de_règle]
[ GROUPE      liste_groupes ]
[ SI          condition]

IL_PEUT_SE_PRODUIRE
TRANSITION nom_transition
[ LIBELLE    "description" ]
LOI          { EXP | EXPONENTIELLE,
              INS | INSTANTANEE,
              T_C | TEMPS_CONSTANT }
              ( expression )
[ PROVOQUE   action ]

| DEFAILLANCE nom_de_defaillance | nom_de_panne
[ LIBELLE    "description" ]
LOI          { EXP | EXPONENTIELLE,
              INS | INSTANTANEE,
              T_C | TEMPS_CONSTANT }
              ( expression )
[ PROVOQUE   action ]

| REPARATION nom_reparation
[ LIBELLE    "description" ]
LOI          { EXP | EXPONENTIELLE,
              INS | INSTANTANEE,
              T_C | TEMPS_CONSTANT }
              ( expression )
[ REPARE     liste_pannes ]
[ PROVOQUE   action ]

| INDISPONIBILITE nom_indisponibilité | nom_de_panne
[ LIBELLE    "description" ]
LOI          { EXP | EXPONENTIELLE,
              INS | INSTANTANEE,
              T_C | TEMPS_CONSTANT }
              ( expression )
[ PROVOQUE   action ]

[ OU_BIEN    TRANSITION ... | DEFAILLANCE ... |
              REPARATION ... | INDISPONIBILITE ... ]
[ OU_BIEN    ... ]

```

Remarque : La syntaxe des règles d'occurrence des objets est semblable à celle des règles d'occurrence des types.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Les règles d'occurrence définies pour un type sont introduites par le mot-clé OCCURRENCE

La syntaxe d'une règle d'occurrence est la suivante :

Nom de règle (facultatif)

Permet d'attribuer un nom à une règle.

```

EXEMPLE :
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' ;
OCCURRENCE
    def_fonc
    SI etat = marche
    IL_PEUT_SE_PRODUIRE
        ..... ;

```

Facette GROUPE (facultative)

Permet de spécifier l'appartenance de la règle à un ou plusieurs groupes (cf. Notions Avancées).

Facette SI (facultative)

Décrit la condition nécessaire pour que la règle d'occurrence soit déclenchable. Si aucune condition n'est associée à la règle (la règle doit se déclencher quel que soit l'état du système), la facette SI n'est pas mentionnée.

Facette IL_PEUT_SE_PRODUIRE (obligatoire)

Permet de décrire quels sont les événements qui se déclenchent lorsque la condition de la règle est vérifiée pour l'état du système. Chaque événement est identifié par son nom et qualifié par le mot-clé décrivant la nature de l'événement (TRANSITION, DEFAILLANCE, REPARATION, INDISPONIBILITE).

```

EXEMPLE :
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' ;
OCCURRENCE
    SI etat = marche
    IL_PEUT_SE_PRODUIRE
        TRANSITION defaillance_fonc
        ..... ;

```

Dans le cas d'une règle instantanée contenant plusieurs événements :

- Le premier est introduit par la facette IL_PEUT_SE_PRODUIRE
- Les événements suivants sont introduits par la facette OU_BIEN .

```

EXEMPLE :
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' 'sollicite' ;
OCCURRENCE
    SI etat = sollicite
    IL_PEUT_SE_PRODUIRE
        DEFAILLANCE defaillance_soll
        .....
    OU_BIEN
        TRANSITION demarrage
        ..... ;

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Chaque événement de la règle peut être décrit précisément par les facettes suivantes :

Facette LIBELLE (facultative)

Permet de mettre un texte en clair (entre guillemets) descriptif de l'événement. Cette facette libellé est facultative (mais recommandée pour obtenir des résultats d'étude lisibles).

```

EXEMPLE :
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' ;
OCCURRENCE
    SI etat = marche
    IL_PEUT_SE_PRODUIRE
        TRANSITION defaillance_fonc
        LIBELLE "Défaillance en fonctionnement de la pompe"
        ..... ;

```

Facette REPARE (facultative)

Elle n'est présente que lorsque le type d'événement est REPARATION (cf. Paragraphe Réparations). Elle permet de déclarer la liste des pannes supprimées à la suite d'une réparation. La facette doit être suivie des noms des variables PANNE, séparés par des virgules.

Facette LOI (obligatoire)

Permet de préciser la loi de probabilité associée à l'événement. La loi sera déclarée avec une syntaxe de la forme :

nom_de_loi (expression_numérique)

Cette facette n'est pas obligatoire pour le dernier événement associé à une règle instantanée (dernière facette OU_BIEN de la règle). Si elle n'est pas renseignée, Figaro rajoutera automatiquement un paramètre loi instantanée de valeur égale à 1- somme des paramètres loi des autres événements déclenchés par la règle.

Le nom de loi peut prendre les valeurs EXP ou EXPONENTIELLE (loi exponentielle), INS ou INSTANTANEE (loi instantanée), T_C ou TEMPS_CONSTANT (loi temps constant).

L'expression numérique permettant d'attribuer une valeur au paramètre de la loi associée à l'événement peut être principalement :

- Une expression constante faisant intervenir des champs PARAMETRE_LOI et des champs CONSTANTE (c'est le cas le plus fréquent) :

```

EXEMPLE :
TYPE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arret' ;
PARAMETRE_LOI
    lambda_pompe;
OCCURRENCE
    SI etat = marche
    IL_PEUT_SE_PRODUIRE
        DEFAILLANCE defaillance_fonc
        LOI EXP (lambda_pompe)
        ..... ;

```

- Une expression faisant intervenir des attributs variables. Dans ce cas, pour les lois EXP et INS, la valeur du paramètre loi sera évaluée en fonction de l'état du système au moment du tirage de l'événement. Pour une loi T_C, le problème est plus délicat, puisque la valeur du paramètre temps constant peut changer pendant l'écoulement du temps régi par la loi. Dans ce cas, on ne modifie pas la

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

valeur du paramètre loi. On se base sur la valeur calculée dans l'état du système pour lequel la règle est pour la première fois applicable.

La facette LOI est obligatoire exceptée dans la dernière alternative d'une règle instantanée (introduite par le mot-clé OU_BIEN). En effet, Figaro sait que la loi est de type instantanée et que la valeur de son paramètre de loi est :

$$1 - \sum (\text{paramètres des autres alternatives})$$

En effet, la somme de tous les paramètres loi d'une règle instantanée doit être égale à 1.

```

EXEMPLE :
TYPE pompe ;
PARAMETRE_LOI
  gamma_pompe ;
ATTRIBUT
  etat DOMAINE 'marche' 'arret' 'sollicite' ;
OCCURRENCE
  SI etat = sollicite
  IL_PEUT_SE_PRODUIRE
    DEFAILLANCE defaillance_soll
    LOI INS (gamma_pompe)
    PROVOQUE .....
  OU_BIEN
    TRANSITION demarrage
    PROVOQUE..... ;

```

Facette PROVOQUE (facultative)

Indique les conséquences immédiates de l'événement, c'est-à-dire les **actions** à effectuer.

```

EXEMPLE :
TYPE pompe ;
ATTRIBUT
  etat DOMAINE 'marche' 'arret';
PARAMETRE_LOI
  lambda_pompe;
OCCURRENCE
  SI etat = marche
  IL_PEUT_SE_PRODUIRE
    DEFAILLANCE defaillance_fonc
    LOI EXP (lambda_pompe)
    PROVOQUE      etat <-- 'arret' ;

```

Nous allons maintenant décrire chaque type d'événement et leurs liens avec les pannes.

6.3.3.2. Transitions normales

Les *transitions normales* sont des événements décrivant le comportement normal du système étudié.

```

Exemple de transitions :
TYPE alternance_jour_nuit ;
PARAMETRE_LOI
  jn ;
  nj ;
ATTRIBUT
  valeur DOMAINE 'jour' 'nuit' ;
OCCURRENCE

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

```

SI valeur = 'jour'
IL_PEUT_SE_PRODUIRE
TRANSITION pjn
    LIBELLE "passage de jour à nuit"
    LOI TEMPS_CONSTANT ( jn )
    PAR_DEFAULT jn = 12
    PROVOQUE valeur <-- 'nuit' ;

SI valeur = 'nuit'
IL_PEUT_SE_PRODUIRE
TRANSITION pnj
    LIBELLE "passage de nuit à jour"
    LOI TEMPS_CONSTANT ( nj )
    PAR_DEFAULT nj = 24 - jn
    PROVOQUE valeur <-- 'jour' ;

```

Ce type permet de définir un objet qui va traduire l'alternance entre le jour et la nuit (utile par exemple si certains événements ne peuvent intervenir que la nuit dans le fonctionnement d'un système). Le passage de jour à nuit pourra donc se faire (positionnement de la variable d'état valeur à 'nuit') suivant une loi à temps constant de paramètre jn si c'est le jour (test de la variable d'état valeur). On écrit, tout aussi aisément, la règle passage_nuit_jour.

Les événements pjn et pnj sont des informations purement qualitatives qui n'ont pas à être déclarés de manière explicite.

6.3.3.3. Défaillances

On définit deux types de défaillances : les défaillances à la sollicitation et les défaillances en fonctionnement.

Les sollicitations contiennent la description de ce qui va se produire de manière instantanée (loi instantanée) quand on va demander à un objet de réaliser l'une des fonctions pour laquelle il a été conçu. Cet objet pourra alors réagir de deux manières :

- soit il refuse d'adopter le comportement qu'on lui demande : on dit qu'il y a une *défaillance à la sollicitation*,
- soit il réalise ce qu'on attend de lui : la sollicitation est réussie et va être décrite éventuellement par une transition normale.

Les *défaillances en fonctionnement* sont des défaillances qui surviennent pendant le fonctionnement du système, sans qu'on explicite leur cause, et autres que les défaillances à la sollicitation (loi exponentielle).

- Lorsque l'événement défaillance n'est pas associé à une variable PANNE, le mot-clé DEFAILLANCE est une information qualitative qui ne sera pas exploitée par les traitements.

- Lorsque l'événement défaillance est associé à une variable PANNE (en précisant le nom d'une variable panne comme nom d'événement de défaillance), le processus suivant s'applique :

1. Par définition, une défaillance ne peut se produire que si le composant n'est pas déjà défaillant.

Ainsi il n'est pas nécessaire d'intégrer à la condition d'occurrence de l'événement, la condition '<variable de panne> = FAUX'. Celle-ci est implicite et sera prise automatiquement en compte par les traitements.

2. De même, l'occurrence d'un événement défaillance provoque toujours la mise à VRAI de la variable PANNE associée.

Il n'est donc pas nécessaire de décrire explicitement cette action dans les règles.

Exemple de défaillance à la sollicitation :

Prenons l'exemple d'un composant de type filtre. Lorsque ce composant est sollicité, deux alternatives peuvent se produire :

- * Le filtre peut être bouché, ce qui provoque la panne du composant. La probabilité de cet événement est une loi instantanée de paramètre g_bo.
- * Le filtre n'est pas bouché et la sollicitation réussit. La probabilité de cet événement est une loi instantanée de paramètre (1-g_bo).

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Pour modéliser ce comportement, on écrit la règle d'occurrence suivante :

```

TYPE filtre SORTE_DE composant_thermohydraulique ;
ATTRIBUT sollicite DOMAINE BOOLEEN PAR_DEFAULT VRAI;
PANNE bouchage ;
PARAMETRE_LOI g_bo ;
OCCURRENCE
SI sollicite
  IL_PEUT_SE_PRODUIRE
  DEFAILLANCE bouchage
    LIBELLE "bouchage à la sollicitation"
    LOI INSTANTANEE ( g_bo )
    PROVOQUE sollicite <-- FAUX
  OU BIEN
  TRANSITION non_bo
    PROVOQUE sollicite <-- VRAI ;

```

Cette règle est comprise par Figaro comme si on avait écrit :

```

TYPE filtre SORTE_DE composant_thermohydraulique ;
ATTRIBUT sollicite DOMAINE BOOLEEN PAR_DEFAULT VRAI;
PANNE bouchage ;
PARAMETRE_LOI g_bo ;
OCCURRENCE
SI sollicite ET NON bouchage
  IL_PEUT_SE_PRODUIRE
  TRANSITION bo
    LIBELLE "bouchage à la sollicitation"
    LOI INSTANTANEE ( g_bo )
    PROVOQUE sollicite <-- FAUX,
              bouchage <-- VRAI
  OU BIEN
  TRANSITION non_bo
    LOI INSTANTANEE (1-g_bo)
    PROVOQUE sollicite <-- VRAI ;

```

Les informations soulignées sont celles qui étaient implicites dans la première version de la règle.

Exemple de défaillance en fonctionnement :

Prenons l'exemple d'un composant de type disjoncteur sur lequel peut survenir l'événement 'ouverture intempestive'.

```

TYPE disjoncteur SORTE_DE composant_electrique ;
ATTRIBUT position DOMAINE 'ouvert' 'ferme'
              PAR_DEFAULT 'ferme' ;

PANNE oi ;
OCCURRENCE
ouverture_intempestive
SI position = 'ferme'
  IL_PEUT_SE_PRODUIRE
  DEFAILLANCE oi
    LIBELLE "ouverture intempestive"
    LOI EXPONENTIELLE ( l_oi ) ;

```

Cette règle est comprise par Figaro comme si on avait écrit :

```

TYPE disjoncteur SORTE_DE composant_electrique ;
ATTRIBUT position DOMAINE 'ouvert' 'ferme'
              PAR_DEFAULT 'ferme' ;

PANNE oi ;
OCCURRENCE
ouverture_intempestive
SI position = 'ferme' ET NON oi
  IL_PEUT_SE_PRODUIRE
  DEFAILLANCE oi
    LIBELLE "ouverture intempestive"

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

```
LOI EXPONENTIELLE ( l_oï )
PROVOQUE oï <-- VRAI ;
```

6.3.3.4. Indisponibilités

Les *indisponibilités* représentent des événements qui se sont produits dans le passé du système et qui se révèlent à la première sollicitation des composants. Les indisponibilités sont donc un type particulier de défaillance à la sollicitation et elles sont toujours associées à une **loi instantanée**.

Les indisponibilités seront traitées exactement comme des défaillances à la sollicitation (cf. Paragraphe précédent). Autrement dit, si dans un système un composant est amené à être sollicité plusieurs fois, et si certaines défaillances ne peuvent se produire qu'à la **première sollicitation**, le concepteur de la base de connaissances devra modéliser différemment les conditions d'occurrence des événements indisponibilités et les conditions d'occurrence des événements défaillances à la sollicitation.

6.3.3.5. Réparations

Les *réparations* représentent les changements d'état dus à la maintenance du système. Ce sont des transitions qui ont pour effet de supprimer les pannes d'un objet. Pour cela on leur associe une facette indiquant les pannes supprimées par la réparation.

```
Exemple de réparations :
TYPE disjoncteur SORTE_DE composant_electrique ;
ATTRIBUT position DOMAINE 'ouvert' 'ferme'
PAR_DEFAULT 'ouvert' ;

PANNE oï ;
PARAMETRE_LOI m_oï ;
OCCURRENCE
IL_PEUT_SE_PRODUIRE
REPARATION rep_oï
LIBELLE "réparation de l'ouverture intempestive"
REPARE oï
LOI EXPONENTIELLE ( m_oï ) ;
```

Le mot-clé REPARE est suivi de la liste des pannes supprimées par la réparation en question.

Si l'événement REPARATION est associé à un ensemble de variables PANNE (par la facette REPARE), le processus suivant s'applique :

1. Par définition, une panne ne peut être réparée que si elle a eu lieu.

Ainsi il n'est pas nécessaire d'intégrer à la condition d'occurrence de l'événement, la condition '<variable de panne> = VRAI'. Celle-ci est implicite et sera prise automatiquement en compte par les traitements.

Lorsque plusieurs pannes ont été associées à une même réparation, cette dernière ne peut avoir lieu que si toutes les pannes en question se sont produites.

2. De même, l'occurrence d'un événement réparation provoque toujours la mise à FAUX de la variable PANNE associée.

Il n'est donc pas nécessaire de décrire explicitement cette action dans les règles.

Remarque : Les événements de type REPARATION sont traités de façon particulière par le générateur d'arbres de défaillances. Si une feuille de l'arbre correspond à une PANNE associée à un événement REPARATION, alors l'événement de base de la feuille sera déclaré comme un événement de base de type "Réparable" et le paramètre loi de l'événement REPARATION sera associé à cet événement de base.

6.3.4. Les règles d'interaction

Syntaxe d'une règle d'interaction :

```
INTERACTION [nom_de_regle]
[ GROUPE liste_groupes ]
[ ETAPE nom_etape ]
[ SOIT variable UN nom_d_ensemble
[ DE_TYPE type ]
[ VERIFIANT condition ] ]
```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

```
[ SI          condition ]
ALORS action
[ SINON       action ]
```

*Remarque : **Remarque :** La syntaxe des règles d'interaction des objets est semblable à celle des règles d'interaction des types.*

Les règles d'interaction permettent de propager les conséquences d'un événement à tout le système. Autrement dit, les règles d'interaction permettent d'exprimer les **relations logiques** existant entre un objet et les autres objets du système. Cette relation logique peut s'exprimer principalement de deux façons :

- Soit en modifiant l'état d'un objet en fonction de l'état des autres objets.

TYPE pompe

```
TYPE pompe_secours SORTE_DE pompe;
ATTRIBUT
    demande_demarrage DOMAINE BOOLEEN ;
INTERFACE secourue GENRE pompe
INTERACTION
SI IL_EXISTE x UN secourue TEL_QUE PANNE(x)
ALORS demande_demarrage <-- VRAI ;
```

- Soit en modifiant l'état des autres objets en fonction de l'état d'un objet.

```
TYPE pompe
INTERFACE pompe_secours GENRE pompe ;
INTERACTION
SI PANNE
ALORS
POUR_TOUT x UNE pompe_secours FAIRE demande_demarrage(x) <-- VRAI ;
```

Une règle d'interaction permet de modifier l'état d'un objet en fonction de l'état des autres objets d'un système. En effet, il est important de pouvoir faire interagir un objet avec les objets qui sont en amont ou en aval de lui.

Les règles d'interaction définies pour un type sont introduites par le mot-clé **INTERACTION**.

Pour décrire une règle d'interaction, on dispose des facettes suivantes :

Nom de règle (facultatif)

Permet d'attribuer un nom à une règle.

Facette GROUPE (facultative)

Permet de spécifier l'appartenance de la règle à un ou plusieurs groupes (cf. Notions Avancées).

Facette ETAPE (facultative)

Permet de spécifier l'appartenance de la règle à **une** étape (cf. Notions Avancées).

Facette SI (facultative)

Décrit la condition nécessaire pour que la règle d'interaction soit déclenchable. Si aucune condition n'est associée à la règle (la règle doit se déclencher quel que soit l'état du système), la facette **SI** n'est pas mentionnée.

Facette ALORS (obligatoire)

Permet de spécifier les actions qui sont déclenchées lorsque la condition de la règle est VRAIE.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Facette SINON (facultative)

Permet de déclarer les actions qui sont déclenchées lorsque la condition de la règle est FAUSSE.

```
Exemple :
TYPE composant ;
ATTRIBUT
    relie DOMAINE BOOLEEN PAR_DEFAULT FAUX ;
INTERFACE
    montant GENRE composant ;
SI IL_EXISTE x UN montant TEL_QUE relie (x)
ALORS relie <-- VRAI
SINON relie <-- FAUX ;
```

Facette SOIT (facultative)

Comme nous l'avons vu au chapitre 2, lorsqu'on applique les règles de la base de connaissances à la base de faits, une règle définie dans un type sera examinée (on dit aussi "instanciée") pour chaque objet de ce type. Les traitements examinent si la règle est applicable et l'appliquent éventuellement.

La facette SOIT permet de définir une règle d'interaction (**condition et actions**) pour un **ensemble** d'objets défini par une variable liée (ensemble d'objets en interface ou ensemble d'objets d'un type particulier). Dans ce cas, la règle sera examinée ("instanciée") pour tous les couples d'objets définis par :

(Objet O1 du type possédant la règle, Objet O2 appartenant à l'ensemble défini par la facette SOIT)

Si l'ensemble défini par la facette SOIT est vide, la règle ne sera pas instanciée.

Prenons l'exemple d'une pompe qui a plusieurs pompes secours en interface. On veut exprimer le fait que, si la pompe tombe en panne, toutes les pompes secours qui sont à l'arrêt doivent être sollicitées. Autrement dit, on veut que pour chaque couple (pompe, pompe secours) du système la règle soit examinée. La facette SOIT permet d'écrire **une seule règle** pour modéliser ce processus :

```
TYPE pompe ;
INTERFACE
    pompe_secours GENRE pompe ;
ATTRIBUT
    etat DOMAINE 'marche' 'arrêt' 'sollicite';
PANNE
    defaillant ;
SOIT x UNE pompe_secours
SI defaillant ET etat(x) = 'arrêt'
ALORS etat(x) <-- 'sollicite' ;
```

Si un système contient une pompe P1 normale secourue par les pompes P2 et P3, cette règle sera "instanciée" deux fois :

- pour le couple (P1, P2)

```
SI defaillant(P1) ET etat(P2) = 'arrêt'
ALORS etat(P2) <-- 'sollicite' ;
```

- pour le couple (P1, P3)

```
SI defaillant(P1) ET etat(P3) = 'arrêt'
ALORS etat(P3) <-- 'sollicite' ;
```

La portée de la variable liée définie s'étend sur l'ensemble de la règle d'interaction. Ainsi, une variable définie par un SOIT pourra être utilisée dans les autres facettes de la règle (facettes SI, ALORS, SINON) si elle est décrite avant celles-ci.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

L'ensemble défini par la facette SOIT est soit un nom de champ interface (multivaluée) soit un nom de type. Comme pour les opérateurs complexes, il est possible de préciser des contraintes optionnelles (DE_TYPE et VERIFIANT) pour ne traiter qu'un sous-ensemble d'objets.

Exemple :

```
SOIT x UN <ensemble d'objets>
    VERIFIANT < conditions de restriction de l'ensemble >
    DE_TYPE < nom de type >

SOIT x UN montant
    DE_TYPE vanne
    VERIFIANT debit DE x > 2500
SI ...
```

Il est possible de définir plusieurs variables dans une même facette SOIT et d'imposer certaines contraintes entre ces variables.

```
SOIT x UN <ensemble d'objets>
    ET y UN <ensemble d'objets>
    VERIFIANT <conditions de restrictions de l'ensemble>

SOIT x UN montant
    ET y UN montant
    VERIFIANT x <> y
SI ...
```

Dans ce cas-là, Figaro instancie $card(amont) * card(amont)$ fois la règle d'interaction, où $card(amont)$ représente le nombre d'objets de l'ensemble *amont*.

Remarque : Etant donné que la portée des variables déclarées dans le SOIT s'étend sur l'ensemble de la règle, si d'autres variables de même nom sont utilisées dans d'autres facettes par des opérateurs complexes (QQSOIT, IL_EXISTE, IL_EXISTE AU_MOINS, SOMME, PRODUIT, POUR_TOUT), nous rappelons que ce sont les variables de plus petite portée (donc celle des opérateurs complexes) qui seront d'abord prises en compte. L'exemple suivant montre un cas où une surcharge de variable liée est effectuée.

```
Exemple :
TYPE pompe ;
INTERFACE
    montant GENRE vanne ;
INTERACTION
    SOIT x UNE vanne
    SI IL_EXISTE x UN montant VERIFIANT debit DE x > 2500
        (* il s'agit du x de IL_EXISTE *)
    ALORS debit DE x <-- 3000 ;
        (* il s'agit du x du SOIT *)
```

7. Notions avancées

Nous allons étudier dans cette section les particularités de Figaro qui sont d'approche un peu plus complexe et qui sont moins usitées lors de l'écriture d'une première base de connaissances.

7.1. La précompilation et les macros

Les outils Figaro, en préalable à tout traitement sur une base de connaissances, lui appliquent l'opérateur de précompilation standard des compilateurs. Grâce à cela, il est possible de définir des « macros » introduites par le mot-clé #define (mot-clé pour le précompilateur). Cela permet de gagner beaucoup de temps dans l'écriture de bases de connaissances contenant des structures répétitives telles que la déclaration de nombreuses défaillances. Une macro peut faire appel à d'autres macros, comme dans l'exemple suivant où l'on commence par définir la concaténation de deux arguments, avant de l'utiliser dans la déclaration d'une panne.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Exemple

```
(* macro de concaténation de deux chaînes de caractères *)
#define colle2(debut,fin) debut/**/fin

(* macro définissant une déf. en fct, utilisant la macro colle2 *)
#define DEFAILLANCE_FONC(nomdef,comment,param,valeur) \
    CONSTANCE param DOMAINE REEL EDITION NON VISIBLE ROLE CONCEPTION \
    PAR_DEFAULT valeur; \
    ATTRIBUT colle2(__inhibate__,nomdef) DOMAINE BOOLEEN EDITION VISIBLE, \
    MODIFIABLE PAR_DEFAULT FAUX; \
    PANNE nomdef LIBELLE comment EDITION VISIBLE, MODIFIABLE \
    PAR_DEFAULT FAUX \
    DONNEES_FIABILISTES \
    GROUPE Groupe_add \
    SI ((visu DE GLOBAL_OBJ = 'inconnu') ET (NON \
    colle2(__inhibate__,nomdef))) \
    MODELE_PAR_DEFAULT MODELE_GLM \
    MODELE_GLM GAMMA 0 LAMBDA param MU 0 MODELE_FIGE;
```

Les caractères “\” en fin de ligne servent à dire que la macro se prolonge à la ligne suivante. Ces caractères doivent être suivis **immédiatement** du caractère fin de ligne.

Exemple d'appel à la macro précédente :

```
DEFAILLANCE_FONC(DF,"perte du fluide dans %OBJET",l_df,1e-4)
```

Préalablement à tout traitement à partir de la base de connaissances, le préprocesseur sera appelé et « dépliera » la ligne d'appel à la macro en la remplaçant par le texte de la macro, lui-même modifié par le remplacement de la chaîne

- nomdef par DF,
- comment par "perte du fluide dans %OBJET"
- param par l_df
- valeur par 1e-4

Ce processus est récursif puisqu'une macro peut contenir l'appel à une autre macro.

Outre le fait de fournir des raccourcis pour des structures répétitives, les macros peuvent servir à simuler la définition de fonctions simples.

Exemple

```
#define surface(rayon)\
3.1416*rayon**2
```

Enfin, grâce aux constantes de précompilation, il est possible de définir des variantes d'une base de

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

connaissances. Dans l'exemple suivant, on choisit de rendre l'attribut *calcul* visible ou non selon la valeur de la constante de précompilation `__DEBUG__`.

Exemple

```
#ifdef __DEBUG__
    #define __VISUALISATION__ VISIBLE
#else
    #define __VISUALISATION__ NON VISIBLE
#endif
```

```
TYPE t ;
ATTRIBUT calcul EDITION __VISUALISATION__;
```

C'est au moment de charger la base de connaissances dans KB3 qu'on doit choisir la valeur à donner aux constantes de précompilation car cela va modifier le comportement des modèles saisis avec cette base de connaissances. KB3 lit ces valeurs dans le fichier de paramétrage de l'interface graphique, d'extension .bdc.

7.2. Eléments globaux

Pour modéliser une classe de systèmes, on a parfois besoin de définir des règles ou des variables attachées à la base de connaissances elle-même, donc au système complet que saisira l'utilisateur, et non à un composant particulier de ce système.

Pour permettre la définition de tels éléments, le langage Figaro permet de déclarer des objets particuliers, les OBJET_SYSTEME. Ces objets existent dans tout système décrit avec cette base de connaissances et leurs variables sont accessibles depuis tout autre objet du modèle (ils ne contiennent aucune déclaration d'interface). Les OBJET_SYSTEME sont les seuls que l'on a le droit de déclarer dans une base de connaissances.

7.3. Héritage et surcharge

Les règles de gestion des mécanismes de surcharge et d'héritage mises en oeuvre dans Figaro sont complètement explicitées en référence /2/. Dans ce manuel, nous nous contenterons d'exposer les principes de ces mécanismes et les interdits associés.

Le mécanisme de surcharge est mis en oeuvre avec les deux principes suivants :

- La surcharge (redéfinition) d'une caractéristique (ou d'une partie seulement) s'effectue sur son nom.
- Le "dernier qui parle a raison" : un type peut redéfinir localement les caractéristiques du ou des types dont il hérite. La surcharge peut également s'effectuer entre types hérités. Dans ce cas, c'est l'ordre d'apparition des types pères dans la facette `SORTE_DE` qui détermine l'ordre de prise en compte des caractéristiques et de leurs facettes.

Voici les principaux interdits associés à ce mécanisme :

- Il est interdit de modifier dans un type la nature (attribut, constante, paramètre loi, effet, panne) d'un champ défini dans un père.

```
TYPE pompe ;
ATTRIBUT
    a DOMAINE BOOLEEN ;

TYPE pompe_secours
    SORTE_DE pompe ;
CONSTANTE
    a DOMAINE BOOLEEN ; (*INTERDIT*)
```

Le champ 'a' défini initialement comme attribut est redéfini comme constante.

- Il est interdit de surcharger le domaine d'un champ :

```
TYPE pompe ;
ATTRIBUT
    a DOMAINE BOOLEEN ;
```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

```

TYPE pompe_secours
  SORTE_DE pompe ;
  ATTRIBUT
    a DOMAINE REEL ; (*INTERDIT*)

```

Le champ 'a' défini initialement comme booléen est redéfini comme réel.

Il est en revanche autorisé de surcharger un domaine énuméré (de l'étendre, de le modifier, de le restreindre, etc.). La surcharge d'un domaine énuméré comporte les restrictions suivantes :

1. Le domaine doit rester un domaine énuméré.
2. La surcharge d'un domaine énuméré est autorisée à condition qu'au moins un élément du nouveau domaine soit commun aux deux domaines.

- La surcharge d'une interface est autorisée uniquement si le type avec lequel on surcharge est un sous-type du type surchargé.

```

TYPE composant_hydraulique ;
  INTERFACE
    secours GENRE composant_hydraulique ;

TYPE pompe;
  INTERFACE
    secours GENRE pompe;

```

- Il est interdit de surcharger une règle d'interaction par une règle d'occurrence et réciproquement.
- Il est interdit de surcharger les caractéristiques d'un type dans un objet particulier instance de ce type.

7.4. Les Groupes de règles

En principe, lors de l'exploitation d'un modèle Figaro, toutes les règles d'interaction et d'occurrence appartenant à des objets du système étudié sont analysées. En fait, ce principe de base peut être raffiné par l'introduction de la notion de **groupes de règles**.

Dans une base de connaissances, il est possible de définir des **groupes** particuliers de règles à partir de l'ensemble des règles du modèle. Cette possibilité permet, lors de l'exploitation d'un modèle Figaro, d'effectuer une analyse sélective du modèle en restreignant l'ensemble des règles à analyser à un ou plusieurs groupes de règles.

Prenons par exemple le cas d'une base de connaissances destinée à être utilisée pour générer automatiquement des arbres de défaillances sur des systèmes thermohydrauliques. Dans cette base, les redondances passives entre composants ne sont pas modélisées car, pour permettre l'étude du système par arbre de défaillances, on fait l'approximation que toutes les redondances du système sont actives.

On veut ensuite utiliser cette même base de connaissances pour construire un modèle du type "graphe d'états" permettant de valider cette approximation. Pour ce faire, on va enrichir la base de connaissances initiale des règles modélisant les redondances actives entre composants et créer deux groupes de règles, le groupe "passif" et le groupe "actif".

- Toutes les règles modélisant le système sans tenir compte des redondances passives seront attribuées au groupe "actif".
- Toutes les règles modélisant les reconfigurations de type normal/secours seront attribuées au groupe "passif".

Lorsque la base de connaissances sera utilisée pour générer un modèle du type graphe d'états, on analysera uniquement les règles du groupe "passif".

Les groupes de règles de la base de connaissances sont déclarés en début de base de connaissances dans la section NOM_DES_GROUPES (**l'information recensant les identifiants des groupes utilisés dans la base est une information relative à l'ensemble de la base et non à un type particulier**).

```

NOMS_DES_GROUPES
  passif ;

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

actif ;

Il est alors possible de spécifier pour une règle d'occurrence ou pour une règle d'interaction que la règle appartient à un ou plusieurs groupes (énumération de groupes) en utilisant la facette GROUPE .

```

INTERACTION
regle_prop
GROUPE passif actif
SI IL EXISTE a UN amont TEL_QUE relie DE a
ALORS relie ;

```

L'attribution d'une règle à un groupe particulier n'est pas obligatoire. Pour l'exploitation d'un modèle Figaro, les conventions suivantes ont été adoptées :

- Si aucun nom de groupe n'est spécifié, l'ensemble des règles du modèle est analysé.
- L'utilisateur peut demander l'exploitation d'un modèle Figaro sur l'union des groupes d'une liste particulière.

Ex : analyse des groupes passif UNION actif

- Les règles n'appartenant à aucun groupe pourront être exploitées spécifiquement en demandant l'analyse du groupe identifié par SANS_NOM.

Ex : analyse des groupes SANS_NOM UNION actif

Il est très important de noter que l'organisation des règles d'une base de connaissances en groupes de règles est une étape importante de la conception de base de connaissances. Plus particulièrement, le rôle de chaque groupe de règles (y compris du groupe SANS_NOM) devra être explicité pour les utilisateurs de la base.

Il faut noter que le découpage du modèle Figaro en groupes n'est possible que pour les règles. La définition des champs valués du modèle devra donc être adaptée, quels que soient les groupes de règles analysés.

Lorsque des groupes ont été définis dans la base de connaissances, ils sont exploitables dans les bases de faits associées à cette base de connaissances. La redéfinition d'un GROUPE dans une base de faits est interdite et provoque un message d'erreur de duplication de nom de groupes. En effet, toutes les données définies dans la base de connaissances sont accessibles dans la base de faits. Il n'est donc pas nécessaire de les redéfinir.

7.5. Les étapes

Comme nous l'avons vu au chapitre 2, l'ordre dans lequel sont appliquées les règles d'interaction d'un modèle Figaro n'est absolument pas défini. Autrement dit, les bases de règles doivent être écrites de manière à ce que, quel que soit l'ordre d'application, le résultat obtenu soit le même.

Cependant, il peut arriver que l'on ait besoin d'introduire un ordre dans l'application des règles d'interaction. C'est pourquoi le langage Figaro permet de **regrouper les règles d'interaction en blocs de règles** appelés ETAPES et **d'ordonner ces étapes** afin que les règles soient examinées dans un ordre précis.

Prenons l'exemple d'une classe de systèmes thermohydrauliques pour laquelle on veut modéliser la propagation du fluide. Cette classe de systèmes contient des objets de type "composant" qui peuvent être ou non des sources de fluide. Pour modéliser la propagation du fluide, il faut modéliser deux règles :

- les objets "source" sont toujours alimentés,
- les objets de type "composant" sont alimentés s'il existe un composant amont en interface alimenté.

Pour minimiser le nombre de règles à appliquer, il est intéressant d'appliquer la première de ces deux règles en premier. C'est pourquoi on introduit deux étapes dans la base de connaissances : une étape "initialisation" et une étape "propagation".

```

ORDRE DES ETAPES
initialisation ;
propagation ;

TYPE composant ;

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

```

INTERFACE
    montant GENRE composant ;
EFFET
    alimente DOMAINE BOOLEEN ;
CONSTANTE
    source DOMAINE BOOLEEN ;

INTERACTION
    init
    ETAPE initialisation
    SI source = VRAI
    ALORS alimente ;

    prop
    ETAPE propagation
    SI IL EXISTE x UN montant TEL_QUE alimente(x)
    ALORS alimente ;

```

7.5.1. Déclaration des étapes

Une étape peut avoir un ou plusieurs identifiants déclarés sous forme d'une liste d'identifiants (par exemple les identifiants "init" et "initialisation" identifient la même étape).

Il est aussi possible d'associer une condition à une étape. Une condition associée à une étape Figaro est une expression booléenne complexe ou non. Cette condition sera évaluée lorsque les règles associées à l'étape vont être évaluées :

- Si la condition n'est pas vérifiée, aucune règle de l'étape n'est évaluée (ce qui correspond à une optimisation sensible car l'évaluation des conditions de ces règles n'est pas effectuée).
- Si la condition est vérifiée, les conditions des règles de l'étape sont évaluées. Il s'agit du processus normal de déclenchement des règles.

Remarque : Une étape sans condition est considérée comme une étape dont la condition renvoie toujours VRAI. Les règles de l'étape sont donc toujours évaluées.

- Cette spécificité a été ajoutée pour optimiser le déclenchement des règles dans les traitements de l'Atelier Figaro. La condition de l'étape correspond, en quelque sorte, à une factorisation de la partie commune des conditions des règles composant cette étape.

Les noms des étapes et leur ordre d'exécution sont déclarés au début de la base de connaissances dans la section **ORDRE_DES_ETAPES** (l'information recensant les identifiants et l'ordre des étapes utilisées dans la base est une information relative à l'ensemble de la base et non à un type particulier).

Les étapes sont ordonnées en fonction de leur ordre de définition dans la base de connaissances. Lorsque le mot clé **ORDRE_DES_ETAPES** apparaît plusieurs fois dans la base de connaissances, on considère que les étapes sont définies dans l'ordre de lecture de la base de connaissances.

Exemple 1 : Déclaration d'étapes sans condition.

```

ORDRE_DES_ETAPES
    initialisation, init;
    propagation;

```

Dans cet exemple, le numéro d'ordre de l'étape initialisation est 1 et le numéro d'ordre de l'étape propagation est 2.

Exemple 2 : Déclaration d'étapes avec condition.

```

ORDRE_DES_ETAPES
    initialisation CONDITION NON initialisation_effectuee(parametres_globaux);

TYPE parametres ;
EFFET initialisation_effectuee ;

OBJET_SYSTEME parametres_globaux EST_UN parametres ;

```

Exemple 3

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

ORDRE_DES_ETAPES

```
etape1 ;
etape2 ;
```

est équivalent à

ORDRE_DES_ETAPES

```
etape3 ;
```

ORDRE_DES_ETAPES

```
etape1 ;
etape2 ;
etape3 ;
```

La définition de nouvelles étapes est autorisée dans la base de faits. Dans ce cas, les étapes déclarées dans la base de faits sont concaténées à la suite des étapes de la base de connaissances. Leur ordre est donc toujours supérieur à celui des étapes de la base de connaissances.

La duplication dans la base de faits d'une étape déjà définie dans la base de connaissances est interdite et provoque un message d'erreur de duplication de nom d'étape. En effet, toutes les données définies dans la base de connaissances sont accessibles dans la base de faits. Il n'est donc pas nécessaire de les redéfinir.

7.5.2. Regroupement des règles par étapes

Lorsque des étapes sont déclarées, chaque règle du modèle doit **obligatoirement** être associée à **une et une seule étape**. Cette association se fait avec la facette ETAPE des règles d'interaction.

INTERACTION

```
init
ETAPE initialisation
SI source = VRAI
ALORS alimente <-- VRAI ;
```

7.5.3. Application des règles d'interaction en chaînage avant

L'introduction d'étapes dans un modèle Figaro modifie le traitement d'application des règles d'interaction en chaînage avant (cf. Figure 3).

Dans ce cas, les règles sont traitées étape par étape dans l'ordre spécifié par ORDRE_DES_ETAPES. Pour chaque étape, le traitement applique les règles de l'étape autant de fois qu'il est nécessaire pour stabiliser l'état du système avant de passer à l'application des règles de l'étape suivante. (Appliquer les règles d'une étape, c'est examiner pour chaque objet du système les règles d'interaction appartenant à l'étape traitée).

Il est important de noter que l'organisation d'une base de règles d'interaction en étapes permet de structurer la base de règles et d'optimiser les temps d'exploitation d'un modèle.

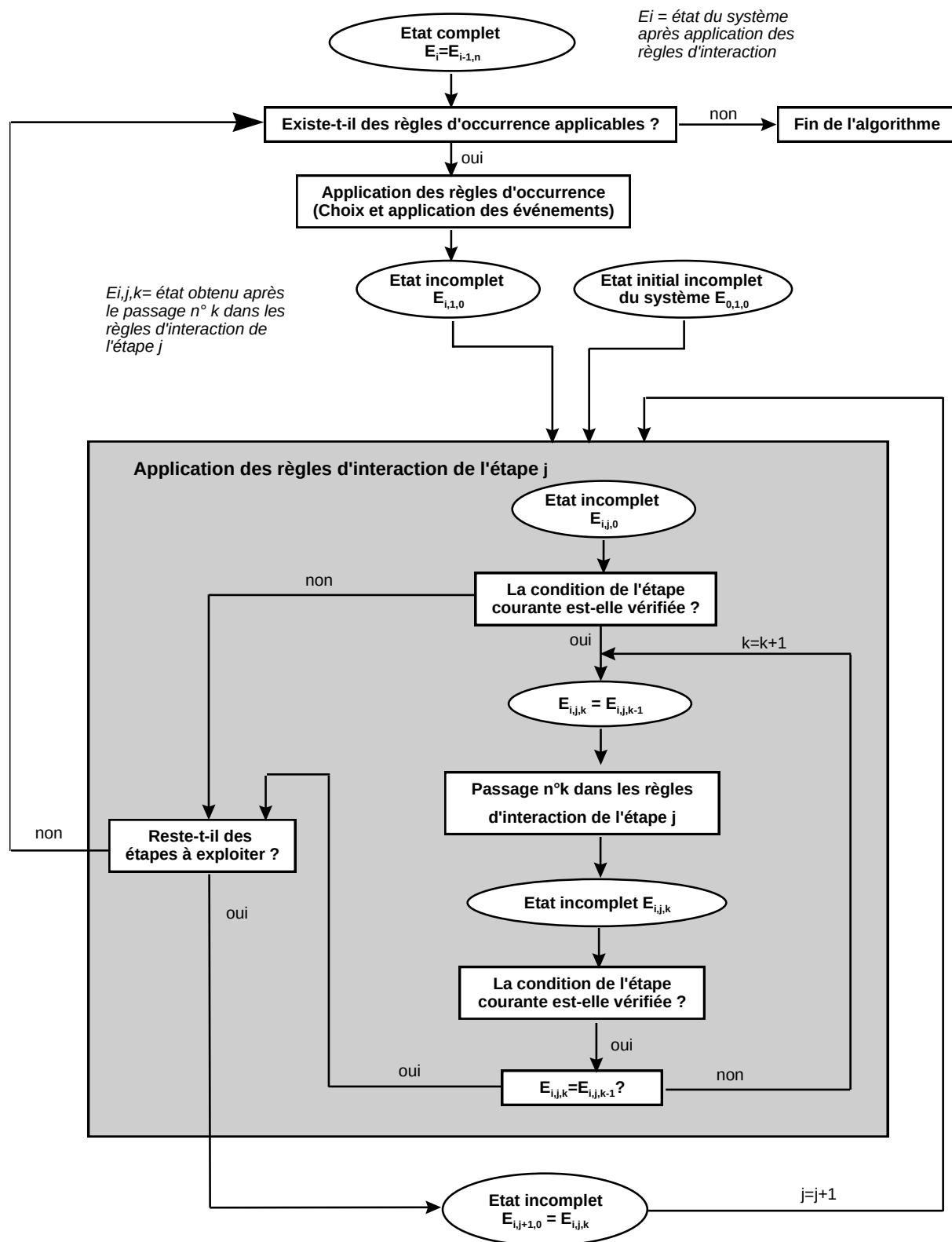


Figure 3

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

7.5.4. Application des règles d'interaction en chaînage arrière

A ce jour, les étapes ne sont pas exploitées par le traitement d'exploitation d'un modèle Figaro en chaînage arrière.

7.6. Les équations

Certains problèmes, tels que le calcul des courants et tensions dans un réseau électrique, sont difficiles à exprimer dans un formalisme en règles. Dans le meilleur des cas, on arrive à une résolution itérative par de nombreuses applications des règles, qui finissent par faire converger les attributs vers les valeurs que l'on veut calculer.

Il est, dans un tel cas, beaucoup plus élégant de poser tout simplement le système d'équations que doivent satisfaire les valeurs d'un certain nombre d'attributs, déclarés (le temps de cette résolution) comme inconnues du système d'équations.

7.6.1. Systèmes d'équations

Lorsqu'une base de connaissances nécessite l'écriture d'un ou plusieurs systèmes d'équations, chaque système doit être identifié par un nom. Les noms des systèmes d'équations utilisés dans la base sont déclarés au début de la base de connaissances dans la section NOMS_DES_SYSTEMES (l'information recensant les identifiants des systèmes utilisés dans la base est une information relative à l'ensemble de la base et non à un type particulier).

```
NOMS_DES_SYSTEMES
courant; tension;
```

7.6.2. Equations

Syntaxe:

```
EQUATION      nom_equation
[ GENRE      LINEAIRE ]
[ SYSTEME_EQUATIONS liste_systemes_equations ]
[ SI        expression ]
[ FORMULE    [ expression = expression ] ; ]
```

Les équations d'un système sont décrites dans les types de la base de connaissances (une équation appartient à un type).

Une équation ne peut appartenir qu'à un unique système d'équations.

Les équations seront définies par le type de champ **EQUATION**.

Détail des facettes :

Nom d'équation (obligatoire)

Permet d'attribuer un nom à l'équation.

Facette SYSTEME_EQUATIONS (obligatoire)

Permet de spécifier les systèmes auxquels appartient l'équation.

Facette GENRE (obligatoire)

Doit être suivi d'un mot clé qui permettra d'orienter vers une méthode de résolution adaptée au problème. Pour l'instant, seul est prévu le genre **LINEAIRE**, pour lequel des algorithmes efficaces existent.

Facette SI (facultatif)

SI est une facette optionnelle permettant d'exprimer une condition qui doit être satisfaite par l'état du modèle pour que la formule soit prise en compte dans le système d'équations.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

Facette FORMULE (*obligatoire*)

Cette facette doit être suivie de l'équation elle-même, dans laquelle les inconnues sont précédées d'un point d'interrogation '?'. Ces inconnues sont obligatoirement des attributs déclarés. La valeur de ces attributs sera calculée au moment de la résolution mathématique de la formule. Ainsi, la valeur des attributs sera modifiée. L'équation doit être écrite sous la forme de l'égalité entre un membre gauche dans lequel apparaissent les variables, au sein d'une expression numérique, et un membre droit, qui doit être une constante.

Afin de simplifier les traitements, on impose que les équations de type linéaire soient écrites sous une forme "standard", dans laquelle le membre de gauche se présente sous la forme : $a_1 * ?x_1 + a_2 * ?x_2 + \dots + a_n * ?x_n$, où les a_i sont des expressions numériques ne faisant intervenir aucune variable de l'équation.

Exemple :

```

TYPE          sommet;
ATTRIBUT
  a      DOMAINE REEL;
  b      DOMAINE REEL;
  c      DOMAINE REEL;
  x      DOMAINE REEL;
  y      DOMAINE REEL;
  isole  DOMAINE BOOLEEN;
EQUATION
  equil
  GENRE LINEAIRE
  SYSTEME EQUATIONS courant
  SI NON isole(s1) (* La formule n'a de sens que dans ce cas *)
  FORMULE a(s1)*?x(s1) + b(s1)*?y(s1) = c(s1);
              (* x et y sont les inconnues de la formule : elles vont
                 être substituées lors de la résolution de la formule.
                 x et y sont des attributs *)

```

Rappel : la résolution d'un système d'équations est déclenchée au moment voulu, par une règle d'interaction.

Exemple : déclenchement de la résolution pour le système "courant".

```

INTERACTION
  r1
  ETAPE resoudre
  ALORS RESOUDRE_SYSTEME(courant);

```

8. Cohérence des modèles

Compte tenu de la complexité de certaines bases de connaissances de KB3 (la plus complexe, celle de l'outil TOPASE dédié aux études de fiabilité des postes THT [8], comporte 27000 lignes de langage Figaro), il semble de plus en plus nécessaire de s'assurer que celles-ci réalisent bien ce que l'auteur avait prévu ou du moins de concevoir un outil qui pourrait **avertir le concepteur que sa base, indépendamment des objectifs qu'elle poursuit, possède des propriétés indésirables, que nous appellerons « incohérences »**. Ces propriétés indésirables peuvent être de plusieurs ordres : contradictions, caractère non borné de valeurs numériques, définitions bouclées (A défini à partir de B et réciproquement) que rien ne permet de résoudre, divisions par zéro, underflow ou overflow dans des calculs sur les nombres réels, etc.

L'objectif de ce chapitre est de décrire les principes de méthodes permettant soit d'assurer par construction la cohérence des bases de connaissances, soit de détecter l'existence d'incohérences. La démonstration de la cohérence, quant à elle, est impossible dans le cas général, c'est à dire pour une base de connaissances dont la construction n'a pas été menée suivant les méthodes que nous préconisons. Cela n'a rien d'étonnant, dans la mesure où le langage Figaro combine toutes les possibilités de la logique des propositions, du calcul sur les nombres réels, et des automates.

A défaut de pouvoir prouver la cohérence au niveau d'une base de connaissances, il est quelquefois possible de traiter les modèles de systèmes au cas par cas. **Cela permet d'envisager l'utilisation de bases de connaissances imparfaites**, en vérifiant pour chaque système étudié que les modèles construits ne tombent pas

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

dans les failles de la base. Un exemple typique est celui des bases de connaissances qui n'assurent la cohérence des modèles qu'à condition que la topologie des systèmes étudiés ne soit pas bouclée.

Ce chapitre est organisé comme suit : nous commençons par explorer les relations entre cohérence à l'ordre 1 et cohérence à l'ordre 0. Les relations établies nous permettent de limiter le travail de démonstration à l'ordre 0. Puis en nous appuyant sur la définition formelle de la sémantique du langage Figaro d'ordre 0 nous faisons une typologie des incohérences qu'un modèle en Figaro 0 est susceptible de contenir. Nous donnons ensuite un ensemble de règles de construction d'une base de connaissances permettant d'éviter les incohérences identifiées. Pour la plus délicate de ces règles : l'organisation des "étapes" du modèle, nous présentons les principes d'un outil qui aide l'utilisateur à faire ce travail. Pour finir, nous nous intéressons au cas des démonstrations de cohérence faisables uniquement à l'ordre 0, c'est à dire au cas par cas sur les modèles de systèmes saisis par l'utilisateur.

8.1. Relation entre incohérence à l'ordre 1 et à l'ordre 0

Démontrer (pour autant que cela soit possible) l'absence d'incohérences dans une base de connaissances revient à démontrer que :

Quel que soit le système (respectant les contraintes décrites dans la base de connaissances) saisi par l'utilisateur, le modèle en langage Figaro d'ordre 0 généré à partir de ce système et la base de connaissances ne contiendra pas d'incohérence.

Les incohérences peuvent apparaître soit dans la phase d'instanciation (transformation du modèle d'ordre 1 en modèle d'ordre 0), soit dans la phase d'exploitation du modèle (quand on l'utilise pour simuler le système modélisé). Les problèmes éventuels d'instanciation peuvent être détectés de manière **exhaustive** par les contrôles syntaxiques que nous décrivons ci-après, effectués **sur la base de connaissances**.

8.1.1. Contrôles syntaxiques

Une base de connaissances est une collection de **types** d'objets décrits en langage Figaro **d'ordre 1** (c'est à dire permettant de décrire des connaissances génériques, grâce notamment à des opérateurs adaptés à la manipulation d'**ensembles**), que l'utilisateur de la base pourra assembler d'une infinité de manières différentes au sein de modèles de systèmes.

Cette liberté est malgré tout contrainte par les indications données dans les facettes GENRE et CARDINAL des interfaces déclarées dans la base de connaissances (ex : en amont d'un composant électrique, on ne peut avoir que des composants électriques, dont le nombre est compris entre 0 et N). Grâce à cette information, la vérification syntaxique de la base de connaissances réalisée par KB3 permet d'assurer qu'aucun problème du type tentative d'accès à une variable inexistante (ex : la pression d'un composant électrique) ne peut être révélé lors de l'instanciation d'un modèle. Cette vérification syntaxique est très complexe car elle doit tenir compte de toutes les règles d'héritage entre types, et son utilité est quotidiennement vérifiée par les concepteurs de bases de connaissances.

Avant toute exploitation d'un modèle saisi par l'utilisateur, KB3 vérifie que toutes les contraintes qui ont été définies par le concepteur de base de connaissances sont respectées.

Cette vérification peut déclencher des messages d'erreur du type : "pas assez (trop) d'objets dans l'interface X de l'objet Y".

Le respect des types d'objets mis en interface d'un objet donné est assuré par l'interface de saisie, qui ne permet pas d'erreur dans ce domaine.

Ces contrôles assurent que l'opération d'instanciation à l'ordre 0 dans le cadre d'un système particulier, préalable à tout traitement fiable, se passera sans problème.

Mais bien sûr, cela ne suffit pas : cette vérification ne protège en aucune manière contre les types d'incohérences décrits dans le préambule de ce chapitre.

8.1.2. Cohérence du comportement

La définition précise de ce que l'on entend par incohérence de comportement nécessite au préalable une définition formelle de la sémantique du modèle spécifiant ce comportement.

C'est en nous appuyant sur la définition de la sémantique du langage Figaro 0 (cf. chapitre 3) que nous montrerons où les incohérences peuvent se manifester. Il sera alors possible d'énoncer des règles de construction d'une base de connaissances qui assureront la cohérence de tout modèle en langage Figaro d'ordre 0 produit à partir de cette base.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

8.2. Typologie des incohérences possibles

A partir de la formalisation du fonctionnement de "l'automate Figaro 0" (nous réutiliserons ici les notations du chapitre 3), il est possible de définir ce que l'on entend par incohérence d'un modèle. Nous définissons également des propriétés souhaitables.

Le langage Figaro d'ordre 0 peut être considéré comme une sorte de synthèse des concepts existants dans les langages d'intelligence artificielle en règles de production, et dans les réseaux de Petri stochastiques. Il est donc naturel de s'inspirer des travaux faits dans ces deux domaines pour mettre au point des méthodes de contrôle de la cohérence en langage Figaro.

Pour les réseaux de Petri, les propriétés qu'il est éventuellement possible d'établir à partir d'un réseau sont : la possibilité de revenir à l'état initial depuis tout état, le caractère borné des marquages, la vivacité des transitions (une transition donnée T est vivante si à partir de tout état atteignable à partir de l'état initial, il est toujours possible de trouver une séquence de franchissements de transitions incluant le franchissement de T), l'absence d'état absorbant, l'existence d'invariants (combinaisons linéaires des marquages de places qui sont constantes, quelles que soient les évolutions du réseau) [10],[11].

Ces propriétés des réseaux de Petri sont très faciles à transposer dans le contexte Figaro, à l'exception de l'existence d'invariants. Elles reviennent aux propriétés suivantes :

- P1 : caractère fini de l'espace des états,
- P2 : vivacité des transitions (avec la même définition que pour les RdP),
- P3 : non existence d'états absorbants,
- P4 : possibilité de revenir à l'état initial à partir de n'importe quel état.

On peut d'ores et déjà remarquer que l'absence de valeurs numériques dans V suffit à assurer que l'ensemble des états réellement atteignables par le modèle, qui est inclus dans $\mathcal{E}$, est **fini** (le fait d'avoir des variables **déduites** numériques ne remet pas en cause cette propriété).

Les deux dernières propriétés sont évidemment à rechercher uniquement lorsqu'on cherche à modéliser un système réparable, par exemple en vue de calculer sa disponibilité asymptotique.

En intelligence artificielle, on retient généralement trois critères pour valider une base de règles, quel que soit son domaine :

- la cohérence au sens de la logique : on ne peut avoir une chose et son contraire simultanément,
- la complétude : il existe une solution au problème quelles que soient les données initiales,
- la pertinence : la base est conforme à la réalité physique (ce point n'est pas traité ici).

La transposition de la notion de cohérence et de complétude à un modèle Figaro 0 demande un peu d'interprétation. En fait, dans le fonctionnement de l'automate, on peut avoir les comportements indésirables suivants :

- **Inc1** : Impossibilité de calculer $I(V_0, Y_0)$, ou bien $I(X)$, X étant l'état obtenu par application d'un ou plusieurs groupes de transitions à partir d'un état précédent. Il y a aussi le cas où le calcul est possible, mais où son résultat dépend de l'ordre des règles. Ces problèmes de complétude et de cohérence d'une base de règles ont été étudiés en 1985 à EDF. Les règles de construction sûre de bases de connaissances données dans cet article sont déduites des théorèmes démontrés dans [5].

- **Inc2** : Incohérence liée à des transitions applicables **au même instant** à partir d'un état *instantané* X donné. La signification "physique" de cette situation est le fait que plusieurs actions de durée nulle (mais au résultat aléatoire) sont lancées en parallèle et doivent donc produire leurs effets exactement en même temps. Typiquement, il peut s'agir de demandes de démarrages simultanées sur plusieurs composants. Grâce à la notion de groupes de transitions, il est inutile d'explorer tous les ordres possibles de déclenchement des transitions, mais cela suppose en général que les groupes de transitions soient indépendants les uns des autres. Indépendance signifie ici deux choses :

- l'application d'une transition ne remet pas en cause la condition de déclenchement d'une **autre** transition (en revanche, il est normal qu'elle rende fausse sa propre condition),
- quel que soit l'ordre d'application des transitions, pour une combinaison donnée de transitions prises chacune dans un groupe, on a le même résultat pour $t_1 \circ t_2 \circ \dots \circ t_k(X)$.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

L'une ou l'autre de ces conditions peut éventuellement ne pas être respectée pour des raisons bien particulières dans certains modèles, mais cela doit être un choix délibéré de l'utilisateur. C'est pourquoi tout outil exploitant le langage Figaro doit signaler à l'utilisateur le non respect d'une de ces conditions pour l'alerter sur une éventuelle faute d'étourderie.

La notion de groupe de transitions est d'un grand secours dans la modélisation parce qu'elle permet de traiter d'une manière très élégante le genre de situation suivant : dans un système ayant n composants indépendants qui doivent démarrer au même instant, il est inutile d'explorer les $n!$ séquences qui mènent au même résultat (c.-à-d. une combinaison donnée de succès et de pannes parmi les 2^n possibilités existantes). Au contraire, l'automate Figaro 0 peut produire directement ces 2^n sorties résultats de l'état instantané initial.

- **Inc3** : A ces incohérences locales peut s'ajouter un comportement indésirable plus global qui est l'enchaînement infini d'une succession de transitions instantanées, possible car cette succession de transitions ramène à un état à partir duquel elle peut à nouveau se déclencher. Il n'existe pas pour l'instant (à notre connaissance) de méthode permettant de prévoir ce type de comportement.

Il est sans doute utile de compléter les remarques ci-dessus par l'observation du fait qu'il serait **absurde** de tenter de démontrer des propriétés de cohérence amalgamant des règles d'interaction et d'occurrence. En effet, pour un système réparable, il est bien évidemment tout à fait souhaitable qu'il existe des règles qui mettent une panne à VRAI, et que dès que cela a été fait, d'autres règles permettent de mettre la même panne à FAUX. Simplement, il ne faut pas que ces opérations puissent toutes se faire en un temps nul, ce qui amènerait une situation du type décrit à l'alinéa précédent.

Nous allons maintenant donner un certain nombre de méthodes d'écriture des bases de connaissances qui permettent d'éviter **par construction** les incohérences Inc1 à Inc3, et/ou d'assurer les propriétés P1 à P4 évoquées plus haut.

8.3. Graphe de dépendances entre variables

Certaines méthodes que nous allons énoncer s'appuient sur le concept simple de graphe de dépendances entre variables. Cette notion étant importante et ayant des applications intéressantes en dehors même du contrôle de cohérence, nous en faisons une section à part entière de cet article.

Dans un modèle en Figaro 0, on dit que la variable $v1$ "influence directement la variable $v2$ " dans l'une des deux situations suivantes :

- il existe une instruction d'affectation ($\leftarrow$) où $v2$ apparaît en partie gauche et $v1$ en partie droite ($v2 \leftarrow v1$ par ex.),
- il existe une règle d'interaction ou d'occurrence dans laquelle $v1$ apparaît en prémisses, et $v2$ dans la partie gauche de la conclusion.

La meilleure façon de représenter un ensemble de relations d'influence entre des variables est d'utiliser un graphe orienté. En effet, dans ce graphe, l'existence d'un chemin entre deux noeuds associés à des variables représente une influence indirecte.

Lorsqu'on considère les variables d'un point de vue probabiliste, ces variables s'identifient à des processus stochastiques à temps continu ; on peut alors parler de dépendance stochastique entre ces processus. On peut démontrer que l'indépendance stochastique entre deux variables-processus est assurée dès lors qu'il n'existe aucun chemin entre ces deux variables dans le graphe que nous venons de définir.

Cette propriété est extrêmement utile, car elle permet de décomposer le modèle global en sous-modèles pouvant être résolus indépendamment. D'un point de vue qualitatif (pour faire du model-checking, par exemple), elle permet de remplacer la construction d'un graphe d'états global du modèle par une série de graphes indépendants beaucoup plus petits, et d'un point de vue quantitatif elle permet de simplifier le calcul de la probabilité d'être dans un état donné pour une variable d'état, en ne considérant que le sous-modèle contenant l'ensemble des variables qui influencent la variable considérée.

Ainsi donc, dans le cas où l'on souhaite calculer les probabilités d'être dans un état donné pour une variable d'état ou d'être dans différents états pour un **groupe** de variables, on peut commencer par **restreindre le modèle à l'ensemble des variables qui influencent ce groupe de variables**. Le sous-graphe qui ne contient que les variables qui influencent directement ou indirectement le groupe de variables considéré permet de définir le sous-modèle minimum à résoudre pour atteindre cet objectif.

En pratique, l'extraction d'un tel sous-modèle revient à éliminer dans le modèle global toute expression contenant une référence à une variable devant disparaître. Ainsi, les règles:

SI $v1$ OU $v2$ ALORS $v3$ SINON $v4$;

SI ($v4$ ET $v5$) OU NON $v3$ ALORS $v6$;

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

deviendront : SI NON (v1 OU v2) ALORS v4; dans le cas où on ne s'intéresse qu'à v4.

Par ailleurs, un autre type de sous-système est intéressant à distinguer : les **composantes fortement connexes du graphe** (cfc), car elles constituent des sous-modèles indivisibles pour la résolution. En effet, toutes les variables d'une cfc sont interdépendantes.

On construit alors un "super graphe" dont les noeuds correspondent chacun à une cfc. Ce super graphe est forcément sans circuit (s'il en contenait un, il existerait une cfc plus grande que celles qui ont été identifiées). Ultérieurement une technique de résolution partant des sources du super-graphe et progressant vers la cfc intéressante pourra être utilisée.

8.4. Méthodes sûres d'écriture de bases de connaissances

8.4.1. Avoir un graphe de dépendances pyramidal

Une méthode très sûre pour construire une base de connaissances est de faire en sorte de donner une structure pyramidale au graphe de dépendances entre variables, en ayant un "supergraphe" dont les cfc racines contiennent la totalité des variables essentielles. L'idéal est d'avoir le plus petit nombre possible de variables dans chaque cfc racine.

L'exemple de base de connaissances permettant de décrire un réseau de télécommunications donné ci-après est typique.

```

TYPE          composant ;
PANNE          indisponibilite ;
PARAMETRE_LOI      gamma_indispo PAR_DEFAULT 0.00005 ;
EFFET          relie LIBELLE "%OBJET connecté à une source" ;
ATTRIBUT indispo_testee DOMAINE BOOLEEN
                PAR_DEFAULT FAUX ;

OCCURRENCE
  SI NON indispo_testee
  IL_PEUT_SE_PRODUIRE
    DEFAILLANCE indisponibilite
    LOI INS ( gamma_indispo )
    PROVOQUE indispo_testee <-- VRAI
  OU_BIEN TRANSITION non_indisponibilite
    PROVOQUE indispo_testee<-- VRAI;
(*-----*)
TYPE noeud  SORTE_DE composant ;
CONSTANTE
  fonction DOMAINE 'source' 'but' 'intermediaire'
                PAR_DEFAULT 'intermediaire' ;
INTERACTION
  SI MARCHE ET fonction ='source' ALORS relie ;
(*-----*)
TYPE arete_uni_dir SORTE_DE composant ;
INTERFACE
  depart  GENRE noeud CARDINAL 1 ;
  arrivee GENRE noeud CARDINAL 1 ;

```

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

INTERACTION

SI MARCHE ET relie DE depart ALORS relie DE arrivee;

(*-----*)

TYPE arete_bi_dir SORTE_DE composant ;

INTERFACE

extremite GENRE noeud CARDINAL 2 ;

INTERACTION

SI MARCHE

ET (IL_EXISTE x UNE extremite TELLE_QUE relie DE x)

ET (QQSOIT x UNE extremite ON_A MARCHE DE x)

ALORS POUR_TOUT y UNE extremite FAIRE relie DE y

Voici un réseau simple à trois noeuds (pour simplifier, on suppose les arêtes sans défaillance) suivi du graphe des dépendances entre variables pour ce système, dans lequel les variables faisant partie de la même cfc sont incluses dans un rectangle :

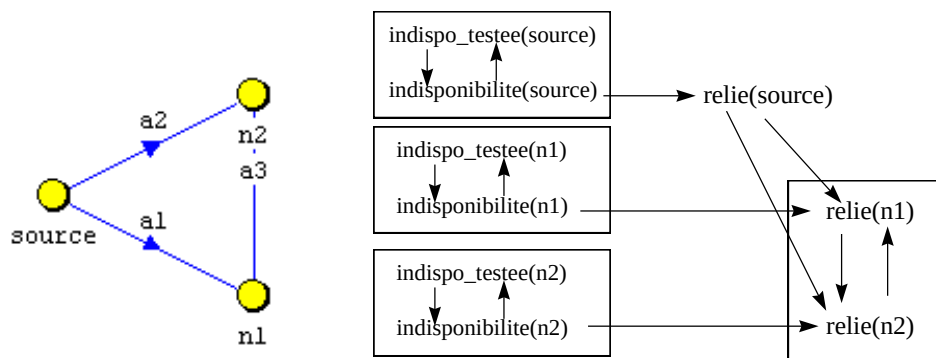


Figure 3. un réseau et le graphe de dépendances correspondant

La majorité des bases développées à ce jour génèrent des graphes de dépendances pyramidaux. En particulier, c'est le cas des bases dont l'objectif est de produire des arbres de défaillances. La plupart des variables essentielles y sont les pannes, et les pannes d'objets différents sont indépendantes entre elles.

Une façon simple et naturelle d'avoir un graphe pyramidal est de ne créer de dépendances entre variables essentielles que de manière interne aux types (donc aux objets après instanciation sur un système particulier) et de ne créer aucun lien de dépendance dans le sens variables déduites vers variables essentielles.

En donnant ce type d'architecture à une base de connaissances, on est capable de démontrer l'absence d'incohérences de types Inc2 et Inc3, ainsi que les propriétés P1 à P4 (sous réserve qu'on n'ait pas d'incohérence de type Inc1) par des raisonnements très locaux, portant sur quelques règles d'occurrence appartenant à un même type.

Ces démonstrations peuvent être faites au niveau de la base de connaissances, donc **pour tout système qui sera modélisé avec cette base.**

8.4.2. Reasonner en inférence monotone

L'objectif de la règle de construction que nous allons donner maintenant est d'éviter les incohérences de type Inc1.

Dans la référence [9] Hery et Laleuf ont démontré un théorème donnant une condition suffisante de convergence commutative d'une inférence conduite comme celle d'une étape des règles d'interaction en langage Figaro 0. Une conséquence immédiate de ce théorème est le **corollaire** suivant :

Si

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

- toutes les actions de règles sont des mises à VRAI de variables booléennes initialisées à FAUX avant l'inférence (en général, ce sont des EFFETS),
- toutes les conditions portant sur cet ensemble de variables testent si elles sont à VRAI,

Alors la convergence commutative de l'inférence est assurée.

Ce cadre très simple, mais employé dans la grande majorité des bases de connaissances dont l'objectif est de produire des arbres de défaillances est celui dit de "**l'inférence monotone**". Ce nom vient du fait que lorsqu'un booléen a été mis à VRAI, aucune règle ne permet de revenir sur cette conclusion. La deuxième condition du théorème est là pour assurer qu'aucun booléen ne sera mis à VRAI intempestivement, avant stabilisation de ceux dont il dépend.

L'énorme intérêt de ce théorème est qu'il est extrêmement facile de vérifier (même sans outil) ses hypothèses au niveau de la base de connaissances, ainsi qu'on peut le voir sur l'exemple de modélisation de réseaux de télécommunications donné dans [3]. Il est donc possible de prouver **au niveau de la base de connaissances que quel que soit l'assemblage de types réalisé par un utilisateur de cette base, aucune incohérence de type Inc1 ne pourra être créée**. Cette propriété très forte a été exploitée dans toutes les bases de connaissances opérationnelles utilisées à EDF pour les études de sûreté des centrales nucléaires.

Grâce à l'inférence monotone, il est possible de modéliser **sans risque d'incohérence et d'une manière très concise et naturelle les propagations de flux dans des systèmes bouclés**, ainsi qu'on peut le voir dans l'exemple ci-dessus du réseau de télécommunication. Or, dans la grande majorité des formalismes classiques (arbres de défaillances, logique booléenne, réseaux de Petri, formalismes de Model Checking...), ce problème de modélisation conduit soit à des impossibilités, soit à des expressions très lourdes et peu lisibles, alors qu'on le rencontre constamment dans l'étude des systèmes réels.

8.4.3. Bien exploiter les étapes

Pour éviter les incohérences de type Inc1 dans des situations où l'inférence monotone est un cadre trop restrictif, on peut utiliser les étapes.

Le théorème sur la convergence commutative de [9], plus général que l'application restreinte que nous en avons faite au paragraphe précédent, montre l'importance d'avoir au niveau d'une inférence un maximum de variables d'état dites "non réceptrices", autrement dit qui ne sont modifiées par aucune action des règles. Un moyen efficace pour diminuer le nombre de variables réceptrices est bien sûr de diminuer le nombre de règles à prendre en considération. C'est précisément ce que permet le découpage en étapes. Une fois que l'inférence correspondant à une étape donnée a été terminée, les variables qui étaient réceptrices pour cette étape peuvent (éventuellement) devenir non réceptrices pour les étapes suivantes.

Voici un exemple très simple d'application des étapes. Supposons que l'on ait besoin d'écrire une règle du type "SI NON effet1 ALORS effet2". A cause de la négation dans la condition, on sort du cadre de l'inférence monotone, **sauf** si l'on met les règles agissant sur effet1 dans une étape précédant l'étape dans laquelle on met la règle ci-dessus.

8.5. Séquencement automatique des règles d'interaction

Lors de l'écriture des règles d'interaction, il peut s'avérer difficile d'établir de manière systématique une organisation en étapes de ces règles qui assure le bon fonctionnement de l'ensemble du modèle. En général, le concepteur de base de connaissances peut établir facilement des successions locales de règles ou de groupes de règles, mais il est plus délicat de gérer l'ensemble des règles et encore plus d'ajouter une nouvelle règle dans une base de connaissances existante.

Pour diminuer ces difficultés, un outil de séquencement automatique a été mis au point. Cet outil suppose qu'une règle ne doit être utilisée que lorsque ses prémisses sont stabilisées. Dans ce cadre, l'ordre d'exécution des règles peut être directement déduit du graphe des dépendances entre règles, un concept analogue à celui de graphe de dépendances entre variables, que nous introduisons ci-après.

8.5.1. Définition des interdépendances entre règles

On définit qu'une règle R2 dépend d'une règle R1 s'il existe une variable V, modifiée par R1, qui intervient dans les prémisses ou dans les termes de calcul de R2.

Dans le cas d'un modèle Figaro 0, cette définition n'offre aucune difficulté, mais le nombre de règles et de variables considérées peut être très important et, compte tenu du fait que l'optimisation doit être recalculée à chaque modification du modèle (ajout ou suppression d'objet), le calcul est souvent trop lourd pour être repris en

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

temps réel.

La définition a donc été adaptée aux bases de connaissances en tenant compte des possibilités d'héritage entre types.

On considère donc comme distincts les éléments des différents types, même s'ils sont construits par simple héritage du type père : la variable V du type T est considérée comme différente de la variable V d'un des fils de T. On définit alors qu'une règle R2 dépend d'une règle R1 par l'intermédiaire de la variable V du type T dans le cas suivant :

- la variable V d'un type T1 père de T est modifiée par R1,
- la variable V d'un type T2 père de T intervient dans les prémisses ou dans les termes de calcul de R2.

Les types T, T1 et T2 peuvent être confondus.

8.5.2. Ordonnancement des règles

Une fois les interdépendances premières construites, il faut compléter les dépendances de manière à construire une relation d'ordre partiel ; c'est-à-dire une relation transitive et antisymétrique.

La transitivité est obtenue en ajoutant les dépendances nécessaires : si R3 dépend de R2 qui dépend de R1, on ajoute une dépendance entre R1 et R3.

Lorsque les relations d'interdépendance ajoutées assurent la transitivité de l'ensemble, il faut assurer l'antisymétrie : si R1 dépend de R2 et R2 dépend de R1 alors $R1 = R2$. Il est évident que l'on ne peut pas fusionner simplement les règles, l'antisymétrie est alors obtenue en regroupant les règles au sein d'étapes. La relation d'ordre n'est plus exprimée entre des règles mais entre des étapes : une étape E1 dépend d'une autre étape E2 si une des règles de E1 dépend d'une règle de E2. Le regroupement des règles est poursuivi jusqu'à ce que la relation entre étapes soit antisymétrique.

Le regroupement des règles violant la contrainte d'antisymétrie au sein d'une étape s'explique par le fait que ces règles dépendent les unes des autres de manière bouclée. Il est donc nécessaire de regrouper ces règles et de les exécuter ensemble jusqu'à convergence du résultat. Dans le cas extrême d'un ensemble de règles entièrement interdépendantes, l'algorithme conduit à la création d'une seule étape contenant toutes les règles.

Une fois la relation d'ordre partiel entre étapes établie, elle peut directement être utilisée pour ordonner l'exécution des étapes ; on commence par les étapes indépendantes puis on exécute les étapes ne dépendant que de celles déjà prises en compte et ainsi de suite jusqu'à épuisement des étapes.

8.5.3. Utilisation de l'outil de séquençement des règles

L'outil de séquençement peut être utilisé soit lors de la création d'une nouvelle base de connaissances, soit lors de la modification d'une base existante.

Pour une nouvelle base de connaissances, l'algorithme définit les groupes de règles interdépendantes et permet ainsi de contrôler la pertinence de ces groupes vis à vis du modèle physique.

L'algorithme peut aussi être utilisé pour compléter une définition partielle de l'ordre des règles. Le concepteur exprime certaines successions obligatoires comme par exemple le fait que le groupe de règles G1 doive être effectué avant le groupe G2 et l'algorithme complète l'ordre en signalant si les successions prédéfinies ne peuvent être respectées (cas où le groupe G1 dépend du groupe G2).

Enfin, il est possible, lorsqu'une répartition en étapes a déjà été effectuée par le concepteur, de subdiviser chaque étape de manière à obtenir l'ordre d'exécution optimal des règles, afin d'accélérer les inférences.

Lors de la modification d'une base de connaissances, le principe consiste à ajouter les nouvelles règles hors des étapes définies dans la base de connaissances et à confier à l'algorithme le soin de placer ces règles correctement dans de nouvelles étapes ou dans les étapes existantes. L'algorithme prévient lorsque les nouvelles règles introduisent des dépendances bouclées entre les étapes existantes.

8.6. Détection d'incohérences sur un modèle Figaro 0

Malgré tous les éléments que nous avons donnés jusqu'ici, il y a bien sûr des cas où rien ne peut être démontré au niveau de la base de connaissances, car les règles de construction sûre sont trop contraignantes pour permettre de modéliser certains types de systèmes. Il reste alors possible de faire des vérifications sur un modèle Figaro d'ordre 0 particulier.

Soit par exemple la base de connaissances suivante, permettant de déterminer quels noeuds sont reliés à des

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

sources dans un réseau de topologie donnée (NB : elle est complètement déterministe - l'automate peut prendre un seul état : l'état initial complet calculé à partir de l'état initial incomplet choisi par l'utilisateur) :

TYPE composant ;
 ATTRIBUT relie DOMAINE BOOLEEN PAR_DEFAULT FAUX;

TYPE noeud SORTE_DE composant ;
INTERFACE amont GENRE composant;
INTERACTION
 SI IL_EXISTE x UN amont TEL_QUE relie DE x
 ALORS relie
 SINON NON relie;

TYPE source SORTE_DE composant ;
INTERACTION
 ALORS relie ;

La variable d'état "relie" est un ATTRIBUT, et non un EFFET. Elle n'est donc pas réinitialisée avant chaque passage des règles d'interaction. Or, dans une topologie bouclée contenant deux noeuds n1 et n2, tels que n1 ait n2 dans son interface amont et réciproquement, les règles contenues dans les deux noeuds, après instanciation, se résument aux deux affectations $relie(n1) \leftarrow relie(n2)$ et $relie(n2) \leftarrow relie(n1)$. Donc, on voit que selon les valeurs initiales choisies par l'utilisateur (on suppose qu'il peut se tromper) pour $relie(n1)$ et $relie(n2)$, deux états sont stables par application des règles d'interaction : celui où n1 et n2 sont reliés, et celui où ils ne le sont pas. En fait, seul le deuxième état a un sens physique, étant donné l'absence de source dans le système.

Malgré ce défaut (qui serait corrigé tout simplement en déclarant "relie" comme un EFFET), cette base de connaissances est utilisable dans des topologies non bouclées. En effet, dans une topologie non bouclée, le graphe des dépendances entre les différents attributs "relie" du système est sans circuit. L'inférence va progressivement stabiliser les valeurs des différents attributs "relie", en partant des sources ou noeuds racines de la topologie du système, dont l'attribut "relie" est mis à VRAI pour les sources, et FAUX pour les noeuds racines, quel que soit l'état initial déclaré par l'utilisateur.

Plus généralement, il arrive souvent qu'en utilisant des règles en équivalence (SI ... ALORS ... SINON) on doive se contenter d'utiliser la base de connaissances uniquement sur des systèmes dont la topologie n'est pas bouclée.

Il est donc de loin préférable d'utiliser chaque fois que cela est possible l'inférence monotone, et ceci bien qu'elle conduise éventuellement à des bases de connaissances un peu moins lisibles, car les diverses conditions qui permettent de mettre une variable à VRAI peuvent être dispersées dans différentes règles.

Un autre exemple de détection d'incohérence possible seulement au niveau Figaro 0 et même en cours d'exécution du modèle est celui de la détection d'incohérences de type Inc2. Ce type de conflit est facile à illustrer par un réseau de Petri traduit en langage Figaro. L'examen du modèle permettrait de détecter ce que l'on appelle les conflits **structurels** du réseau, dus au fait qu'une place P est en amont de deux transitions instantanées t1 et t2 (cf. [11]). Mais il peut y en avoir beaucoup dans un modèle sans que ce soit un problème. C'est au moment de l'exécution que certains de ces conflits structurels se transforment éventuellement en conflits **effectifs** (cf. [11]). C'est le cas dans l'exemple pris ci-dessus, lorsque le tir de t1 consomme les jetons qui permettaient de valider t2 et/ou réciproquement. L'outil Figseq détecte ces conflits, avertit l'utilisateur de leur existence et affiche la séquence d'événements qui a amené le système de l'état initial à celui dans lequel le conflit est effectif.

8.7. Conclusion sur la cohérence des modèles

Nous avons montré que le langage de modélisation Figaro permet de répondre à de légitimes interrogations

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

quant à la cohérence de modèles de systèmes de plus en plus complexes.

En particulier, grâce au respect de règles simples de conception des bases de connaissances exploitées avec KB3, il est possible d'assurer **que tous les modèles qui seront construits par les utilisateurs de ces bases seront cohérents** (y compris ceux dont la topologie est bouclée).

Ces principes ont été mis en œuvre en particulier dans les bases de connaissances utilisées à EDF **pour les études probabilistes de sûreté des centrales nucléaires**. Plus généralement, **la grande majorité** des études de fiabilité ou disponibilité qui ont été réalisées à l'aide de l'outil KB3 l'ont été avec des bases respectant ces principes.

Lorsqu'il est impossible de les appliquer intégralement, comme c'est le cas pour la base très complexe (27000 lignes de langage Figaro d'ordre 1) TOPASE, il est au moins possible de s'appuyer sur des outils d'analyse des dépendances entre règles qui aident à organiser celles-ci en sous-ensembles plus faciles à maîtriser.

Enfin, certains contrôles peuvent être effectués sur les modèles de systèmes particuliers, à défaut d'être faisables au niveau d'une base de connaissances.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

9. Références

- [1] Un moyen d'unifier diverses modélisations pour les études probabilistes : le langage Figaro
Bouissou M., Bouhadana H., Bannelier M.
Note HT- 53/90/42A
- [2] Conception d'un ensemble d'outils pour la représentation, l'analyse et l'exploitation de la connaissance en sûreté de fonctionnement : l'Atelier Figaro.
Bouhadana H.
Thèse de doctorat - Juin 92
- [3] Extensions du langage Figaro 1
Humbert S., Bannelier M.
NIT T53/93/41
- [4] Définition du langage Figaro0
Bouissou M., Dia M., Ladeuil V.
Note HT - 53/93/040
- [5] Syntaxe du langage de modélisation Figaro
Bouissou M., Buffoni L., Houdebine J.C.
Juin 2016
- [6] ATELIER Figaro – spécifications du sous-système Figaro1
Humbert S., Ladeuil V., Leroy F.
Note HT - 53/95/016A
- [7] Réflexions sur la génération d'arbres de défaillance dans l'atelier Figaro
Bannelier M., Villatte N.
Note HT - 53/93/22A
- [8] M. Bulot, I. Renault, «Reliability studies for high voltage substations using a knowledge base: TOPASE project concepts and applications», Note interne EDF 96NR00101, ISSN 1161-0581, 1996.
- [9] J.F. Hery, J.C. Laleuf, «Cohérence d'une base de connaissances : la convergence commutative en langage L.R.C.», Note interne EDF HT 14/22/85, février 1985.
- [10] R. David, H. Alla, «Du Grafcet aux Réseaux de Petri», 2ème édition, Hermes, 1992.
- [11] J. L. Peterson, «Petri Net Theory and the Modelling of Systems», Prentice-Hall, Englewood Cliffs, New Jersey, 1981.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

10. ANNEXE 1 : Evolutions du langage Figaro

Dans cette annexe, nous présentons toutes les évolutions qui ont été apportées au langage depuis 1995, année de sortie de la version officielle précédente du manuel de référence.

Ces évolutions sont dues à l'élargissement de la palette des traitements disponibles, c'est pourquoi les deux parties qui constituent cette annexe sont relatives à ce qui a été ajouté pour la simulation de Monte Carlo et ce qui a été ajouté pour la génération des arbres de défaillances.

10.1. Ajouts pour la simulation de Monte Carlo

Lors d'une simulation de Monte Carlo, il est possible d'accéder pour chaque événement simulé à la date (de l'histoire simulée) à laquelle l'événement se produit. C'est ce qui a permis d'introduire le mot-clé `DATE_COURANTE` qui se comporte comme une variable globale que l'on peut consulter dans les règles. Pour la même raison il a été possible d'introduire les fonctions `TEMPS_DE_SEJOUR()` et `INTEGRALE()` dont les arguments sont des expressions booléennes.

Par ailleurs, puisque dans chaque histoire chaque variable aléatoire est échantillonnée, on a pu ajouter la fonction `RAND` (sans argument) qui produit un nombre aléatoire entre 0 et 1 ainsi que des fonctions donnant des nombres aléatoires tirés suivant différentes lois définies par leurs paramètres. La principale utilisation de ces lois est la définition des temps aléatoires avant défaillances ou réparations, mais il est aussi possible de les utiliser dans les règles.

Enfin, la fonction `DEJA_REALISE()` dont l'argument est une expression booléenne permet de faire un calcul de fiabilité sans être obligé de rendre les états de panne absorbants en les déclarant comme états cibles.

10.2. Ajouts pour la génération d'arbres de défaillances

Les modèles de fiabilité associés aux pannes ont été ajoutés de façon à permettre de spécifier simplement dans une base de connaissances un ou plusieurs modèles (avec un modèle par défaut) que l'on retrouvera dans les événements de base des arbres de défaillances générés.

Avant cet ajout, le générateur d'arbres devait en quelque sorte reconnaître à partir des règles d'occurrence si une panne donnée était à la sollicitation ou en fonctionnement, si elle était réparable ou non... Et il paraissait bien difficile d'aller au-delà de ces quelques caractéristiques simples. Maintenant, le modélisateur dispose de tout un catalogue de modèles qu'il peut associer aux pannes.

Ce catalogue a été constitué en faisant une compilation des modèles disponibles dans plusieurs outils de traitement des arbres de défaillances, et pourra être étendu si de nouvelles possibilités apparaissent dans ces outils.

10.1. Remplacement du pseudo-type GLOBAL par les objets système

L'existence du pseudo-type `GLOBAL` était une singularité par rapport à la philosophie orientée objet de Figaro. Elle répondait à une nécessité : celle de pouvoir accéder à des variables « globales » depuis tous les objets. Maintenant ce pseudo-type a disparu, mais en revanche on peut déclarer dans une base de connaissances des `OBJET_SYSTEME` (il peut y en avoir plusieurs). Ces objets existent dans tout système décrit avec cette base de connaissances et leurs variables sont accessibles depuis tout autre objet du modèle.

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

11. ANNEXE 2 : INDEX

?

? · 85

A

Action · 62
ALORS · 74
ATTRIBUT · 59
AU_MOINS ... PARMI · 47

B

Base de connaissances · 11
BOOLEEN · 57

C

CARDINAL · 15, 55
CARDINAL (opérateur) · 48
Chaînage arrière · 26
Chaînage avant · 22
Chaîne de caractères · 30
Champs · 16
Commentaire · 31
Condition de règle · 62
CONSTANTES · 56

D

DE · 37, 38
DE_TYPE · 35, 36
DEFAILLANCE · 66, 71
DES_TERMES · 48, 49
DOMAINE · 39, 56

E

EDITION · 54
EFFET · 59, 61
Ensemble d'objets · 33
ENTIER · 57
Énumération · 30, 51, 57, 80
EQUATION · 84
ET · 41
ETAPE · 74
Etat d'un modèle FIGARO · 17
Etat initial d'un modèle FIGARO · 52, 55, 57, 60
Événement · 17, 66, 67
EXP, EXPONENTIELLE · 69
Expressions · 33

F

FAIRE · 63
FAUX · 31
FORMULE · 85

G

GENRE · 55, 84
GLOBAL · 38, 78
GROUPE · 68, 74, 80

H

Héritage · 12, 51, 78

I

Identificateurs · 31
IL_EXISTE · 42
IL_EXISTE AU_MOINS · 44
IL_PEUT_SE_PRODUIRE · 68
INCLUS_DANS · 44, 46
INDISPONIBILITE · 66, 73
INFINI · 56
INS, INSTANTANEE · 69
INTERACTION · 74
INTERFACE · 13, 55

J

JUSQUA · 55

L

LIBELLE · 56, 58, 60, 69
LINEAIRE · 84
Liste · 30, 54, 63, 81
LOI · 69

M

MARCHE · 46
Modèle FIGARO · 11
MODIFIABLE · 54
Mots-clés · 32

EDF R&D	Manuel de référence du langage Figaro	Version D
------------	---------------------------------------	-----------

N

NOM_DES_GROUPEs · 79
 NOM_DES_SYSTEMES · 84
 NON · 41
 NON MODIFIABLE · 54
 NON OBLIGATOIRE · 54
 NON VISIBLE · 54

O

Objet · 10
 OBLIGATOIRE · 54
 OCCURRENCE · 68
 ON_A · 43
 ORDRE_DES_ETAPES · 81
 OU · 41
 OU_BIEN · 65, 68

P

PANNE (opérateur) · 47
 PANNE (variable d'état) · 59, 60, 66
 PAR_DEFAULT · 53
 PARAMETRE_LOI · 57
 POUR_TOUT · 48, 49, 63
 PRODUIT · 49
 PROVOQUE · 70

Q

QQSOIT · 43

R

REEL · 57

Règle · 17, 61
 Règle d'interaction · 73
 Règle d'occurrence · 64
 REINITIALISATION · 58, 60
 REPARATION · 66, 73
 REPARE · 69, 73
 RESOUDRE_SYSTEME · 64

S

SI · 68, 74
 SINON · 75
 SOIT · 75
 SOMME · 48
 SORTE_DE · 51
Surcharge · 12, 78
 Système d'équations · 64
 SYSTEME_EQUATIONS · 84

T

T_C, TEMPS_CONSTANT · 69
 TEL_QUE · 42
 TRANSITION · 66, 70
 TYPE · 10, 50

U

UN · 42, 43

V

variable liée · 37
 Variable liée · 33
 VERIFIANT · 37
 VISIBLE · 54
 VRAI · 31